Mutopia ��� Compendio T��cnico

Estructuras de datos, algoritmos y fundamentos de teoría musical en el ecosistema de 154 herramientas de composición asistida

Documento técnico generado a partir del código fuente

Septiembre 2026

Contents

Introducción 6

analyzer.py — Análisis comparativo y de trayectoria entre MIDIs 8

anticipation_composer.py — Anticipatory Music Transformer (adaptación de Stanford CRFM) 9

arc_supervisor.py — Supervisor de arco emocional para obras multi-sección 10

audio_diffusion_cli.py — Generación de audio mediante difusión sobre espectrogramas Mel 11

audio_effects.py — Librería DSP de efectos de audio (motor determinista para mix_evolver.py) 12

audio_lab.py — Síntesis aditiva, análisis espectral y edición de espectrogramas como imagen 13

audio_reference_scorer.py — Puntuación de afinidad de un WAV frente a un corpus de referencia 14

audiolm.py — AudioLM: modelado de lenguaje sobre audio en tres etapas 15

bass_line_composer.py — Línea de bajo como voz compositiva autónoma 16

cadence_designer.py — Diseñador de arquitectura de cadencias 17

chord_copilot.py — Autocompletado de progresiones por n-gramas de corpus 18

chord_pattern_voicer.py — Reescritura del patrón rítmico de la pista de armonía 19

chord_progression_generator.py — Intención musical → progresión de acordes fundamentada 20

chord_table.py — Catálogo curado de 70 progresiones con tensión y emoción 21

chordgan.py — Transferencia de estilo condicionada por chroma (ChordGAN sin TensorFlow) 22

clara.py — Generación de música con AWD-LSTM (adaptación del proyecto CLARA) 23

claude_analyzer.py — Analizador musical y emocional de MIDI (11.000 líneas) 24

climax_engineer.py — Ingeniería deliberada del punto culminante 25

cmt_composer.py — Generación de melodía condicionada por acordes (Chord-Melody Transformer) 26

La familia "combined_simplifier": reducción pedagógica de dificultad pianística 27

completer.py — Continuación inteligente de obras musicales incompletas 29

corpus_explorer.py — De la intención musical a la música, sin fricción 30

counterexample.py — Composición paramétrica por contraejemplo 31

counterpoint.py — Motor de contrapunto por especies (Fux / Gradus ad Parnassum) 32

csl_l2m.py — Generación controlable de melodía a partir de letra (CSL-L2M, AAAI-2025) 33

La familia "cycle_gan_style_transfer": transferencia de estilo bidireccional sin redes neuronales 34

cyclegan_composer.py — CycleGAN neuronal completo sobre piano rolls 35

ddsp_synth.py — Síntesis diferenciable de audio (DDSP: Differentiable Digital Signal Processing) 36

La familia "diffusion_composer": composición end-to-end mediante difusión 37

drummer.py — Motor de percusión rítmica con catálogo de arquetipos estilísticos 38

ear_trainer.py — Generador de dictados de entrenamiento auditivo 39

etude_generator.py — Generador de ejercicios técnicos de piano 40

evolver.py — Motor de evolución genética de secciones MIDI 41

expectation_engine.py — Manipulación de expectativas armónicas (Meyer/Huron) 42

La familia "fingering": digitación pianística automática (v1 → v3) y su puente a Synthesia 43

flow_composer.py — Composición mediante Normalizing Flows condicionales 45

generar_midi.py — Sistema de composición YAML + Python para una obra concreta 46

genetic_style_transfer_v3.py — Transferencia de estilo por algoritmo genético + modelo aprendido 47

La familia "genopatch": matching de patches de síntesis por búsqueda evolutiva e inferencia amortizada 48

graded_variations.py — Variaciones musicales por dimensión, a un grado pedagógico fijo 50

grammar_composer.py — Composición dirigida por gramáticas formales (modo compose + modo learn) 51

grammar_transformer.py — Transformación estructural de MIDI mediante gramática Re-Pair (3.181 líneas) **52**

harmonic_analyzer.py — Análisis armónico funcional didáctico (numerales romanos con explicación) **53**

harmonic_gnn_analyzer.py — Análisis armónico mediante grafo de notas (redes neuronales de grafos) **54**

harmony_checker.py — Corrector pedagógico de conducción de voces SATB **55**

harvester.py — Extracción de fragmentos musicales reutilizables desde MIDIs **56**

instrument_cloner.py — Clonación neuronal de timbres a partir de ~16 s de audio (DDSP) **57**

latent_composer.py — Composición end-to-end mediante VAE multi-rol **58**

latent_lab.py — Codec neuronal de audio y puentes espectrograma↔latente (3.876 líneas) **59**

latentscore_composer.py — Síntesis procedimental de audio ambiente desde texto **60**

leadsheet_realizer.py — De cifrado de acordes a acompañamiento pianístico tocable **61**

leitmotif_analyst.py — Identificación de leitmotifs y sus transformaciones **62**

leitmotif_tracker.py — Gestión, siembra y rastreo de leitmotifs en composición dirigida **63**

lyrics_melody.py — Melodía condicionada por el contenido emocional del texto (v3.1) **64**

lyrics_writer.py — Generación automática de letras para melodías ABC (LLM) **65**

melody_adapter.py — Adaptación de melodías a progresiones de acordes externas **66**

La familia “melody_conditioned”: generación de melodía condicionada a acordes (v1.1 → v5.0) **67**

melody_critic.py — Crítica constructiva pedagógica de una melodía **68**

melody_generator.py — Generación de melodías originales desde cero (“Bloque 0” melódico) **69**

melody_harmonizer.py — Adaptación de melodía a una armonía dada, con ornamentación (v2.0) **70**

meter_adapter.py — Adaptación métrica: reescritura rítmica entre compases distintos **71**

metric_analyzer.py — Análisis de estructura métrica y síncopa **72**

midi2yaml.py — Conversión de fragmentos MIDI a sintaxis de partitura.yaml **73**

midi_align.py — Alineación temporal de MIDIs mediante puntos de anclaje **74**

midi_concat.py y midi_merge.py — Composición temporal y espacial de MIDIs 75

midi_dna_unified.py — Mezclador unificado de "ADN" musical (v1.4, fusión de tres generaciones) 76

midi_pianoroll_analyzer.py — Análisis multi-pista con visualización HTML interactiva 77

midi_split_harmony.py — Separación automática de melodía y acordes 78

midi_test_generators.py — Generadores de MIDIs sintéticos multi-pista para pruebas 79

midinet_composer.py — Generación de melodía condicionada en acordes mediante GAN bar-a-bar (MidiNet) 80

mix_evolver.py — Algoritmo genético para encontrar la mejor cadena de efectos de mezcla 81

ml_architect.py — Composición estructural dirigida por gramáticas aprendidas de corpus (v8.0) 82

ml_expander.py — Expansión orquestal de bocetos pianísticos mediante ML entrenado por reducción sintética 83

ml_inpainter.py — Relleno inteligente de compases vacíos (v2.0) 84

modulator.py — Modulación continua y musicalmente informada de parámetros MIDI (v2.0) 85

morpher.py — Morphing gradual entre dos obras MIDI 86

mosaic_composer.py — Collage y ensamblaje dramático de fragmentos musicales (v2.0) 87

motif_table_v2.py — Catálogo curado de 100 motivos y frases melódicas (v2.0) 88

mscz2vec.py — Vectorización perceptual de partituras (7.472 líneas) 89

muscriptor_transcribe.py — Transcripción de audio a MIDI con transformer decoder-only (MuScriptor) 90

musegan.py — Generación multi-pista mediante GAN sobre piano rolls binarios (BinaryMuseGAN, PyTorch) 91

music_transformer.py — Generación con estructura a largo plazo mediante atención relativa (Huang et al. 2019) 92

musiclm.py — Generación de música a partir de texto (MusicLM, pipeline completo) 93

mutator.py — Mutación semántica de MIDIs guiada por lenguaje natural 94

narrator.py — Diseñador de arquitectura narrativa para obras musicales 95

openmusic.py — Text-to-Music con QA-MDT (Quality-Aware Masked Diffusion Transformer) 96

orchestral_colorist.py — Color tímbrico emocional para obras MIDI 97

orchestrator.py — Orquestación automática y percusión para mocks orquestales 98

part_extractor.py — Extracción de partes instrumentales individuales con transposición 99

performer.py — Renderizado expresivo de interpretación para MIDI cuantizado 100

phrase_builder.py — Motor de sintaxis melódica: antecedente/consecuente 101

piano_autocomplete.py — Autocompletado de interpretaciones de piano en tiempo real 102

La familia "piano_duo_analyzer": visualizador HTML de piano a dos manos con análisis armónico (v2.0 → v5) 103

piano_expander.py — Expansión orquestal de bocetos pianísticos (v2.0, línea por línea) 104

piano_reducer.py — Reducción pianística de obras orquestales 105

piano_synth.py — Síntesis física de piano y teclados, con optimización espectral (5.096 líneas) 106

piano_to_orchestra.py — Piano MIDI → orquesta completa, autónomo (v2.3, 4.007 líneas) 107

pitch_transcriber.py — Adaptación de Basic Pitch (Spotify): audio → MIDI 108

playability_auditor.py — Auditoría de tocabilidad idiomática instrumental 109

practice_tutor.py — Graduador de dificultad y generador de plan de estudio 110

preference_trainer.py — Aprendizaje de preferencias musicales personales (v1.2) 111

quality_scorer.py — Evaluación multidimensional de calidad musical (v1.2) 112

reharmonizer.py — Reharmonización guiada de melodías con progresiones alternativas (v1.1) 113

remi.py — Pop Music Transformer (REMI): generación y continuación de piano pop 114

La familia "remi_diffusion": modelo de lenguaje de difusión enmascarada sobre tokens REMI (v1 y v2) 115

rhythm_table.py — Catálogo curado de patrones rítmicos con tensión y emoción 116

runner.py — Ejecutor de pipelines de composición 117

schenker_composer.py y schenkerian_reducer.py — Elaboración generativa y reducción analítica schenkeriana 118

section_score.py — Detección de secciones musicalmente incoherentes 119

select.py — Selección interactiva de MIDIs por clustering UMAP + KMeans 120

silence_designer.py — El silencio como recurso dramático deliberado 121

song_architect.py — De frases sueltas a estructura de canción popular completa (v2.0) 122

spectro_midi.py — Transcripción de espectrogramas NPZ a MIDI (CNN + Transformer, agnóstico de instrumento) 123

stem_separator.py — Separación de fuentes de audio (HTDemucs, sin la librería demucs) 124

stitcher.py — Orquestador de fragmentos MIDI con coherencia armónica ("Coherence Stitcher") 125

style_blender.py — Fusión dinámica de dos estilos a lo largo de una obra 126

style_transfer.py — Transferencia de estilo: aplica el ADN de un MIDI a otro 127

synth_forge.py — Motor de síntesis multi-paradigma (seis técnicas clásicas en Python puro) 128

tempo_designer.py — Diseño de la arquitectura de tempo (agógica global) 129

tension_designer.py — Diseñador visual interactivo de curvas de tensión 130

theme_transformer.py — Mapeo escalar y transformación temática, incluyendo armonía negativa (v2.1) 131

theorist.py — De intención musical vaga a plan de composición fundamentado 132

tonicization_engine.py — Inserción quirúrgica de dominantes secundarios 133

tonicizer.py — Motor de modulación tonal y puentes armónicos entre tonalidades 134

La familia "unifier": unificación coherente de secciones-ancla (v1.1 → v3.0) 135

variation_engine.py — Motor de variaciones musicales automáticas (catálogo de referencia) 136

voice_aligner.py — Alineación de consonancia entre voces MIDI independientes (v2.0) 137

voice_leader.py — Motor de voice leading SATB para bloques de acordes 138

wfc_composer.py — Composición mediante Wave Function Collapse + SMT solver (v3.0) 139

yue_composer.py — Generación de canciones completas con letra mediante YuE 140

Introducción

Este documento recoge, herramienta por herramienta, un análisis técnico del ecosistema **mutopia**: una colección de 154 scripts Python de fichero único dedicados a la generación, análisis y transformación de música simbólica (principalmente MIDI). Cada entrada describe, para la herramienta correspondiente:

1. **Propósito y flujo de uso** (subcomandos, pipeline).

2. **Estructuras de datos** que maneja internamente (representaciones de eventos musicales, tensores, grafos, tokens, etc.).
3. **Algoritmos** empleados (desde heurísticas deterministas hasta arquitecturas de aprendizaje profundo).
4. **Conceptos musicales** subyacentes, explicados a nivel de una persona con formación técnica avanzada pero no necesariamente experta en teoría musical formal.

El documento sigue el orden alfabético de los ficheros fuente.

analyzer.py — Análisis comparativo y de trayectoria entre MIDIs

Propósito. `analyzer.py` responde preguntas de similitud y evolución sobre obras MIDI: comparación dimensión a dimensión entre dos piezas, búsqueda de los N vecinos más cercanos en una colección, "diff" semántico (qué tiene A que no tenga B), matriz de similitud N×N, perfil vectorial de una obra y, en su versión más reciente, la *trayectoria* interna de una pieza a lo largo de ventanas temporales y el *camino vectorial* que conecta dos obras (o una secuencia de ellas).

Estructuras de datos. El núcleo es un vector de características por MIDI, representado como `dict[str, float | list[float]]`. Existen dos niveles de vectorización: uno *fast*, calculado directamente con `mido` (densidad de notas, velocidades medias, sin dependencias externas), y uno *full*, delegado a `mscz2vec.py` mediante subproceso, que produce 17 dimensiones musicales usando `music21` para análisis armónico y de clave. Los vectores heterogéneos (algunos escalares, otros arrays multidimensionales como el embebido "unified") se normalizan con `flatten_vector` a un escalar en [0,1] para comparaciones dimensión-a-dimensión, o se expanden con `to_comparable_vector` a un vector plano (p. ej. `unified_0 … unified_5`) para comparaciones globales. Una clase `VectorCache` persiste estos vectores en disco indexados por ruta absoluta, invalidándose automáticamente si el fichero MIDI subyacente cambia (comparación de timestamp/hash), lo cual evita recalcular vectores costosos (`music21` es lento) en colecciones grandes.

Algoritmos. Las métricas de distancia implementadas son las tres clásicas del análisis de vecinos: euclídea, coseno y Manhattan, con conversión a una similitud acotada en [0,1]. El modo `nearest` es esencialmente un *k-NN* sobre el espacio vectorial de la colección. El modo `diff` no se limita a restar vectores: calcula la diferencia por dimensión y la filtra por un umbral (`--threshold`) para reportar solo los cambios "significativos", añadiendo una descripción semántica de la dirección del cambio (p. ej. "más denso", "más agudo"). El modo `trajectory` divide la obra en ventanas temporales, vectoriza cada una por separado y traza la serie temporal resultante por dimensión — una forma de análisis de series temporales aplicado a features musicales. El modo `path` interpola vectorialmente entre dos (o más) obras, generando puntos intermedios en el espacio de características; conceptualmente es un camino lineal (o compuesto por segmentos) en un espacio euclídeo de alta dimensión, útil para pensar en "morphing" estilístico entre piezas sin necesidad de un modelo generativo. Opcionalmente usa `UMAP` para proyectar estos espacios de alta dimensión a 2D con fines de visualización, una técnica de reducción de dimensionalidad no lineal basada en aproximar la variedad (manifold) subyacente mediante un grafo de vecinos difuso y optimizar una proyección de baja dimensión que preserve su topología.

Conceptos musicales. Las dimensiones vectorizadas por `mscz2vec.py` (invocado como backend "full") codifican propiedades como densidad melódica, complejidad armónica, rango de alturas, variedad rítmica y perfil de velocidades — es decir, una traducción de intuiciones de análisis musical (¿es denso?, ¿es tonalmente estable?, ¿cambia mucho de registro?) a un vector numérico comparable algebraicamente. El "diff" semántico traduce de vuelta esas diferencias vectoriales a lenguaje descriptivo, cerrando el ciclo entre número y percepción musical.

anticipation_composer.py — Anticipatory Music Transformer (adaptación de Stanford CRFM)

Propósito. Es una adaptación al estilo "fichero único" del *Anticipatory Music Transformer* de Stanford CRFM: una herramienta para tokenizar MIDI, manipular esas secuencias de tokens y generar música (desde cero, como continuación, o como acompañamiento condicionado a una melodía) usando modelos pre-entrenados descargados de HuggingFace Hub.

Estructuras de datos. El corazón del sistema son tres representaciones tokenizadas intercambiables de un MIDI: **compound tokens** (representación intermedia rica, un token por evento con todos sus atributos), **event tokens** (formato "arrival-time": tripletas tiempo-de-llegada/duración/nota) y **interarrival tokens** (formato tipo "MIDI-like": delta-tiempos entre eventos consecutivos, similar al formato usado por Music Transformer de Magenta). Las funciones `midi_to_compound`, `compound_to_events`, `events_to_compound`, `midi_to_interarrival`, etc. implementan todas las conversiones entre estas representaciones, formando un pequeño grafo de transformaciones con el MIDI como nodo de entrada/salida universal. Además existe una noción de "eventos" vs. "controles": una secuencia puede contener una mezcla intercalada de tokens que representan notas a generar y tokens que representan condicionantes ya fijados (p. ej. una melodía dada), separables con `op_split` y recombinables con `op_combine`/`op_anticipate`.

Algoritmos. El algoritmo distintivo es la **anticipación**: en vez de condicionar un modelo autorregresivo solo en el pasado (como un LLM de texto estándar), el Anticipatory Transformer intercala en la secuencia de entrada tokens de control que representan eventos *futuros* conocidos, desplazados una cantidad `delta` hacia el pasado en la secuencia — de ahí "anticipados". Esto permite que un único modelo autorregresivo funcione tanto en modo puramente generativo (autorregresivo puro, `sample_generate_ar`) como en modo condicionado-al-futuro (modo AAR: autorregresivo + anticipación, `sample_generate`), por ejemplo generando un acompañamiento que "sabe" de antemano dónde caerán las notas de la melodía. El muestreo usa **nucleus sampling** (`_nucleus`, top-p): en cada paso se trunca la distribución de probabilidad sobre el vocabulario a los tokens cuya masa acumulada supera un umbral `p`, y se muestrea solo dentro de ese subconjunto — evita tanto la degeneración determinista del *greedy decoding* como las colas de baja probabilidad que producen incoherencias.

Conceptos musicales. El formato arrival-time separa explícitamente el "cuándo ocurre" de un evento de su contenido tonal, lo cual es natural para pensar en términos de textura polifónica: varias voces (partes instrumentales, identificadas por su programa MIDI) pueden entrelazarse temporalmente. La función `extract_melody` separa instrumentos concretos (p. ej. programa 0-1 para piano) del resto de la textura, formalizando la distinción melodía/acompañamiento como una partición del conjunto de tokens por canal instrumental — la base para el modo `accompany`, que genera una textura de acompañamiento condicionada a una melodía ya compuesta.

arc_supervisor.py — Supervisor de arco emocional para obras multi-sección

Propósito. Dada una obra dividida en secciones (ya sea como ficheros MIDI separados o como un único MIDI con puntos de corte en compases), mide si la evolución emocional real de la pieza coincide con un arco planificado de antemano, diagnostica discrepancias y sugiere valores concretos para corregirlas.

Estructuras de datos. Cada sección se representa como un `SectionVector`: un punto en un espacio de cinco dimensiones emocionales — *tensión*, *actividad*, *registro*, *armonía* y *contraste* — cada una normalizada a [0,1]. El plan deseado se representa de forma paralela como `PlanVector`, que puede construirse a partir de un JSON de curvas (`tension_designer.py`), de palabras clave por sección ("calma tensión clímax") traducidas mediante un **léxico emocional** (`word_to_vector`, con interpolación por distancia de coseno cuando la palabra no está en el léxico), de una notación numérica abreviada, o de una especificación paramétrica de "arco de clímax". `SectionDiagnosis` recoge, por sección, la discrepancia entre lo real y lo planificado junto con sugerencias correctivas.

Algoritmos. Cada dimensión emocional se deriva con una heurística explícita sobre las notas extraídas del MIDI (`extract_notes`): la *tensión* armónica se calcula como la proporción de pares de notas simultáneas (que se solapan temporalmente) cuyo intervalo es disonante; la *actividad* es densidad de notas por beat ponderada por velocidad media, normalizada respecto a una referencia empírica (2 notas/beat a velocidad 90 ≈ 1.0); el *registro* es el pitch medio ponderado por duración, normalizado al rango de un piano (A0–C8); la *armonía* combina la proporción de ventanas temporales con 4 o más clases de altura simultáneas con el grado de cromatismo (fracción de las 12 clases de altura usadas). El *contraste* es una métrica derivada: la distancia entre vectores de secciones consecutivas. Para arcos de "clímax explosivo" se usa una función `climax_density_curve` no lineal: una **meseta sostenida** seguida de un **salto sigmoide abrupto** (no una rampa lineal) centrado en una posición y con una nitidez parametrizables — la misma familia de curvas que usa `melody_harmonizer.py`, permitiendo que el plan emocional y la adaptación real de la melodía compartan la misma forma funcional. El diagnóstico final compara vector real vs. plan y aplica umbrales configurables sobre contraste mínimo/máximo y saltos abruptos no planificados.

Conceptos musicales. El sistema formaliza una intuición de forma musical y retórica: toda obra de cierta duración se sostiene mediante una progresión perceptible de tensión, densidad, registro y complejidad armónica —el "arco dramático"— y los errores de composición asistida por IA con frecuencia se manifiestan como una falta de contraste entre secciones o saltos abruptos no intencionados; `arc_supervisor.py` cuantifica ambos síntomas.

audio_diffusion_cli.py — Generación de audio mediante difusión sobre espectrogramas Mel

Propósito. Interfaz de línea de comandos sobre la librería `audio-diffusion` (Robert Dargavel Smith / teticio) que genera, varía, interpola y extiende (outpainting) clips de audio usando modelos de difusión pre-entrenados en HuggingFace Hub, con variante "latente" (VAE + DDIM) más rápida.

Estructuras de datos. La representación intermedia es el **espectrograma Mel**, tratado literalmente como una imagen: el pipeline convierte el audio en una imagen de magnitud Mel, aplica el proceso de difusión sobre esa imagen, y la reconvierte a audio (mediante Griffin-Lim o un vocoder aprendido, según el modelo). El estado latente de un audio dado —el tensor de ruido gaussiano que, al ser "des-ruidado" de forma determinista, reproduce ese audio— se puede serializar a JSON con el subcomando `encode`, lo cual permite operaciones deterministas y reproducibles sobre ese punto del espacio latente.

Algoritmos. El generador base es un **modelo de difusión probabilístico (DDPM)**, opcionalmente muestreado con el esquema determinista **DDIM** (Denoising Diffusion Implicit Models), que permite invertir el proceso: dado un espectrograma-imagen ya existente, `_image_to_noise` recupera (solo si el scheduler es DDIM, precisamente porque DDIM es determinista y por tanto invertible) el tensor de ruido latente que lo generaría, habilitando ediciones tipo "variación" (`variate`, con un parámetro `strength` que controla cuánto ruido se reintroduce antes de volver a des-ruidar: 0 = copia exacta, 1 = generación libre). La **interpolación** (`interpolate`) entre dos audios se realiza mediante **slerp** (*spherical linear interpolation*) sobre sus respectivos tensores de ruido latente — la interpolación esférica es preferible a la lineal directa en espacios de alta dimensión donde los vectores de ruido gaussiano se concentran cerca de la superficie de una hiperesfera (fenómeno de concentración de la medida), evitando que los puntos intermedios de una interpolación lineal caigan en una región de baja densidad y produzcan artefactos. El modo `outpaint` extiende un audio existente preservando el inicio y/o el final como condición de contorno del proceso de des-ruidado.

Conceptos musicales. Aunque el modelo opera directamente sobre representaciones tiempo-frecuencia sin ninguna noción explícita de nota, acorde o compás, la interpolación esférica entre "personalidades tímbricas" (los distintos checkpoints, entrenados sobre corpus como *breaks* de batería) permite explorar morphing tímbrico continuo entre dos texturas sonoras — un tipo de exploración que no tiene equivalente directo en la representación simbólica (MIDI) que domina el resto del ecosistema mutopia.

audio_effects.py — Librería DSP de efectos de audio (motor determinista para mix_evolver.py)

Propósito. Aplica, a un WAV ya renderizado, una cadena de efectos de procesamiento de señal conocida de antemano: ecualizador paramétrico, compresor, reverb, limitador, delay, saturación, entre otros. No genera ni busca combinaciones — es el "motor de ejecución" determinista que utiliza `mix_evolver.py` para evaluar el *fitness* de cada cadena candidata durante su búsqueda evolutiva.

Estructuras de datos. Una cadena de efectos se representa mediante una lista ordenada de `EffectGene` (par nombre-de-efecto + parámetros), construida ya sea desde un JSON externo o desde una secuencia de flags de línea de comandos cuyo **orden de aparición se preserva explícitamente** mediante una subclase personalizada de `argparse.Action` (`_EffectFlagAction`) — una decisión de diseño deliberada, porque en una cadena de audio el orden de los efectos altera el resultado (un EQ antes de un compresor no es equivalente a aplicarlo después). Un `ManifestError` dedicado convierte cualquier estructura de manifest inválida en un fallo duro e inmediato, ya que ese manifest define el espacio de búsqueda del algoritmo genético consumidor.

Algoritmos. Cada efecto es una implementación DSP explícita y auditable: el EQ paramétrico usa un **filtro biquad peaking** con los coeficientes estándar del *RBJ Audio EQ Cookbook*; el compresor de rango dinámico usa un **envelope follower** secuencial (deliberadamente no vectorizado, pues cada muestra depende del estado del filtro en la muestra anterior) seguido de reducción de ganancia con *attack*/*release* configurables; el reverb sintetiza una **respuesta al impulso (IR)** artificial —ruido con envolvente exponencial decreciente, con semilla determinista fija por defecto para que la evaluación de fitness sea reproducible— y la convoluciona con la señal; el limitador es de tipo *brick-wall* con *lookahead*; el delay se implementa como un **filtro IIR en peine** (comb filter) con realimentación negativa, cuya estabilidad exige que el coeficiente de *feedback* se mantenga estrictamente menor que 1; la saturación aplica una función de compresión no lineal (tipo *waveshaping*) mezclada con la señal seca según un parámetro `mix`.

Conceptos musicales/de producción. La librería formaliza, con fórmulas DSP estándar de la industria del audio, las herramientas básicas de un ingeniero de mezcla: dar forma al espectro (EQ), controlar el rango dinámico (compresor/limitador), añadir espacio (reverb) y textura armónica (saturación, que añade armónicos no presentes en la señal original). El procesamiento estéreo trata cada canal de forma independiente salvo en el reverb (misma semilla en ambos canales para mantener coherencia espacial), una simplificación consciente frente a un compresor "linked" de un DAW profesional.

audio_lab.py — Síntesis aditiva, análisis espectral y edición de espectrogramas como imagen

Propósito. Conjunto de utilidades DSP de fichero único que cubre análisis espectral (FFT/STFT), síntesis aditiva de notas a partir de definiciones de timbre y envolvente, renderizado de MIDI a audio sin depender de un soundfont externo, y un flujo de edición creativa que convierte espectrogramas en imágenes PNG editables con herramientas gráficas convencionales.

Estructuras de datos. `Timbre` define un instrumento como una lista de armónicos (razón de frecuencia respecto a la fundamental + amplitud relativa), cargable desde JSON. `EnvelopeADR` codifica una envolvente de amplitud (ataque-decaimiento-release) con curvas configurables. El **espectrograma** se representa como un array NumPy de magnitudes (`frames × bins`) más un array de fases y un vector de frecuencias, serializable a `.npz`. La pieza más original del diseño es el **puente NPZ↔PNG**: el espectrograma de magnitud se normaliza y almacena como imagen en escala de grises (eje X = tiempo, eje Y = frecuencia, valor = magnitud lineal o logarítmica), con los metadatos de reconstrucción (frecuencia de muestreo, tamaño de ventana, relación de solape, escala, piso en dB) embebidos en los chunks `tEXt` del propio PNG — de modo que la imagen es autocontenida y puede editarse en GIMP o Photoshop (borrar/pintar/filtrar regiones tiempo-frecuencia) y reconvertirse a audio.

Algoritmos. Incluye tanto una **DFT ingenua** (`dft_analyse`, con complejidad $O(n^2)$, incluida explícitamente con fines didácticos) como la **FFT** de NumPy ($O(n \log n)$) para el caso de uso real. El análisis por ventanas deslizantes usa **STFT con ventana de Hann** para minimizar el *spectral leakage* (fuga espectral) que produciría una ventana rectangular. Como el pipeline PNG descarta la fase (solo conserva magnitud), la reconstrucción de audio a partir de una imagen editada requiere **estimar** una fase coherente: para ello se implementa el algoritmo de **Griffin-Lim**, un método iterativo que alterna entre el dominio tiempo y frecuencia, proyectando en cada iteración sobre la restricción de magnitud conocida y sobre la restricción de consistencia de una STFT válida, convergiendo a una estimación de fase mucho más limpia que la fase cero trivial. La síntesis de notas es **aditiva pura**: cada nota se genera como la suma ponderada de sus armónicos (senoides a frecuencias múltiplo de la fundamental, con amplitudes dadas por el `Timbre`), multiplicada por la envolvente ADR — el modelo más directo y transparente de síntesis de sonido, en contraste con FM, síntesis granular o muestreo. El *pitch-shift* se implementa por remuestreo (cambia también la duración; válido hasta unos ±7 semitonos antes de degradarse, más allá de lo cual el propio docstring recomienda WSOLA).

Conceptos musicales. El *timbre* se modela explícitamente como el perfil de amplitudes relativas de la serie de armónicos —la definición acústica estándar de timbre—, y la envolvente ADR captura la evolución temporal de la intensidad de una nota, dos de los parámetros perceptuales más básicos en síntesis de sonido.

audio_reference_scorer.py — Puntuación de afinidad de un WAV frente a un corpus de referencia

Propósito. Calcula una puntuación 0-1 de cuán "cercano" es un audio candidato a un corpus de referencia (grabaciones que le gustan al usuario), pensado como señal de recompensa para bucles de aprendizaje por refuerzo (es el cuarto `EvalProvider` del ecosistema, junto a `quality_scorer`, `preference_trainer` y evaluación humana).

Estructuras de datos. `ReferenceCorpusStats` almacena la media y la matriz de covarianza (y su inversa) del corpus de referencia en el espacio de *features* elegido. `FloorBaseline` guarda estadísticas equivalentes calculadas sobre los propios renders anteriores del sistema (no sobre el corpus objetivo), para poder separar "qué tan lejos estoy de mis referencias" de "qué tan lejos estoy comparado con mi punto de partida habitual" — distinción crítica quando el corpus de referencia es de un dominio distinto (grabaciones reales) al de los candidatos (renders de soundfont), señalada explícitamente con `--domain-tag {same,cross}`.

Algoritmos. La métrica central es la **distancia de Mahalanobis**: a diferencia de la distancia euclídea, pondera cada dimensión por la inversa de la matriz de covarianza del corpus, de modo que la distancia tiene en cuenta tanto la escala como la correlación entre *features* — dos candidatos igual de "lejos" en euclídea pueden tener Mahalanobis muy distinta si una de las direcciones de desviación coincide con la varianza natural del corpus. Hay dos *backends* de extracción de *features*: uno **espectral** (banco de filtros Mel construido a mano según la fórmula triangular estándar, aplicado a un espectrograma de magnitud STFT, sin ningún modelo entrenado) y uno **latente**, que reutiliza los tokens semánticos aprendidos por `audiolm.py` — reduciendo, aunque sin eliminar, el *domain gap* entre soundfont y grabación real, porque los tokens semánticos capturan estructura de más alto nivel que el espectro crudo. El modo `fad-batch` calcula una métrica tipo **FAD** (Fréchet Audio Distance): la distancia de Fréchet entre dos gaussianas multivariantes (candidatos vs. referencia) definidas por su media y covarianza en el espacio de *features* — la misma familia matemática que FID en visión por computador.

Conceptos musicales. La herramienta es explícita sobre una trampa metodológica común en evaluación de audio generado: si el corpus de referencia son grabaciones reales y los candidatos son renders sintéticos, la métrica termina midiendo sobre todo "¿es un soundfont o una grabación real?" en vez de "¿es buena música?" — un caso concreto de *confounding* entre la variable de interés (calidad musical) y una variable espuria correlacionada (fuente de renderizado), que el `--floor-corpus` mitiga parcialmente.

audiolm.py — AudioLM: modelado de lenguaje sobre audio en tres etapas

Propósito. Wrapper CLI sobre `audiolm-pytorch` que replica la arquitectura de AudioLM (Google Research, 2022): tratar la generación de audio como un problema de **predicción de tokens discretos**, igual que un modelo de lenguaje de texto, mediante tres transformers encadenados que operan a distintos niveles de granularidad acústica.

Estructuras de datos. El audio se tokeniza en tres niveles jerárquicos: **tokens semánticos** (extraídos con HuBERT + k-means, capturan estructura de alto nivel: prosodia, contenido, identidad del "mensaje" sonoro, con muy baja tasa de bits), **tokens acústicos gruesos** (los primeros cuantizadores residuales del códec SoundStream/EnCodec, capturan timbre general, envolvente y estructura armónica) y **tokens acústicos finos** (cuantizadores residuales restantes, capturan detalle de alta frecuencia). Cada nivel corresponde a un modelo distinto: `SemanticTransformer`, `CoarseTransformer` y `FineTransformer`.

Algoritmos. La pieza tecnológica subyacente es la **cuantización vectorial residual (RVQ)** del códec de audio neuronal (SoundStream/EnCodec): en vez de un único libro de códigos, se aplican sucesivos cuantizadores, cada uno codificando el residuo (error) que dejó el anterior, lo que permite representar audio continuo como una secuencia corta de índices discretos con fidelidad ajustable según cuántos niveles residuales se usen. El entrenamiento es **secuencial y desacoplado**: primero se entrena el códec (`train-codec`), después el `SemanticTransformer` (que no necesita el códec, solo tokens HuBERT), después el `CoarseTransformer` (condicionado en tokens semánticos, predice tokens acústicos gruesos) y finalmente el `FineTransformer` (condicionado en los tokens gruesos ya generados, refina los cuantizadores finos). La generación (`cmd_generate`) encadena los tres modelos de forma autorregresiva: semántico → grueso → fino → decodificación de audio, con soporte opcional de condicionamiento textual (si los modelos se entrenaron con `--has-condition`) o de *audio prompting* (continuar un fragmento de referencia dado, `cmd_continue`).

Conceptos musicales. La jerarquía semántico/grueso/fino es una forma de **separar contenido y estilo/textura** en audio de manera análoga a cómo un pentagrama separa altura y duración de una nota del timbre del instrumento que la toca: los tokens semánticos codifican "qué se está diciendo/tocando" con independencia casi total del timbre, mientras que los niveles acústicos progresivamente más finos añaden la textura sonora concreta — permitiendo, en teoría, generar la misma estructura semántica con distintos timbres, o viceversa.

bass_line_composer.py — Línea de bajo como voz compositiva autónoma

Propósito. A diferencia de las líneas de bajo generadas como subproducto de otras herramientas del ecosistema, este módulo trata el bajo como una **voz independiente** con su propio contrapunto contra la melodía, arco expresivo y lógica estilística, ofreciendo nueve estilos (walking jazz, clásico, barroco, tumbao latino, blues shuffle, pop, pedal, ostinato, y una detección "auto" del estilo más apropiado).

Estructuras de datos. `ChordEvent` representa un acorde con su posición temporal y sus tonos constituyentes; `BassNote` una nota individual del bajo con su clase de altura derivada. Cada estilo se implementa como una subclase de `BassStyleGenerator` (patrón *Strategy*): `WalkingBassGenerator`, `ClassicalBassGenerator`, `BaroqueBassGenerator`, `LatinBassGenerator`, `BluesBassGenerator`, `PopBassGenerator`, `PedalBassGenerator`, cada una implementando su propio método `generate` sobre la secuencia de `ChordEvent`.

Algoritmos. El diseño cataloga explícitamente diez "técnicas compositivas" reutilizables entre estilos: notas de aproximación cromáticas y diatónicas (semitono o grado de escala hacia la raíz del siguiente acorde), *enclosure* (aproximación simultánea desde arriba y abajo), arpegio descendente raíz→7ª→5ª→3ª, notas de paso escalares, anticipación rítmica (atacar la siguiente raíz en el último octavo del compás), repetición rítmica, salto de octava con resolución, pedal armónico y contrapunto melódico por movimiento contrario contra la voz superior. Cada generador de estilo combina un subconjunto de estas técnicas según reglas deterministas condicionadas en el grado de tensión armónica del momento (`tension_at`) — por ejemplo, el walking bass jazzístico usa la 3ª o 5ª en el segundo tiempo y decide entre 5ª/7ª en el tercero según la tensión, y siempre resuelve con una *approach note* hacia la raíz siguiente en el cuarto tiempo.

Conceptos musicales. El fichero es, en esencia, una codificación explícita de convenciones estilísticas de acompañamiento: el walking bass de jazz (negra por tiempo, aproximaciones cromáticas), el bajo Alberti clásico, la realización de bajo continuo barroco (movimiento conjunto, preparación-resolución de disonancias), el tumbao cubano (anticipación sincopada característica del son/salsa) y el patrón shuffle de blues (root-5th-6th-b7th con *swing*). Cada uno de estos patrones es, matemáticamente, una función determinista o semi-aleatoria de la secuencia de acordes de entrada a una secuencia de eventos de nota con reglas de selección de grado y ritmo específicas del idioma estilístico correspondiente.

cadence_designer.py — Diseñador de arquitectura de cadencias

Propósito. Diseña, desde cero o sobre un esqueleto de obra, la ubicación, tipo y fuerza de cada **cadencia** (punto de cierre armónico) a lo largo de una pieza, garantizando coherencia narrativa mediante una jerarquía de cierres donde el último es el más fuerte.

Estructuras de datos. `Cadence` almacena tipo, posición, fuerza; `CadencePlan` es la colección ordenada de cadencias para toda la obra, exportable a JSON. Se maneja un catálogo cerrado de doce tipos de cadencia (auténtica perfecta/imperfecta/engañosa/interrumpida, plagal y plagal menor, semicadencia, semicadencia frigia, cadencia evadida, cadencia con 6/4 ornamental, engaño extendido y cadencia picarda), cada uno definido por su progresión armónica característica y su "fuerza" relativa de cierre.

Algoritmos. `plan_cadences` combina una **curva de tensión** por compás (generada mediante perfiles predefinidos —arco, crescendo, decrescendo, onda, clímax tardío— o proporcionada explícitamente) con la **forma** de la obra (ABA, sonata, verso-estribillo, etc.) para determinar cuántas cadencias colocar y en qué posiciones (`get_cadence_positions`), asignando a cada posición el tipo de cadencia más adecuado según su tensión local, si es la cadencia final, y el modo tonal (`select_cadence_for_position`). Las operaciones `strengthen_cadences` y `weaken_cadences` recorren el plan aplicando una transformación monótona (escalonar la fuerza hacia el final, o introducir más cadencias engañosas/semicadencias) parametrizada por una intensidad. El módulo también puede operar en sentido inverso: `analyze_existing_cadences` extrae acordes de un MIDI real por ventanas temporales, estima la tonalidad, y usa `detect_cadence_pattern` para reconocer, en pares de acordes consecutivos, cuál de los doce patrones catalogados coincide.

Conceptos musicales. El documento formaliza la jerarquía retórica de las cadencias tonales: la auténtica perfecta (V→I con ambos acordes en estado fundamental) como el cierre más definitivo; la plagal (IV→I, el "Amén" eclesiástico) como cierre suave; la semicadencia (que termina en V) como final "en suspenso"; la engañosa (V→vi) como sorpresa retórica que pospone el cierre; y variantes cromáticas como la cadencia frigia (descenso cromático característico del barroco) o la picarda (resolución mayor de una pieza en modo menor, un artificio retórico de "luz al final"). Diseñar el mapa de cadencias de una obra equivale a diseñar su puntuación retórica: dónde respira la frase, dónde se sorprende el oyente y dónde se cierra definitivamente el discurso.

chord_copilot.py — Autocompletado de progresiones por n-gramas de corpus

Propósito. Copiloto de progresiones al estilo Hookpad Aria/ChordChord: dado un contexto reciente de acordes, sugiere el acorde más probable a continuación, entrenado estadísticamente sobre un corpus MIDI real mediante modelos de **n-gramas**, en contraste con `chord_progression_generator.py`, que genera progresiones completas desde reglas teóricas hardcodeadas.

Estructuras de datos. El modelo entrenado es esencialmente una tabla de frecuencias condicionales: para cada contexto de longitud *n-1* (orden 2 o 3, es decir, bigramas o trigramas de numerales romanos relativos a la tonalidad detectada), se almacena la distribución empírica del acorde siguiente, serializada como JSON. Los acordes se normalizan a **numerales romanos relativos a la tonalidad** antes de contar, lo que permite que el modelo generalice entre tonalidades distintas del corpus (una progresión en Do mayor y su transposición en Sol mayor cuentan como el mismo patrón).

Algoritmos. La detección de tonalidad usa el algoritmo de **Krumhansl-Schmuckler**, implementado de forma autocontenida (correlación entre un histograma de clases de altura ponderado por duración y perfiles tonales empíricos de mayor/menor obtenidos de estudios de percepción tonal), con una vía alternativa opcional a través de `music21`. La estimación de probabilidades usa **suavizado add-k** (Laplace generalizado, parámetro `--k`, por defecto 0.5): sin suavizado, cualquier transición no vista en el corpus de entrenamiento tendría probabilidad cero, lo cual es indeseable en un modelo generativo con datos escasos; add-k redistribuye una pequeña masa de probabilidad hacia todas las transiciones posibles. El módulo implementa también su propio reconocimiento de **dominantes secundarios**: dado un acorde, `secondary_dominant_target` determina si su estructura (tríada mayor o séptima de dominante "inesperada" para su grado diatónico) sugiere que tonicize otro grado, y en ese caso lo etiqueta como V/x en la notación de numerales.

Conceptos musicales. El sistema formaliza la intuición de que la "gramática" de las progresiones de acordes de un estilo musical concreto (pop, jazz, etc.) es aprendible empíricamente a partir de ejemplos reales, sin necesidad de codificar reglas de teoría armónica explícitas — un enfoque de modelado del lenguaje musical (n-gramas sobre "palabras" armónicas) directamente análogo a los primeros modelos de lenguaje natural basados en n-gramas de palabras.

chord_pattern_voicer.py — Reescritura del patrón rítmico de la pista de armonía

Propósito. Detecta automáticamente cuál de las pistas de un MIDI multi-pista es la "pista de armonía" (la más polifónica) y reescribe únicamente esa pista, redistribuyendo temporalmente las notas de cada acorde según un patrón configurable — de bloques simultáneos a arpegios, patrones tipo "bombo-caja", bajo Alberti, o patrones a medida — dejando intactas el resto de pistas (melodía, bajo, batería).

Estructuras de datos. Cada acorde detectado se representa como un grupo de notas con onset cercano (`group_chords`, agrupación por ventana temporal), ordenadas de grave a agudo. Un **DSL textual ligero** define los patrones a medida: tokens de rol (`bajo`, `agudo`, `medio`, `resto`, `todas`), índices numéricos (incluyendo negativos, al estilo de indexación de listas de Python) y combinaciones simultáneas entre corchetes (`[0,1]` o `[bajo,medio]`), parseadas con un tokenizador consciente de anidamiento (`split_top_level`) que evita partir un grupo simultáneo al separar por comas.

Algoritmos. La detección de la pista de armonía usa una métrica de **polifonía**: se agrupan las notas de cada pista por proximidad de onset y se calcula el tamaño medio de esos grupos (`polyphony_score`) — una pista de acordes tendrá sistemáticamente varias notas por onset, mientras que una melodía tendrá mayoritariamente grupos de tamaño 1; la pista con mayor polifonía media (excluyendo el canal de batería) se elige como pista de armonía. El sistema de **segmentos por rango de compases** (`--segments`) permite aplicar patrones distintos a distintas secciones de la obra mediante una especificación textual `INICIO-FIN:pattern=NOMBRE,reps=N,...`, resuelta compás a compás con `resolve_segment_for_bar` (el primer segmento cuyo rango cubre ese compás, o los valores globales por defecto).

Conceptos musicales. El módulo formaliza la noción de **figuración de acompañamiento**: el mismo contenido armónico (el mismo conjunto de notas de un acorde) puede "tocarse" de maneras rítmicamente muy distintas — en bloque (himno), en arpegio (textura pianística romántica), en patrón Alberti (clasicismo vienés) o en "bombo-caja" (imitación de batería popular) — sin alterar en absoluto la armonía subyacente, ilustrando la independencia entre contenido armónico y figuración rítmica.

chord_progression_generator.py — Intención musical → progresión de acordes fundamentada

Propósito. Genera progresiones de acordes desde cero, a partir de una descripción en lenguaje natural ("melancólico y tenso"), de parámetros explícitos (tonalidad, modo, estilo, complejidad), o de planes producidos por otras herramientas del ecosistema (`theorist.py`, `tension_designer.py`) — construyendo la progresión desde reglas teóricas puras, sin requerir ningún MIDI de referencia previo.

Estructuras de datos. Internamente, una progresión es una lista ordenada de numerales romanos con duración en beats (un "patrón"), que se resuelve a acordes concretos (`resolve_numeral`) mediante un mapeo numeral→(nombre de acorde, clases de altura), parametrizado por la tónica y por un nivel de `complexity` que decide si añadir extensiones (séptimas, novenas). Existe un `SEMANTIC_MAP` que traduce palabras del lenguaje natural (léxico emocional/estilístico) a parámetros musicales concretos, aplicado de forma **aditiva** sobre una descripción libre.

Algoritmos. El flujo interno es un pequeño pipeline de cinco etapas: (1) asignación de **función armónica** (Tónica/Subdominante/Dominante/predominante-Alterado, T/S/D/AD) a cada compás según la curva de tensión objetivo; (2) selección de un *pool* de patrones candidatos del catálogo, filtrado por estilo y modo; (3) resolución de numerales a acordes concretos; (4) **scoring**: cada patrón candidato se puntúa por su adecuación a la curva de tensión deseada y por variedad (`score_pattern`), seleccionándose los N mejores (`select_best_patterns`); (5) exportación a texto, JSON o MIDI de acordes en bloque. Los **dominantes secundarios** (V/ii, V/vi, V/V…) se resuelven automáticamente calculando la quinta superior del grado tonicizado y añadiendo la séptima de dominante correspondiente, con función armónica propia (predominante-Alterado, tensión fija 0.85) — formalizando la técnica de tonicización temporal sin modulación real.

Conceptos musicales. El sistema codifica ocho idiomas armónicos distintos (diatónico, barroco, jazz, modal, romántico, impresionista, pop, flamenco) cada uno como un catálogo de patrones característicos de esa práctica compositiva: la funcionalidad barroca I-V-vi-iii-IV-I-V, el ii-V-I de jazz con tensiones extendidas y sustitución tritonal, el movimiento modal que evita deliberadamente cadencias V-I, las mediantes cromáticas y préstamos modales del romanticismo, el paralelismo de acordes en tonos enteros del impresionismo, los "loops" repetitivos de cuatro acordes del pop, y la cadencia andaluza (modo frigio dominante) del flamenco.

chord_table.py — Catálogo curado de 70 progresiones con tensión y emoción

Propósito. Complementa a `chord_progression_generator.py`: en lugar de construir progresiones algorítmicamente, ofrece un **catálogo curado y estático** de 70 progresiones etiquetadas con nivel de tensión (escala 1-10) y descriptor emocional explícito, buscable por estilo, modo, emoción, rango de tensión o texto libre, y exportable a texto/JSON/MIDI.

Estructuras de datos. Cada entrada del catálogo es un patrón de numerales romanos con metadatos (estilo, modo, tensión, emoción) y duración por acorde. Un parser de numerales genérico (`parse_generic_numeral`) va más allá de un diccionario fijo, reconociendo alteraciones (bVI, #iv), grado diatónico y una amplia variedad de sufijos de calidad (7, M7, dim, dim7, semidisminuido ø7, aumentado, sus2/sus4, 9, 6, 13, b9). Además incluye un **parser MIDI nativo sin dependencias externas** (`_parse_midi_raw`), que interpreta directamente el formato binario estándar MIDI incluyendo *running status* (optimización del formato que omite el byte de estado si es igual al del mensaje anterior) y meta-mensajes.

Algoritmos. `resolve_entry` expande un patrón catalogado a acordes concretos en una tónica dada, con dos modos de repetición: ajuste al número de compases solicitado (pudiendo truncar el último ciclo) o repetición exacta de la progresión completa un número de veces dado (`--reps`, sin truncar). `midi_to_chord_sequence` invierte el proceso: lee un MIDI real y reconstruye una secuencia de acordes limpia a partir de los eventos de nota, permitiendo comparar progresiones reales contra el catálogo.

Conceptos musicales. El valor distintivo de esta herramienta frente al generador algorítmico es la **etiqueta emocional explícita**: cada progresión del catálogo asocia una estructura armónica concreta (p. ej. i-VI-III-VII en modo eolio) con un descriptor perceptual (melancolía, angustia, euforia, exotismo…), codificando de forma curada —no derivada automáticamente— la asociación empírica entre ciertas configuraciones armónicas y su efecto emocional convencional en la práctica compositiva occidental y de otras tradiciones (modo frigio dominante para exotismo flamenco, por ejemplo).

chordgan.py — Transferencia de estilo condicionada por chroma (ChordGAN sin TensorFlow)

Propósito. Reimplementación, sin TensorFlow y sin arquitectura GAN real, del paper *ChordGAN* (Lu & Dubnov, UCSD 2021): transfiere el "estilo" pianístico de un corpus MIDI de referencia a un MIDI nuevo, condicionando la generación en el contenido armónico (chroma) del MIDI de entrada, de modo que la armonía original se preserva pero la textura/figuración se sustituye por la aprendida del corpus.

Estructuras de datos. La representación central es un **piano roll binario extendido**: 78 notas activas (MIDI 24-101, C1 a D#7) más un segundo bloque de 78 dimensiones de "onset" (detección de ataques de nota), dando un vector de 156 dimensiones por paso temporal, segmentado en ventanas de `num_steps` pasos a una frecuencia de muestreo fija (8 pasos/segundo). El **chroma** (vector de 12 dimensiones, una por clase de altura, acumulando la presencia de esa clase en todas las octavas) se extrae de este piano roll y sirve como condicionante.

Algoritmos. El generador original basado en GAN se sustituye por un **regresor condicional** (Ridge o k-NN, seleccionable) que aprende directamente el mapeo chroma→piano-roll sobre pares extraídos del corpus de entrenamiento (`_build_mapper`, `fit`). Se añade una **pérdida de ciclo** (*cycle consistency loss*, F(G(x)) ≈ x) mediante un segundo mapeador inverso roll→chroma, siguiendo la misma idea de CycleGAN que en `cycle_gan_style_transfer.py`: el objetivo es que, tras generar un roll a partir de un chroma y volver a extraer su chroma, se recupere aproximadamente el chroma original, lo cual penaliza transformaciones que destruyan la armonía original en vez de limitarse a reestilizar la figuración.

Conceptos musicales. La idea central es la separación entre **contenido armónico** (qué acordes/notas están sonando, capturado por el chroma, invariante a la octava) y **estilo de ejecución/figuración** (cómo se distribuyen esas notas en el registro y el tiempo, capturado por el piano roll completo): transferir estilo equivale a mantener fijo el primero y sustituir el segundo por el patrón aprendido de un corpus distinto, un principio de separación de "qué" y "cómo" análogo al usado en `chordgan.py` para audio y en otras herramientas de transferencia de estilo del ecosistema.

clara.py — Generación de música con AWD-LSTM (adaptación del proyecto CLARA)

Propósito. Adaptación del proyecto CLARA (Christine McLeavey et al.) a fichero único: entrena una red recurrente **AWD-LSTM** (PyTorch + FastAI) sobre un corpus MIDI codificado como texto, y genera piezas nuevas; incluye además un clasificador "crítico" (real vs. generado) y un clasificador de compositores, junto con un servidor HTTP para reproducir y explorar los resultados.

Estructuras de datos. El paso más distintivo del pipeline es la **codificación de MIDI como texto**: cada pieza se convierte a una secuencia de tokens tipo "palabra" en uno de dos esquemas — *chordwise* (un token por paso temporal de muestreo fijo, representando el conjunto de notas activas en ese instante, con marcas especiales para silencio y para "nota sostenida") o *notewise* (una secuencia evento por evento, nota a nota, con tokens de duración de espera entre eventos). Este texto se procesa después con la infraestructura estándar de modelado de lenguaje de FastAI (tokenizador, vocabulario, *language model databunch*).

Algoritmos. El modelo generador es un **AWD-LSTM** (ASGD Weight-Dropped LSTM, Merity et al. 2017): una LSTM con varias formas de regularización específicas para modelado de lenguaje — *DropConnect* sobre los pesos recurrentes en vez de sobre las activaciones, *embedding dropout*, *weight tying* entre la capa de entrada y salida, y optimización con promedio de pesos vía NT-ASGD — que en su momento fue estado del arte en modelado de lenguaje de texto antes de la era Transformer, aquí reutilizado para modelar secuencias de "palabras musicales". La generación (`_generate_batch`) es autorregresiva, muestreando la distribución de salida de la LSTM paso a paso, con un parámetro de frecuencia de aleatorización (`rand_freq`) que controla cuánto se aparta el muestreo del máximo de probabilidad. El "crítico" y el clasificador de compositores son clasificadores supervisados estándar entrenados sobre las mismas representaciones textuales, útiles como señal de evaluación automática de calidad y de estilo.

Conceptos musicales. La codificación *chordwise* vs. *notewise* refleja dos formas distintas de "leer" tiempo musical como texto: la primera es fiel a una rejilla temporal fija (como una partitura cuantizada en corcheas), la segunda es fiel a la secuencia real de eventos (más cercana a un MIDI sin cuantizar), con distintas ventajas para el aprendizaje del modelo de lenguaje según la regularidad rítmica del corpus.

claude_analyzer.py — Analizador musical y emocional de MIDI (11.000 líneas)

Propósito. El fichero más extenso del ecosistema: un analizador exhaustivo de un único MIDI que combina detección de tonalidad, de acordes, de cadencias, de tensión, de expectativa melódica, de conducción de voces y de estructura formal en un único informe textual (con representación ASCII de curvas), producido sección a sección de la obra.

Estructuras de datos. `NoteEvent`, `ChordEvent`, `TensionPoint` y `Section` (con propiedades derivadas como duración, densidad, velocidad media, pitch medio, rango de alturas y perfil de chroma) forman el modelo de datos central. El análisis armónico construye además un **grafo de transiciones armónicas** (`build_harmonic_graph`): nodos = acordes distintos observados, aristas ponderadas por frecuencia de transición — una representación de la "gramática" armónica de la obra concreta como cadena de Markov de primer orden implícita.

Algoritmos. La detección de tonalidad usa **Krumhansl-Schmuckler**, extendida más allá de mayor/menor a los siete modos griegos (dórico, frigio, lidio, mixolidio, locrio), correlacionando el histograma de clases de altura de la obra con perfiles empíricos de cada modo. La **curva de tensión** nota a nota combina siete factores ponderados: disonancia armónica entre notas simultáneas, tensión del acorde activo (según su función tonal), presencia de sensible, distancia de registro, densidad local, dinámica (velocidad) y distancia armónica a la tónica (centricidad tonal) — una síntesis multi-factorial de indicadores de tensión perceptual bien establecidos en la literatura de cognición musical. El análisis de **expectativa melódica** implementa una versión simplificada de la teoría de implicación-realización de Narmour (*Implication-Realization model*), combinada con el principio gestáltico de "buena continuación" (intervalos pequeños son más esperados que saltos) y con refuerzo por repetición de patrones, calculando una "sorpresa" por nota como proxy computacional de intensidad emocional percibida. El análisis de **conducción de voces** detecta explícitamente quintas y octavas paralelas (la infracción compositiva más clásica del contrapunto estricto), movimiento cromático, saltos grandes y resolución de la sensible. `detect_pedal_points` identifica notas pedal (una voz sostenida mientras la armonía cambia por encima) y las clasifica semánticamente según si el pedal es de tónica (estabilidad) o de dominante (tensión urgente).

Conceptos musicales. El fichero es, en efecto, una codificación computacional muy completa de un curso de análisis auditivo tradicional: tonalidad y modo, funciones armónicas (tónica/subdominante/dominante), cadencias, tensión-distensión, conducción de voces según reglas de contrapunto, y expectativa melódica según teorías cognitivas de la música — todo ello derivado automáticamente de los datos crudos de un MIDI, sin anotación manual previa.

climax_engineer.py — Ingeniería deliberada del punto culminante

Propósito. Mientras `arc_supervisor.py` **detecta** si el arco emocional de una obra se cumple, `climax_engineer.py` lo **construye** activamente: localiza (por defecto según la **proporción áurea**, posición 0.618) dónde debe estar el clímax de la pieza, remodela todas las dimensiones de energía hacia ese punto, vacía dramáticamente los compases previos, refuerza el pico con duplicaciones a la octava, y atenúa clímax secundarios que puedan robarle protagonismo al principal.

Estructuras de datos. Trabaja directamente sobre los eventos de un `MidiFile` de `mido`, calculando por compás una **curva de energía** compuesta (densidad de notas + dinámica + registro + polifonía), suavizada, y comparándola con una **curva objetivo** paramétrica (un único pico asimétrico centrado en la posición y anchura deseadas).

Algoritmos. El pipeline tiene seis etapas explícitas: (1) cálculo de la curva de energía real por compás; (2) construcción de la curva objetivo (pico único, asimétrico, con anchura configurable); (3) reescalado de las velocidades de nota por compás para acercar la curva real a la objetivo, con un límite máximo de cambio por paso; (4) "vaciado" — reducción deliberada de dinámica y adelgazamiento de textura en los compases inmediatamente anteriores al pico, técnica retórica de crear expectación por contraste; (5) refuerzo del pico mediante duplicación a la octava de la línea melódica en la ventana alrededor del clímax; (6) supresión de "falsos clímax" — picos secundarios de energía que superen un umbral relativo al pico principal se atenúan explícitamente para no competir con él. La posición áurea ($\varphi \approx 0.618$) por defecto no es arbitraria: replica la observación empírica, común en análisis de forma musical clásica y romántica, de que los clímax retóricamente efectivos tienden a situarse cerca de la sección áurea de la obra en vez de en su punto medio exacto.

Conceptos musicales. Formaliza la técnica retórica del "vacío antes del clímax" (reducir dinámica y densidad justo antes del punto culminante para que el contraste posterior se perciba con más fuerza) y la del refuerzo octava-doblada de la melodía en el clímax, dos recursos orquestales y pianísticos estándar para maximizar el impacto perceptual de un punto culminante.

cmt_composer.py — Generación de melodía condicionada por acordes (Chord-Melody Transformer)

Propósito. Adaptación de una arquitectura de generación de melodía condicionada armónicamente: dado una progresión de acordes, genera una línea melódica coherente mediante un pipeline de tres modelos encadenados — codificador de acordes, decodificador rítmico y decodificador de altura — entrenado en dos fases sobre un corpus de MIDIs orquestales convertidos a formato de dos pistas (melodía + acordes).

Estructuras de datos. La representación de entrenamiento separa explícitamente **ritmo** y **altura** como dos secuencias paralelas condicionadas en la misma codificación de acordes, serializadas como instancias `.pkl`. Un MIDI orquestal se reduce primero (`convert`) a solo dos pistas — melodía (detectada automáticamente o especificada manualmente) y acordes (con un número configurable de voces) — antes de tokenizarse.

Algoritmos. La arquitectura es un encoder-decoder de tres etapas: un **Chord Encoder** basado en BiLSTM procesa la secuencia de acordes bidireccionalmente (capturando contexto armónico pasado y futuro para cada punto), cuya salida condiciona dos decodificadores Transformer **causales** (autorregresivos, con máscara que impide atender al futuro) en cascada: primero el **Rhythm Decoder** genera la secuencia de duraciones/onsets, y después el **Pitch Decoder**, condicionado tanto en los acordes como en el ritmo ya generado, decide las alturas concretas. Ambos decodificadores usan **atención relativa multi-cabeza** (en vez de codificación posicional absoluta), una variante que representa las posiciones relativas entre tokens directamente en el cálculo de atención — más adecuada para música, donde los patrones rítmicos y melódicos se repiten con independencia de su posición absoluta en la pieza. El entrenamiento en dos fases (primero ritmo, luego altura condicionada en el ritmo ya fijado) descompone el problema conjunto en dos subproblemas más tratables.

Conceptos musicales. La separación ritmo/altura como dos “capas” independientes del proceso compositivo (decidir primero *cuándo* suenan las notas, después *qué* notas son) refleja una intuición pedagógica común en composición: la escritura rítmica y la melódica pueden trabajarse como dimensiones semi-independientes de la línea musical.

La familia “combined_simplifier”: reducción pedagógica de dificultad pianística

Cinco variantes del ecosistema abordan el mismo problema —convertir una partitura de piano en versiones progresivamente más fáciles de tocar (grados pedagógicos 1 a 8), preservando su identidad armónica y melódica— con distintos grados de sofisticación. Se documentan juntas porque comparten el modelo de datos (`MidiEvent`, `MidiTrackData`, `MidiData`) y el núcleo conceptual, difiriendo en la estrategia de reducción.

combined_simplifier.py (v1.1/v1.2) — motor estructural de base

Algoritmo central. Separa la textura pianística en voces de melodía y bajo mediante un algoritmo de tipo **skyline** (en cada instante, la nota más aguda define la melodía y la más grave el bajo), detecta la armonía por compás, y clasifica cada nota individual según su **función tonal no-armónica**: tono del acorde, nota de paso, nota de bordado (*neighbor tone*), apoyatura, suspensión, anticipación, nota escapada — la taxonomía estándar del contrapunto tonal para clasificar disonancias en relación a su preparación y resolución. Cada nota recibe un **peso estructural** derivado de esta clasificación. A partir de esos pesos se calcula un “suelo” de notas protegidas mediante **reducción schenkeriana progresiva** (niveles jerárquicos de importancia estructural, en el espíritu del análisis de Heinrich Schenker, donde la superficie musical se reduce progresivamente a su esqueleto armónico-melódico fundamental) — las notas del suelo nunca se eliminan. Sobre las notas no protegidas, un algoritmo **greedy** retira, compás a compás, las menos importantes hasta alcanzar el grado de dificultad objetivo, medido con una fórmula de cinco factores: velocidad de notas por segundo, extensión de mano, polifonía simultánea, saltos melódicos e independencia rítmica entre manos. Como mecanismo de garantía, si el suelo estructural es demasiado denso (piezas con muchos acordes en bloque) para alcanzar el grado pedido solo con el greedy, se activa una **rejilla de respaldo**: cuantización progresiva de onsets a rejillas temporales cada vez más gruesas, limitando la polifonía por mano (agudo-primero en la derecha, grave-primero en la izquierda), aplicada peldaño a peldaño solo hasta alcanzar el grado deseado, minimizando la distancia con el original.

combined_simplifier_transformed.py (v1.2) — optimizador adaptativo de transformaciones

Extiende el motor original con una capa de **transformaciones que no eliminan notas**: fusión de notas repetidas contiguas (reduce ataques percibidos sin perder contenido armónico), cuantización rítmica suave respetando el suelo protegido, alineación de manos (desplazar onsets de la mano izquierda hacia los de la derecha para reducir independencia rítmica), reducción de extensión de acordes y de saltos melódicos (moviendo notas una octava cuando esto reduce el *span*), arpegiado de acordes densos, y recorte de solapamientos (`_trim_overlaps`) como red de seguridad tras cualquier técnica que desplace onsets. A diferencia del v1.1, el bucle es de **aceptación adaptativa**: cada técnica se prueba y solo se acepta si de verdad mejora el grado medido, iterando varias pasadas — un optimizador local tipo *hill-climbing* sobre el espacio de transformaciones válidas.

combined_simplifier_hybrid.py y combined_simplifier_hybrid_v2.py (v3.0) — síntesis validada empíricamente

Fusionan lo mejor de las dos líneas anteriores según un informe de comparación empírica explícito en el propio código: conservan la sustitución armónica genuina de v2.0 (`simplify_harmony/clarify_bass`, que cambia alturas sin nunca reducir el número de notas), el bucle de aceptación adaptativa de v1.2 (en vez de la escalera fija de v2.0), las funciones `align_hands` y `merge_repeated_notes` de v1.2, `trim_overlaps` como red de seguridad, y el

hallazgo empírico de que la rejilla de respaldo rinde mejor si arranca desde el snapshot estructural original en vez de desde el ya transformado. Deliberadamente **excluyen** `_simplify_chords` (recorte de voces por posición) por considerarla una eliminación de notas disfrazada de técnica armónica, sustituyéndola por el adelgazado de *voicings* del motor estructural (`--max-voices-per-chord`), que decide qué nota eliminar por criterio funcional (peso estructural) en vez de puramente posicional. La v2 introduce mejoras incrementales de robustez sobre la v1 (misma arquitectura).

combined_simplifier_transformed_2.py (v1.1, variante intermedia)

Un punto intermedio en la evolución del proyecto: ya incluye la taxonomía completa de seis categorías de técnicas (estructural, rejilla, más las transformaciones sin eliminación) documentada en las versiones posteriores, pero antes de la consolidación final en la línea "hybrid".

Conceptos musicales comunes a toda la familia. El problema pedagógico de "arreglos fáciles" (*easy arrangements*) de piano se formaliza como optimización con restricciones: minimizar una métrica objetiva de dificultad motriz (velocidad de ejecución, extensión de mano, polifonía, saltos, independencia entre manos) sujeta a la restricción implícita de preservar el "esqueleto" schenkeriano de la obra — es decir, tratando la reducción schenkeriana no como una herramienta puramente analítica sino como una restricción generativa que garantiza identidad musical mínima durante la simplificación.

completer.py — Continuación inteligente de obras musicales incompletas

Propósito. Dado un fragmento MIDI (boceto, intro, frase suelta), genera una continuación coherente que respeta el "ADN musical" del original — tonalidad, ritmo, contorno melódico, curvas emocionales, densidad, registro y posición narrativa dentro de una forma mayor. Actúa como orquestador de alto nivel que delega en otras herramientas del ecosistema según la estrategia elegida.

Estructuras de datos. `SourceDNA` encapsula el perfil extraído del fragmento fuente (delegando en `midi_dna_unified`), sirviendo de especificación compartida para todas las estrategias. Cinco modos de completado (`extend`, `fill` — rellenar un hueco entre dos fragmentos dados A→?→B, `develop`, `recapitulate`, `coda`) determinan qué relación temporal y formal tiene la continuación con el material fuente.

Algoritmos. Ofrece ocho estrategias intercambiables: cadena de **Markov** entrenada sobre el propio fragmento (fidelidad estilística orgánica, sin generalizar más allá de las transiciones observadas), extrapolación de **contorno melódico** (sigue la "inercia" direccional de la melodía), síntesis de la **sintaxis antecedente/consecuente** (delegando en `phrase_builder`, aplica la gramática clásica de frase pregunta-respuesta), generación y encadenamiento de **variaciones** del material existente (`variation_engine`), búsqueda de fragmentos compatibles en una colección externa y su ensamblaje coherente (`harvester + stitcher`), inferencia de la posición narrativa dentro de un arco dramático mayor (`narrator`), **evolución genética** guiada por similitud al fragmento fuente (`evolver`, para resultados más sorprendentes/creativos), y un modo `blend` que ejecuta varias estrategias en paralelo y selecciona la mejor según un score combinado de coherencia, compatibilidad estilística y calidad musical intrínseca. Esta arquitectura de "meta-herramienta" es representativa de un patrón recurrente en mutopia: muchos ficheros de alto nivel no reimplementan algoritmos sino que orquestan combinaciones de las piezas ya construidas por otros scripts del ecosistema.

Conceptos musicales. El fichero formaliza la idea de que "continuar" una obra musical no es una única operación bien definida sino una familia de operaciones retóricamente distintas (extender linealmente, rellenar un hueco, desarrollar temáticamente, recapitular con variación, o cerrar con una coda) que corresponden a secciones formales distintas de la forma sonata y de otras formas clásicas.

corpus_explorer.py — De la intención musical a la música, sin fricción

Propósito. Flujo de trabajo en cinco pasos, cada uno independiente, que parte de un corpus MIDI sin etiquetar y termina en audio reproducible: diagnóstico de salud del corpus, indexado vectorial, y pasos posteriores de búsqueda semántica y generación (usando la API de Anthropic para interpretar intenciones en lenguaje natural, a juzgar por la dependencia `anthropic`).

Estructuras de datos. El **índice vectorial** del corpus se construye vectorizando cada MIDI a 10 dimensiones musicales calculadas sin dependencias pesadas: centro y rango de alturas, salto melódico medio (*interval_mean*), contorno, densidad, varianza rítmica, polifonía, media y varianza de velocidad, y proporción de silencio. Este índice se serializa como `.npz` (array NumPy comprimido), diseñado explícitamente para escalar a corpus de 100.000 MIDIs en 1-3 horas de procesamiento único.

Algoritmos. El paso `health` realiza un diagnóstico estadístico sobre una muestra del corpus (por defecto 500 ficheros): porcentaje de archivos utilizables (parseables sin error), histograma de distribución de duraciones, distribución de número de pistas/tempo/densidad, y una estimación del tiempo total de indexado extrapolada de la muestra — un patrón estándar de ingeniería de datos (muestreo para estimar costes de un pipeline sobre el conjunto completo antes de ejecutarlo). El paso `index` es el equivalente, para el dominio simbólico, de construir un índice de *embeddings* para búsqueda semántica: una vez vectorizado, el corpus completo puede consultarse por similitud vectorial sin volver a tocar los archivos MIDI originales.

Conceptos musicales. Las 10 dimensiones elegidas (más ligeras que las 17 de `mscz2vec`) representan un compromiso deliberado entre expresividad musical y coste computacional, pensado para operar a la escala de corpus muy grandes donde extraer *features* con `music21` completo sería prohibitivamente lento.

counterexample.py — Composición paramétrica por contraejemplo

Propósito. Dado un MIDI "deseado" (lo que le gusta al usuario), un MIDI "rechazado" (lo que no le gusta) y un MIDI de referencia (material genético de base), calcula el vector diferencial musical entre deseo y rechazo y lo usa como función de *fitness* dirigida en un algoritmo genético, seleccionando candidatos que se acercan al deseado y se alejan del rechazado.

Estructuras de datos. Los tres MIDIs de entrada se vectorizan a un espacio de **17 dimensiones perceptuales** compartido con `analyzer.py/mscz2vec.py`: altura media y su desviación, rango melódico, salto medio y su variabilidad, densidad, variedad rítmica, ratio de síncopa, dinámica media y su variabilidad, tensión armónica media y su pico, proporción de contorno ascendente, proporción de saltos grandes (>5 semitonos), número de pistas, riqueza instrumental y una dimensión de "clímax" adicional.

Algoritmos. El núcleo matemático es sencillo pero efectivo: se calcula el **vector diferencial** Δ = vector(deseado) − vector(rechazado), que apunta en la dirección del espacio de características que separa lo que gusta de lo que no gusta; este vector se **proyecta** sobre el MIDI de referencia (usando la referencia como punto de partida del espacio de búsqueda) y define una **función de fitness dirigida**: `fitness = similitud(candidato, deseado) − λ·similitud(candidato, rechazado) + α·score_musical_intrínseco(candidato).` `CounterexampleEvolver` extiende el algoritmo genético genérico de `evolver.py` inyectando esta función de fitness personalizada, de modo que la búsqueda evolutiva converge hacia la región del espacio vectorial que maximiza la afinidad al ejemplo positivo y minimiza la afinidad al negativo, en vez de optimizar un único objetivo absoluto.

Conceptos musicales. Este es un ejemplo directo de **aprendizaje por preferencias con dos ejemplos** aplicado a música simbólica: en vez de que el usuario describa explícitamente qué características musicales quiere (densidad, tensión, etc.), basta con mostrar un ejemplo positivo y uno negativo, y el sistema infiere la dirección relevante del espacio de características automáticamente — una alternativa ligera a los modelos de *preference learning* basados en aprendizaje profundo usados en otras partes del ecosistema (`preference_trainer.py`).

counterpoint.py — Motor de contrapunto por especies (Fux / Gradus ad Parnassum)

Propósito. Dado un MIDI de melodía (cantus firmus), genera una segunda voz de contrapunto aplicando las reglas clásicas de **contrapunto por especies** del tratado *Gradus ad Parnassum* de Johann Joseph Fux (1725), el texto pedagógico canónico de la disciplina desde hace tres siglos. Funciona tanto en modo *standalone* como módulo importado por `variation_engine`, `orchestrator` y otros.

Estructuras de datos. `KeyInfo` es un envoltorio ligero sobre la detección de tonalidad de `music21` (con *fallback* propio si no está disponible), exponiendo tónica, modo y las clases de altura de la escala diatónica. `CounterpointStrictError` señala que no existe ninguna solución válida bajo las reglas estrictas activas — un fallo explícito en vez de degradar silenciosamente la calidad del resultado.

Algoritmos. La primera especie (nota contra nota, 1:1) se resuelve mediante **backtracking tipo CSP** (*Constraint Satisfaction Problem*): en el modo `--strict`, las seis reglas clásicas de Fux (H1: solo consonancias verticales; H2: prohibición de quintas justas u octavas paralelas consecutivas; H3: prohibición de movimiento directo hacia una consonancia perfecta; H4: prohibición de cruce de voces; H5: no repetir el mismo intervalo armónico tres veces seguidas; H6: la sensible de la voz de contrapunto debe resolver ascendiendo a la tónica en la cadencia) se convierten en **restricciones duras** que el algoritmo de backtracking nunca viola — si no existe solución válida, se lanza una excepción explícita en vez de generar un contrapunto defectuoso. En el modo por defecto (no estricto), las mismas reglas actúan como **penalizaciones blandas** en una función de coste, y el motor elige el candidato "menos malo" entre los generados, pudiendo aceptar paralelas si ningún candidato limpio existe. Las especies 2 a 5 (nota-contra-dos-notas, nota-contra-cuatro-notas, síncopas/retardos, y contrapunto florido mixto) se resuelven con un algoritmo **greedy** que valida las reglas duras únicamente sobre el "esqueleto" de tiempos fuertes (downbeats) de cada compás, descartando la especie completa con aviso si ese esqueleto ya rompe alguna regla dura.

Conceptos musicales. El fichero es una implementación algorítmica directa y bastante fiel del sistema pedagógico de Fux: consonancia/disonancia como restricción vertical, movimiento paralelo/directo/oblicuo/contrario como restricción horizontal entre voces, y la jerarquía de las cinco especies como niveles progresivos de complejidad rítmica sobre el mismo cantus firmus — el punto de partida histórico de toda la teoría de la conducción de voces occidental.

csl_l2m.py — Generación controlable de melodía a partir de letra (CSL-L2M, AAAI-2025)

Propósito. Adaptación al estilo de fichero único del sistema *CSL-L2M* (Controllable Song-Level Lyric-to-Melody, AAAI-2025): genera melodía condicionada tanto en la letra de una canción como en catorce controles musicales explícitos por frase (altura, duración, ritmo, tonalidad, emoción, estructura), usando una arquitectura encoder-decoder Transformer sobre una representación tipo **REMI-Aligned**.

Estructuras de datos. El formato **REMI** (REvamped MIDI-derived events, del paper Pop Music Transformer) tokeniza el MIDI en eventos discretos (posición dentro del compás, duración, altura, tempo) en vez de una rejilla temporal fija; la variante "Aligned" sincroniza además cada frase melódica con su correspondiente segmento de letra. `VectorQuantizeEMA` implementa cuantización vectorial con actualización por **media móvil exponencial (EMA)** de los centroides del libro de códigos — la técnica estándar de VQ-VAE (Van den Oord et al. 2017) para evitar el colapso del libro de códigos durante el entrenamiento, actualizando los centroides con las asignaciones del batch en vez de mediante gradiente directo (que sería no diferenciable a través de la operación de cuantización discreta).

Algoritmos. El encoder Transformer procesa la secuencia de letra; el decoder Transformer, autorregresivo, genera la secuencia de eventos melódicos condicionado tanto en la codificación de la letra (vía atención cruzada) como en los 14 controles explícitos por frase. Un módulo **VQ-VAE** independiente (con `TransformerEncoder` propio) se entrena para extraer *features* melódicas latentes discretas — una representación comprimida y aprendida del "estilo" melódico de cada frase, útil tanto como control adicional en la generación como para análisis. El pipeline completo (`prepare` → `vocab` → `splits` → `attributes` → `train-vqvae` → `extract-feats` → `train` → `generate`) refleja el patrón estándar de un proyecto de modelado de secuencias serio: preprocesado, construcción de vocabulario, partición de datos, extracción de atributos estadísticos y de *features* aprendidas, y solo entonces entrenamiento del modelo condicional completo.

Conceptos musicales. El condicionamiento explícito en 14 controles musicales por frase (en vez de dejar que el modelo infiera implícitamente la relación letra-melodía) formaliza intuiciones de composición de canciones: la prosodia del texto (acentuación silábica) debe alinearse con el ritmo melódico, y ciertas emociones textuales correlacionan con ciertos perfiles melódicos y armónicos — controles que un compositor de canciones humano aplicaría de forma más o menos consciente.

La familia “cycle_gan_style_transfer”: transferencia de estilo bidireccional sin redes neuronales

Dos versiones (`cycle_gan_style_transfer.py` v2.0 y `cycle_gan_style_transfer_v3.py` v3.1) implementan el marco conceptual de **CycleGAN** (Zhu et al. 2017) — transferencia de estilo entre dos dominios sin pares de ejemplos correspondientes— pero sustituyendo las redes neuronales generativas y discriminativas originales por modelos de aprendizaje automático clásico (Ridge/KNN, distancia de Mahalanobis), en contraste con `cyclegan_composer.py`, que sí usa una arquitectura neuronal completa.

Concepto CycleGAN. El marco original aprende dos mapeos G: A→B y F: B→A junto con dos discriminadores D_A y D_B, sujetos a una **pérdida de ciclo** (*cycle-consistency loss*): F(G(a)) ≈ a y G(F(b)) ≈ b. Esta restricción es lo que permite entrenar sin pares emparejados (a diferencia de un *pix2pix*): basta con tener ejemplos sueltos de los dominios A y B, y la pérdida de ciclo obliga a que la transformación sea (aproximadamente) invertible, evitando que el mapeo colapse toda la información del dominio A en un único punto arbitrario del dominio B.

Adaptación sin redes neuronales. Aquí, G y F son regresores **Ridge o k-NN** entrenados sobre representaciones de piano roll (aplanadas y opcionalmente reducidas con **PCA**), y los discriminadores D_A/D_B se sustituyen por la **distancia de Mahalanobis** al centroide estadístico de cada dominio en el espacio de piano roll — cuanto más lejos del centroide del dominio objetivo, “menos convincente” es la transferencia, jugando el mismo papel que la salida de un discriminador neuronal pero sin necesidad de entrenamiento adversarial.

Diferencia v2→v3. La v2 usa un vector de 22 estadísticas globales (“ADN comprimido”) como espacio de *features*; la v3 usa el **piano roll aplanado** (multi-rol: melodía, contrapunto, acompañamiento, bajo, percusión, cada uno como matriz compás×resolución×altura), preservando estructura temporal concreta en vez de promedios globales — a costa de mayor dimensionalidad, mitigada opcionalmente con PCA. La v3.1 corrige siete errores identificados empíricamente en v3.0, entre ellos: asumir compás 4/4 fijo en el cálculo de ticks (rompía piezas en 3/4), ausencia de comprobación de disponibilidad del ecosistema de “ADN” unificado antes de usarlo, un umbral de percentil mal calibrado para el caso de *rolls* binarizados frente a continuos, carga no perezosa de todo el dataset en memoria (agotaba RAM con corpus grandes, corregido con `np.load` como gestor de contexto), y una búsqueda de vecinos más cercanos con complejidad $O(n^2)$ en Python puro sustituida por la versión vectorizada de scikit-learn.

Conceptos musicales. La segmentación del piano roll en **roles** (melodía, contrapunto, acompañamiento, bajo, percusión) antes de aplanarlo a vector formaliza la idea de que el “estilo” de un género musical no es un patrón único sino una combinación de patrones específicos por función textural — el bajo de un tango y el bajo de un jazz difieren de forma distinta a como difieren sus respectivas melodías, y tratarlos como subespacios separados permite una transferencia de estilo más fiel a cada rol.

cyclegan_composer.py — CycleGAN neuronal completo sobre piano rolls

Propósito. A diferencia de la familia anterior, esta es una implementación neuronal **completa** de CycleGAN aplicada a transferencia de estilo entre dos dominios musicales (p. ej. tango ↔ jazz): dos generadores ResNet, dos discriminadores PatchGAN, y las tres pérdidas estándar del marco (adversarial, ciclo, identidad).

Estructuras de datos. La representación es, de nuevo, un **piano roll multi-rol** segmentado por función textural (melodía, contrapunto, bajo, etc.), condicionado adicionalmente por un vector de **tensión armónica de 8 dimensiones** inyectado a la red mediante **AdaIN** (*Adaptive Instance Normalization*): en vez de concatenar la condición como canal adicional, AdaIN normaliza las activaciones internas de la red y las reescala/desplaza según los parámetros derivados del vector de condición — la misma técnica usada para condicionamiento de estilo en StyleGAN, aquí aplicada a controlar el nivel de tensión armónica de la transferencia.

Algoritmos. Los generadores son redes **ResNet** (bloques residuales, que facilitan el entrenamiento de redes más profundas al permitir que el gradiente fluya directamente a través de las conexiones de salto) con **Dropout2d(0.05)** en cada bloque residual como regularización adicional, explícitamente calibrada para corpus de entrenamiento pequeños (el caso típico en generación musical, donde los datos etiquetados por estilo son escasos comparado con visión por computador). Los discriminadores son de tipo **PatchGAN**: en vez de emitir una única puntuación real/falso para la imagen (piano roll) completa, clasifican cada parche local independientemente, lo que fuerza al generador a producir realismo a nivel de textura local además de a nivel global — coherente con la intuición de que el "estilo" musical se manifiesta en patrones locales repetidos (figuraciones, motivos rítmicos) más que en la estructura global. La función de pérdida adversarial es **LSGAN** (*Least-Squares GAN*), que sustituye la entropía cruzada binaria estándar por un error cuadrático, mitigando el problema de gradientes que se desvanecen cuando el discriminador se vuelve demasiado confiado — una mejora de estabilidad de entrenamiento bien documentada frente al GAN original. El *logging* desglosado por rama (adversarial A→B/B→A, ciclo A/B, identidad A/B, discriminador A/B) permite diagnosticar cuál de los seis términos de la función de pérdida compuesta domina o falla durante el entrenamiento.

Conceptos musicales. El condicionamiento por tensión armónica vía AdaIN añade un grado de control ausente en el CycleGAN original de visión: permite pedir explícitamente "transfiere el estilo B pero mantén (o exagera) este nivel de tensión armónica", separando la dimensión de "género/estilo" de la dimensión de "intensidad dramática", que en principio son ejes ortogonales de la experiencia musical.

ddsp_synth.py — Síntesis diferenciable de audio (DDSP: Differentiable Digital Signal Processing)

Propósito. Implementación de un autoencoder **DDSP** (Engel et al., Google Magenta, 2020): aprende a resintetizar un instrumento real a partir de sus *features* de control (altura fundamental f0 y sonoridad), usando módulos de síntesis clásicos (aditivo armónico + ruido filtrado) que son **diferenciables**, permitiendo entrenar la red completa de extremo a extremo con descenso de gradiente.

Estructuras de datos. `ZEncoder` codifica un espectrograma log-mel del audio a un vector latente `z` que captura el timbre; `Decoder` traduce `(f0, loudness, z)` a los parámetros de control de los sintetizadores (amplitud global, distribución de armónicos, magnitudes de banda de ruido); `DDSPAutoencoder` combina ambos con la etapa de síntesis. Los parámetros de control son señales de control de baja tasa (una por *frame*, p. ej. cada 4ms) que después se **sobremuestrean** (upsampling lineal) a la tasa de audio completa.

Algoritmos. El corazón conceptual de DDSP es sustituir una red neuronal generativa "de caja negra" (que tendría que aprender desde cero las leyes de la síntesis de sonido) por **módulos de síntesis DSP explícitos y diferenciables**, dejando que la red solo aprenda los *parámetros de control* de esos módulos: (1) **síntesis armónica aditiva** (`harmonic_synth_torch`), que genera audio como suma ponderada de sinusoides armónicas de la f0 estimada, con las amplitudes de cada armónico controladas por la red (mediante una `exp_sigmoid` que garantiza positividad y un rango dinámico amplio, la parametrización estándar del paper original); (2) **ruido filtrado** (`filtered_noise_synth_torch`), que da forma espectral a ruido blanco mediante una envolvente de magnitudes controlada por la red, vía STFT/ISTFT — captura los componentes no armónicos del sonido (soplo, ruido de arco, etc.) que la síntesis puramente armónica no puede representar. El entrenamiento usa una **pérdida espectral multi-escala** (`multiscale_spectral_loss`): compara los espectrogramas de magnitud del audio generado y del objetivo a varias resoluciones de FFT simultáneamente, en vez de comparar la forma de onda directamente (que sería extremadamente sensible a desajustes de fase sin significado perceptual). La extracción de f0 usa un **detector de altura por autocorrelación** simple, con un umbral de "sonoridad" (*voicing threshold*) que descarta como no sonoras las tramas cuyo pico de autocorrelación normalizado es débil, evitando alturas espurias en silencios o ruido.

Conceptos musicales. El principio DDSP formaliza directamente el modelo físico-acústico fuente-filtro: separar la altura fundamental (control de "qué nota") de la envolvente tímbrica (control de "qué instrumento/color") y de la componente de ruido (aliento, ataque), permitiendo edición independiente de cada una tras el entrenamiento — algo que un modelo neuronal de caja negra no ofrece de forma nativa.

La familia “diffusion_composer”: composición end-to-end mediante difusión

Tres versiones progresivas (`diffusion_composer.py` v0.1, `diffusion_composer_v3.py` y `diffusion_composer_v4.py`, ambas v3 con correcciones incrementales entre sí) abordan la composición completa de una obra multi-rol mediante **modelos de difusión probabilística (DDPM)**, con una diferencia arquitectónica central entre la v0.1 y las v3/v4.

v0.1 — Difusión Latente. Arquitectura de tres etapas: un encoder CNN comprime el piano roll multi-rol a un **espacio latente continuo**, un DDPM opera sobre ese espacio latente (condicionado en tensión armónica y estilo), y un decoder CNN reconstruye el piano roll desde una muestra denoised del espacio latente — análogo a Stable Diffusion en imagen (difusión en el espacio latente de un autoencoder, no directamente sobre los píxeles/notas).

v3/v4 — Difusión Directa. Elimina el autoencoder intermedio: una **U-Net 2D** aplica el proceso de difusión directamente sobre el piano roll (tratado como una “imagen” compás×altura×rol), con **parametrización x0** del DDPM — en vez de que la red prediga el ruido añadido en cada paso (parametrización ε estándar), predice directamente una estimación del dato limpio x0, una reparametrización matemáticamente equivalente pero que en la práctica puede estabilizar el entrenamiento en dominios distintos de imágenes naturales, como aquí un piano roll disperso y mayormente binario. La v4 mantiene la misma arquitectura documentada que v3, con mejoras de robustez incrementales (p. ej. opciones para reducir roles/rango de pitch en `prepare`, aliviando la carga computacional en corpus grandes).

Algoritmos comunes. Ambas familias exponen los mismos cuatro modos de composición: `sample` (generación pura desde ruido), `denoise` (parte de un MIDI de referencia con ruido parcial añadido —controlado por `--strength`— y lo “limpia” parcialmente, produciendo una variación cercana al original), `blend` (interpola entre dos estilos latentes) y `sweep` (recorre un parámetro continuamente para explorar el espacio). El muestreo usa el esquema **DDIM** acelerado (`--ddim-steps`, con `--eta` controlando la estocasticidad: 0 = determinista) en vez de los mil pasos completos del DDPM original, y un parámetro de `--temperature` para controlar la aleatoriedad del muestreo final.

Conceptos musicales. El condicionamiento explícito en un vector de **tensión armónica** (calculado con la misma heurística multi-factor que otras herramientas del ecosistema) permite guiar el proceso generativo con una noción musical de alto nivel además del propio “estilo” latente, combinando control estructural (dónde quiero tensión) con control estilístico (qué género/corpus).

drummer.py — Motor de percusión rítmica con catálogo de arquetipos estilísticos

Propósito. Añade una pista de batería estilísticamente coherente a una obra MIDI existente, con un flujo de trabajo que va desde aprender grooves de una colección personal (`learn`), extraer y nombrar patrones favoritos (`extract`), añadir percusión a una obra nueva (`add`), y transformarla o evolucionarla (`transform`/`morph`).

Estructuras de datos. Un **groove** se representa como una plantilla de patrones por elemento de batería (bombo, caja, hi-hat, color) sincronizada a una rejilla métrica, con reglas de variación propias y un *fill* característico. El catálogo incluye 14 "arquetipos" de estilo predefinidos (rock recto, funk de semicorcheas, ride de jazz con triplillo, bossa nova, reggae one-drop, afrobeat, clave latina, vals, tango, entre otros), cada uno con su propia plantilla base.

Algoritmos. El comando `learn` extrae estadísticamente patrones rítmicos de una colección de MIDIs de referencia, construyendo una biblioteca de grooves reutilizables — un enfoque de generación por corpus en vez de reglas puramente hardcodeadas, aunque el catálogo de arquetipos por defecto sí codifica reglas específicas por estilo (p. ej. bossa nova: clave 3-2, maracas en negras, bombo suave; reggae one-drop: bombo desplazado al tercer tiempo en vez del primero, la característica distintiva del estilo). Las operaciones `transform` y `morph` aplican variaciones controladas sobre un groove ya generado (transformaciones locales) o interpolan gradualmente entre dos grooves (morph), en la línea de otras herramientas del ecosistema que tratan patrones musicales como puntos en un espacio manipulable algebraicamente.

Conceptos musicales. El catálogo de arquetipos codifica rasgos rítmicos distintivos de tradiciones de percusión muy dispares — la **clave** (patrón asimétrico de referencia rítmica de la música afrocubana, en sus variantes 2-3 y 3-2), el *backbeat* seco del rock (acento en los tiempos 2 y 4), el patrón *one-drop* del reggae, y el triplillo *swing* del ride de jazz —, formalizando en cada caso la relación característica entre el bombo, la caja y el elemento de color (hi-hat/ride/maracas) que define auditivamente el género.

ear_trainer.py — Generador de dictados de entrenamiento auditivo

Propósito. El oído es, según el propio docstring, "la mitad de aprender a componer y a tocar", y el ecosistema carecía de una herramienta dedicada a entrenarlo: genera ejercicios de dictado musical reproducibles como MIDI (intervalos, calidades de acorde, progresiones armónicas cortas) junto con un solucionario separado, siguiendo el modelo pedagógico clásico del entrenamiento auditivo (*ear training*).

Estructuras de datos. Cada ejercicio es una tupla (estímulo sonoro, respuesta correcta), donde el estímulo se serializa a MIDI (reproducible en cualquier reproductor estándar) y la respuesta a un solucionario en texto plano o JSON, mantenidos deliberadamente separados para no revelar la respuesta al escuchar el ejercicio.

Algoritmos. Para `intervals`, se muestrean dos notas (melódicas —secuenciales— o armónicas —simultáneas, según `--harmonic`) separadas por un intervalo elegido de un repertorio que se amplía según el nivel de dificultad (`--level easy|medium|hard`); el "solucionario" es simplemente el nombre teórico del intervalo generado. Para `chords`, se genera un acorde con una calidad concreta (mayor, menor, disminuido, aumentado, con séptima…) elegida del repertorio de nivel correspondiente. Para `progressions`, se genera una secuencia de 2 a 4 acordes en una tonalidad dada, con el solucionario en numerales romanos. El uso de una **semilla aleatoria reproducible** (`--seed`) permite regenerar exactamente el mismo dictado — útil para comparar sesiones de práctica o para que un profesor comparta un ejercicio determinista con varios estudiantes.

Conceptos musicales. El diseño refleja directamente la progresión pedagógica estándar del solfeo auditivo: primero intervalos aislados (la unidad perceptual más básica), después acordes completos (percepción vertical simultánea), y finalmente progresiones armónicas cortas (percepción de la función tonal en contexto) — los tres niveles clásicos de un curso de entrenamiento auditivo de conservatorio.

etude_generator.py — Generador de ejercicios técnicos de piano

Propósito. Complementa a `fingering.py` (que calcula digitación para una pieza ya dada): genera desde cero material técnico diario de estudio pianístico —escalas, arpegios, acordes quebrados tipo Alberti, patrones de cinco dedos al estilo Hanon, y cadencias armónicas con enlace de voces— en cualquier tonalidad y modo, con digitación estándar ya anotada.

Estructuras de datos. Cada tipo de ejercicio se construye a partir de la escala/modo elegido (mayor, menor natural/armónica/melódica, y los modos griegos dórico, mixolidio, frigio, lidio), expandida al número de octavas pedido, con una función de digitación (`_octave_fingering`) que asigna el número de dedo estándar a cada grado de la escala según la mano y el patrón de teclas negras/blancas involucradas (el criterio clásico de digitación pianística: el pulgar nunca sobre tecla negra en las digitaciones estándar de escalas mayores).

Algoritmos. El generador de cadencias implementa la fórmula armónica de enlace de voces clásica **I–IV–I¾–V–I** (tónica, subdominante, tónica en segunda inversión como preparación ornamental de la dominante, dominante, tónica), con las reglas estándar de conducción de voces (movimiento por el intervalo más pequeño posible, resolución de la sensible) entre cada acorde consecutivo. El patrón Hanon (`hanon`) reproduce la figura característica del primer ejercicio de *The Virtuoso Pianist* de Charles-Louis Hanon: un patrón de cinco notas ascendente-descendente que se transpone grado a grado a lo largo de la escala completa.

Conceptos musicales. El fichero formaliza el currículo técnico estándar de la pedagogía pianística clásica (escalas y arpegios en todas las tonalidades, ejercicios mecánicos de independencia digital, cadencias armónicas básicas), con la digitación convencional ya resuelta algorítmicamente en vez de dejada a interpretación del estudiante.

evolver.py — Motor de evolución genética de secciones MIDI

Propósito. Toma N fragmentos MIDI como "genes" de una población inicial, genera variantes por cruce y mutación, las puntúa automáticamente usando el sistema de *scoring* de `midi_dna_unified` (consonancia, variedad, arco expresivo) y selecciona las mejores durante varias generaciones — un **algoritmo genético** clásico aplicado a composición musical, que sirve de motor reutilizable para otras herramientas del ecosistema (`counterexample.py`, `completer.py`).

Estructuras de datos. Cada individuo de la población es un fragmento MIDI completo (no una codificación abstracta), lo cual simplifica la aplicación directa de los operadores genéticos sobre estructuras musicales reales en vez de sobre una representación intermedia.

Algoritmos. Los operadores de **cruce** (*crossover*) combinan material de dos padres a lo largo de tres dimensiones independientes: contorno melódico, progresión armónica y patrón rítmico — permitiendo, por ejemplo, heredar el contorno melódico de un padre pero la armonía del otro. Los operadores de **mutación** cubren cinco tipos: desplazamiento de intervalo (±1-3 semitonos en notas aleatorias), escalado de duraciones rítmicas, variación de tempo (±5-15 BPM), transposición modal (al relativo o a la dominante) y sesgo en las curvas emocionales de tensión/*arousal*. La función de **fitness** combina el *score* estructural de `midi_dna_unified` con una **penalización de clones** (para mantener diversidad genética en la población, evitando la convergencia prematura característica de los algoritmos genéticos con poblaciones pequeñas) y un bonus por coherencia motívica entre generaciones consecutivas (para que la evolución no pierda por completo la identidad temática del material original). El bucle estándar de selección por élite (`--elite N`, los N mejores pasan directamente sin cruce ni mutación) combinado con reproducción del resto de la población es el esquema clásico de un algoritmo genético generacional.

Conceptos musicales. Separar los operadores de cruce por dimensión musical independiente (contorno, armonía, ritmo) en vez de operar sobre la representación MIDI cruda refleja la intuición de que estas tres dimensiones son parcialmente independientes en la percepción musical y pueden recombinarse de forma más significativa por separado que a nivel de bytes o eventos MIDI.

expectation_engine.py — Manipulación de expectativas armónicas (Meyer/Huron)

Propósito. Fundamentado explícitamente en la teoría de la emoción musical de Leonard Meyer y David Huron (*Sweet Anticipation*): la emoción musical surge de generar expectativas y luego retrasarlas, negarlas o cumplirlas en el momento adecuado. Localiza los puntos de resolución previstos (cadencias V→I) e inyecta "dispositivos" retóricos que manipulan esa expectativa, reservando la resolución plena para el clímax final. Se distingue de `reharmonizer.py` y `voice_leader.py` en que estos optimizan corrección armónica, mientras que esta herramienta optimiza deliberadamente la *sorpresa*.

Estructuras de datos. Trabaja sobre acordes detectados por plantilla armónica compás a compás, con la tonalidad estimada vía Krumhansl-Schmuckler, identificando explícitamente los movimientos V→I como "puntos de resolución previstos" candidatos a intervención.

Algoritmos. Cuatro dispositivos retóricos formalizados algorítmicamente: **cadencia rota** (la dominante que "debería" resolver en tónica se redirige a la submediante, V→vi — la resolución esperada se niega); **suspensión** (la nota melódica previa se prolonga por encima del cambio de acorde, retrasando su resolución — el clásico retardo 4-3 del contrapunto); **apoyatura** (nota extraña acentuada sobre el objetivo melódico esperado, que resuelve descendiendo, generando una sensación de anhelo momentáneo); **evasión** (el bajo de la resolución se desplaza a la tercera del acorde —primera inversión— en vez de a la fundamental, debilitando la llegada). El flujo selecciona qué cadencias intervenir según una densidad configurable y una semilla aleatoria, preservando deliberadamente (o incluso reforzando, con `--climax-resolve`) la última cadencia de la obra, para que la resolución final retenida durante toda la pieza llegue con máximo efecto.

Conceptos musicales. El fichero es una implementación computacional directa de la teoría "expectativa-tensión-resolución" (ETR) de la cognición musical: cada dispositivo corresponde a una técnica retórica documentada en la literatura de análisis musical (retardo, apoyatura, cadencia rota/evadida) reinterpretada explícitamente como una manipulación deliberada de la predicción implícita que un oyente hace sobre "qué viene después" en una progresión tonal familiar.

La familia “fingering”: digitación pianística automática (v1 → v3) y su puente a Synthesia

Tres versiones sucesivas implementan y refinan un optimizador de **digitación pianística** automática a partir de MIDI, basado conceptualmente en el proyecto `pianoplayer` de Marco Musy, más una herramienta de exportación al formato de la aplicación Synthesia.

fingering.py (v1) — optimizador base

Algoritmo central. Búsqueda por **ventana deslizante de 9 notas**: en vez de optimizar la digitación de toda la pieza de una vez (intratable combinatoriamente) ni nota a nota de forma puramente voraz (miope, ignora contexto), se optimiza una ventana local de 9 notas consecutivas a la vez, deslizándola progresivamente — un compromiso entre calidad de la solución y tractabilidad computacional. La clase `Hand` modela el optimizador para una mano: dada una secuencia de notas, calcula, para cada posible asignación de dedos, un coste basado en la posición física de las teclas (`keypos_midi`, que convierte un pitch MIDI en una posición horizontal en centímetros sobre el teclado) y en las restricciones anatómicas de la mano (parametrizadas por `--hand-size`, de XXS a XXL, con M ≈ 17 cm entre pulgar y meñique). La separación en manos usa índices de pista explícitos, o —con un único instrumento— una partición por mediana global de altura (notas agudas → mano derecha, graves → izquierda).

fingering_v2.py — memoria de postura, tempo variable, confianza

Añade **memoria de postura** (`relocation_alpha=0.3`): el coste de una nueva posición de mano no depende solo de la comodidad absoluta sino que se pondera hacia la posición de mano anterior, penalizando reposicionamientos innecesarios (un pianista real prefiere mantener la mano donde está si es razonablemente cómodo, en vez de re-optimizar continuamente desde cero). Sustituye la partición global por **separación inteligente de manos por ventana deslizante de 1 segundo** (calcula el pitch mediano local en vez de global, adaptando el punto de corte cuando la melodía completa sube o baja de registro a lo largo de la pieza — la partición global fallaría en piezas con tesitura muy amplia). Introduce un `TempoMap` que traduce tiempo en segundos a (compás, beat) respetando múltiples cambios de tempo dentro de la pieza, y una función de coste de **transición entre acordes consecutivos** (`_transition_cost_chord`) que modela explícitamente el movimiento físico de cada dedo entre dos posiciones sucesivas. Cada nota recibe además una **puntuación de confianza**, marcando con “(alt)” aquellas donde existe una digitación alternativa igualmente válida — señal explícita de ambigüedad para el usuario, en vez de forzar una única solución arbitraria.

fingering_v3.py — separación de voces internas

Añade **separación de voces dentro de una misma pista**: distingue notas de melodía (cortas) de notas de una voz sostenida simultánea (tenor/bajo, de mayor duración relativa al pulso), clasificando cada nota por su duración relativa al beat y comprobando si coincide temporalmente con una nota de melodía — permitiendo digitar de forma independiente una voz sostenida (p. ej. un bajo Alberti) frente a la línea melódica principal que suena por encima, un problema que v1/v2 no distinguían.

fingering_to_synthesia.py — puente al formato de la aplicación Synthesia

Convierte la digitación JSON producida por `fingering_v3.py` al formato propietario `.synthesia` (gramática *LocalFingerInfoList*/*FingerInfo*): dígitos 1-5 codifican mano izquierda (pulgar a meñique), 6-9 y 0 codifican mano derecha, con marcadores especiales para cambio de pista

(`tN:`) y salto de compás (`mN:`). El fichero documenta explícitamente una trampa práctica no evidente: el índice de pista al que se refiere el formato Synthesia es el índice **crudo** (0-based) tal como aparece en el MIDI, no el índice entre solo las pistas que contienen notas — muchos ficheros MIDI tipo 1 tienen una primera pista vacía de solo metadatos, desplazando en uno los índices reales de mano derecha/izquierda si no se detecta automáticamente.

Conceptos musicales. El problema de digitación pianística es, formalmente, un problema de optimización de camino de coste mínimo sobre un grafo cuyos nodos son asignaciones dedo-nota y cuyas aristas están ponderadas por el coste ergonómico de la transición entre dos posiciones consecutivas de la mano — la misma estructura matemática que la alineación de secuencias, aplicada aquí al espacio físico del teclado.

flow_composer.py — Composición mediante Normalizing Flows condicionales

Propósito. Genera obras multi-rol completas usando **flujos normalizadores** (*normalizing flows*, familia RealNVP/Glow) condicionales sobre el piano roll, en contraste explícito con los enfoques VAE (pérdida por reconstrucción aproximada) y difusión (proceso iterativo estocástico) usados en otras herramientas del ecosistema: un flujo normalizador es una **biyección exacta** entre el espacio de datos y un espacio latente gaussiano, por lo que la reconstrucción x→z→x es perfecta salvo el ruido de *dequantización* — propiedad que se explota directamente en el modo `reconstruct`.

Estructuras de datos. El piano roll multi-rol se trata como los "datos" sobre los que opera el flujo; el espacio latente resultante se descompone en `z_style` (código de estilo global) y `z_context` (código condicional adicional), ambos serializables a JSON tras un `encode`.

Algoritmos. La arquitectura apila K "pasos de flujo", cada uno compuesto de tres transformaciones invertibles encadenadas: **ActNorm** (normalización por activación, análoga a batch norm pero con parámetros por canal inicializados a partir del primer batch, e invertible exactamente), una **convolución 1×1 invertible** (generaliza la permutación fija de canales de RealNVP a una mezcla lineal aprendida, invertible por construcción al parametrizarse como matriz con descomposición LU o similar), y un **acoplamiento afín** (*affine coupling layer*) con **máscara tipo tablero de ajedrez** (*checkerboard*): se particiona el tensor en dos mitades según un patrón fijo, una mitad se deja sin cambios y la otra se transforma afinmente (escala + desplazamiento) con parámetros calculados por una subred condicionada en la primera mitad — la construcción que garantiza que el determinante del Jacobiano de la transformación completa sea fácil de calcular, requisito esencial para poder entrenar por **máxima verosimilitud exacta** (a diferencia de un VAE, que solo maximiza una cota inferior de la verosimilitud, el ELBO). El condicionamiento en tensión/estilo se inyecta vía **FiLM** (*Feature-wise Linear Modulation*) en cada subred de acoplamiento: los parámetros de escala y desplazamiento de la subred dependen linealmente del vector de condición, la misma técnica de condicionamiento ligero usada en muchos modelos generativos condicionales.

Conceptos musicales. La ventaja distintiva de los flujos frente a VAE/difusión para composición es la **invertibilidad exacta**: permite operaciones deterministas y perfectamente reversibles entre MIDI real y espacio latente, útil para tareas de edición precisa (mover un punto en el espacio latente y volver a decodificar sabiendo exactamente qué combinación de "estilo" se está variando) sin la pérdida de fidelidad inherente a la reconstrucción aproximada de un autoencoder variacional.

generar_midi.py — Sistema de composición YAML + Python para una obra concreta

Propósito. A diferencia de la mayoría de herramientas del ecosistema (genéricas, aplicables a cualquier MIDI de entrada), este es un **sistema de producción a medida** para una obra orquestal concreta de Pablo ("Entre el Tango y el Abismo", según el contexto del ecosistema): lee una partitura completa especificada en YAML declarativo (`partitura.yaml`) y genera los ficheros MIDI correspondientes, sección por sección, frase por frase, con soporte de *stems* por instrumento, exportación por leitmotiv, comparación de versiones (`--diff`), estadísticas, esquema visual HTML y renderizado a audio.

Estructuras de datos. El YAML de entrada declara: metadatos de la obra (título, compás, tonalidad, modo — con soporte de **14 modos/escalas**: los siete modos griegos, pentatónicas mayor/menor, blues, doble armónica, menor armónica/melódica, escala acústica y octatónica), **leitmotivs** (células temáticas identificadas con id, nombre y notas canónicas — permitiendo referenciarlas por nombre en cualquier frase de la obra), definición de pistas/instrumentos MIDI, y las **secciones** propiamente dichas como bloques de compases con su armonía, melodía y demás voces especificadas declarativamente.

Algoritmos. `build_chord_from_degree` construye acordes por grado modal (I-VII, en notación romana o numérica) resolviendo la calidad del acorde según el modo activo en ese momento — es decir, un sistema de armonía modal explícito donde el mismo grado numérico produce un acorde de calidad distinta según si la obra está en modo jónico, dórico, frigio, etc., en vez de asumir siempre la convención mayor/menor implícita. `_build_modal_voicing` resuelve la disposición concreta de las voces (raíz en el bajo más tres voces superiores) para cada acorde. El sistema de leitmotivs permite componer temáticamente por referencia simbólica en vez de por notas literales repetidas, facilitando variaciones consistentes de un mismo material a lo largo de la obra.

Conceptos musicales. El sistema de modos declarado (14 escalas, incluyendo escalas no diatónicas como la octatónica —alternancia tono-semitono, de uso característico en Stravinsky y el jazz modal— y la doble armónica —con dos segundas aumentadas, de sonoridad "oriental" característica) permite que una misma estructura formal de frases y leitmotivs se reinterprete armónicamente de formas radicalmente distintas simplemente cambiando el parámetro de modo declarado en el YAML.

genetic_style_transfer_v3.py — Transferencia de estilo por algoritmo genético + modelo aprendido

Propósito. Aprende transformaciones estilísticas entre una colección de MIDIs "origen" y una colección "destino" (p. ej. arreglos simples → arreglos jazzísticos) y las aplica a un MIDI nuevo, combinando un **modelo de transformación aprendido** (entrenado y reutilizable, serializado a `.pkl`) con mecanismos de aplicación granular, verbosa e incluso manual sin modelo entrenado.

Estructuras de datos. El modelo entrenado (`train`) encapsula un conjunto de parámetros de transformación aprendidos de los pares origen→destino de la colección de entrenamiento — presumiblemente estadísticas o pesos de un modelo scikit-learn dada la dependencia declarada, aunque el detalle exacto de cada operador de transformación (armónico, rítmico, dinámico…) se resuelve en el cuerpo del fichero.

Algoritmos. El modo `apply` permite aplicar parámetros de transformación concretos **sin necesidad de modelo entrenado**, útil para exploración manual del espacio de transformaciones; `block` ejecuta la transformación normal pero neutraliza selectivamente ciertos operadores indicados en `--block`, mostrando explícitamente el impacto marginal de cada operador individual sobre el resultado — una forma de análisis de sensibilidad *ablation-style* aplicado a un sistema de transformación musical, permitiendo entender qué parte de la "reestilización" aporta cada componente del modelo. El modo `verbose` expone el detalle nota a nota de cada operación aplicada, convirtiendo el proceso normalmente opaco de transformación en un registro auditable operación por operación.

Conceptos musicales. La capacidad de bloquear operadores individuales y observar el resultado formaliza una forma de **interpretabilidad por ablación** aplicada a transformación de estilo musical: en vez de tratar la reestilización como una caja negra, el sistema permite atribuir el cambio percibido a componentes concretos (p. ej. "el 40% del cambio de carácter proviene solo de la reescritura rítmica, el resto de la reescritura armónica").

La familia “genopatch”: matching de patches de síntesis por búsqueda evolutiva e inferencia amortizada

Tres ficheros abordan el mismo problema —el “problema inverso” del sonido: dado un WAV objetivo, encontrar los parámetros de un motor de síntesis paramétrico que mejor lo reproduzcan— con dos paradigmas algorítmicos distintos, inspirados en el “Genopatch” de Synplant 2 (Sonic Charge) pero sin red neuronal como motor principal de búsqueda.

genopatch.py (v1.1) y genopatch_v2.py (v2.0) — búsqueda evolutiva

Algoritmo. Un **algoritmo genético** clásico —selección por torneo, cruce aritmético (interpolación lineal entre los vectores de parámetros de dos padres), mutación gaussiana y elitismo— busca directamente en el espacio de parámetros del motor de síntesis elegido, sin ningún modelo aprendido de por medio: la propia búsqueda evolutiva **es** el modelo, determinista dado un `--seed`, ejecutable enteramente en CPU. La v2.0 amplía drásticamente el catálogo de motores respecto a v1.1: FM de dos operadores con filtro resonante propio, síntesis aditiva (reutilizando `audio_lab.py`), síntesis **sustractiva** clásica (dos osciladores diente de sierra/pulso con PWM + sub-oscilador + ruido + filtro resonante, el modelo “analógico” canónico), síntesis de cuerda pulsada **Karplus-Strong** (un filtro en peine —*comb filter*— vectorizado que simula la propagación y amortiguación de una onda en una cuerda física), síntesis por **tabla de ondas** (wavetable, escaneando entre cuatro formas de onda de un ciclo a lo largo de la nota) y ruido coloreado con resonancia de cuerpo tipo pasa-banda; cada motor se registra como un `EngineSpec` independiente, de modo que añadir un motor nuevo no requiere tocar el algoritmo genético ni la interfaz de línea de comandos. La función de **fitness espectral** compara el logaritmo de la magnitud STFT de candidato y objetivo (agregada en media y desviación estándar sobre el tiempo, para ser invariante a la duración exacta), con un descriptor explícito adicional de vibrato (profundidad y tasa extraídas de la frecuencia instantánea vía transformada de Hilbert) — necesario porque la resolución de la propia STFT no capturaría por sí sola la modulación de vibrato como una característica perceptualmente distinguible.

genopatch_flow.py (v0.2) — inferencia amortizada mediante flujos normalizadores

Motivación y algoritmo. En vez de resolver el problema inverso desde cero en cada consulta (como hace el algoritmo genético), este fichero entrena **una única vez** un flujo condicional $p(\theta|x)$ sobre pares sintéticos (θ muestreado de un prior, x = render(θ) con el simulador de `genopatch_v2.ENGINES`), de modo que la inferencia posterior es un único *forward pass* de la red — sin ningún bucle evolutivo. Esto es **inferencia basada en simulación** (*simulation-based inference*, SBI / *neural posterior estimation*), una familia de técnicas de estadística computacional para problemas donde el simulador es conocido pero la verosimilitud no tiene forma cerrada. La arquitectura es un **RealNVP condicional**: un “embedder” reduce el vector de *features* de audio (~2052 dimensiones, el mismo espacio de features que el fitness espectral de v2) a un embedding compacto de 64 dimensiones, sobre el cual seis capas de acoplamiento afín (con máscara tipo tablero de ajedrez) transforman el vector de parámetros normalizado en ruido gaussiano base, condicionadas en ese embedding, entrenadas por máxima verosimilitud. La ventaja conceptual señalada explícitamente en el código: cuando el problema inverso **no es identificable** (varias combinaciones de parámetros producen sonidos indistinguibles), el algoritmo genético colapsa arbitrariamente a un único punto de esa “cresta” de soluciones equivalentes, mientras que el flujo modela la **distribución posterior completa** $p(\theta|x)$ y puede muestrear varias soluciones válidas y diferentes entre sí — confirmado empíricamente en el motor aditivo, donde varias muestras del flujo para el mismo objetivo recuperan la misma forma armónica normalizada pero con escala bruta libre, en vez de fijar arbitrariamente una escala incorrecta como hacía el GA.

Conceptos musicales/acústicos. Ambos enfoques abordan el “problema inverso” central de la síntesis sonora — dado un timbre deseado, encontrar los parámetros que lo producen — que es notoriamente difícil de resolver analíticamente para motores de síntesis no lineales (FM, sustractiva con filtros resonantes), y que un sound designer humano resuelve tradicionalmente por prueba y error manual; ambas herramientas automatizan ese proceso con paradigmas algorítmicos complementarios: búsqueda vs. inferencia amortizada.

graded_variations.py — Variaciones musicales por dimensión, a un grado pedagógico fijo

Propósito. Genera variaciones de una pieza de piano ya simplificada a un grado pedagógico concreto, donde cada variación aísla deliberadamente **una única dimensión musical** — ornamentación, ritmo, textura, registro, armonía, modo o articulación — permitiendo al estudiante practicar variaciones de un mismo material sin alterar significativamente su dificultad técnica global.

Estructuras de datos. Comparte el modelo de datos MIDI (`MidiEvent`, `MidiTrackData`, `MidiData`, `TimeContext`) con la familia `combined_simplifier`, añadiendo `ScoredNote` (nota con su peso estructural ya calculado) y `VariationResult` (registro de los cambios concretos aplicados por una variación, usado para generar el informe de consola).

Algoritmos. Cada tipo de variación es una transformación paramétrica dirigida a una única dimensión perceptual, manteniendo las demás fijas — un diseño experimental de "un factor a la vez" (*one-factor-at-a-time*) aplicado a composición: la variación ornamental añade figuración (notas de adorno) sin tocar el ritmo base; la variación rítmica reescribe patrones de duración sin cambiar las alturas; la de registro transpone octavas selectivamente; la armónica sustituye acordes por otros de función equivalente; la de modo reinterpreta la pieza en un modo paralelo (p. ej. mayor→menor) preservando el contorno melódico. El grado pedagógico de cada variación puede desviarse en ±1 respecto al grado objetivo, deliberadamente, para aislar mejor el efecto perceptual de la técnica modificada de cualquier cambio incidental en dificultad motriz.

Conceptos musicales. El diseño refleja una técnica pedagógica consolidada en la enseñanza de composición: aislar variables de variación (¿qué pasa si cambio solo el ritmo? ¿solo la armonía?) para que el estudiante perciba con claridad el efecto de cada parámetro musical por separado, en vez de presentar variaciones que cambian varias dimensiones simultáneamente y confunden la atribución causal del cambio perceptual.

grammar_composer.py — Composición dirigida por gramáticas formales (modo compose + modo learn)

Propósito. Motor de composición basado en **gramáticas formales**: en modo `compose`, combina un MIDI de entrada con una gramática textual que define una estructura jerárquica (referenciando compases concretos del MIDI original o notas ABC directas) para generar un nuevo MIDI; en modo `learn`, invierte el proceso — analiza un MIDI y extrae automáticamente su gramática subyacente mediante el algoritmo **Re-Pair**.

Estructuras de datos. Una gramática es un conjunto de reglas de reescritura tipo `<etiqueta> : <elementos>`, donde los elementos pueden ser notas ABC individuales con duración (`C4:1`, `E4:0.5`), referencias a compases concretos del MIDI de entrada (por número o rango, `1-4`), o referencias recursivas a otras reglas (`[label]`) — es decir, formalmente una **gramática libre de contexto** cuyos símbolos terminales son notas o compases MIDI y cuyos símbolos no terminales son las etiquetas de regla, con la raíz definida en la primera línea del fichero de gramática.

Algoritmos. El algoritmo **Re-Pair** (Larsson & Moffat, 1999; aquí descrito como una simplificación de **Sequitur**) es un algoritmo de compresión gramatical: reemplaza repetidamente el par de símbolos adyacentes más frecuente en la secuencia por un nuevo símbolo no terminal, produciendo una gramática libre de contexto que genera exactamente la secuencia original con un número mínimo de reglas — cuando se aplica a una secuencia de eventos musicales (notas con su duración), las reglas resultantes capturan automáticamente los **motivos repetidos** de la pieza (frases, células temáticas, patrones de acompañamiento) sin necesidad de anotación manual, de forma análoga a como se usa en compresión de texto o de secuencias de ADN para descubrir subcadenas repetidas.

Conceptos musicales. La idea de tratar una pieza musical como una jerarquía de motivos anidados reutilizados —una gramática— formaliza computacionalmente la práctica compositiva tradicional de construir formas musicales por repetición y variación de células temáticas (el “desarrollo motívico” del análisis formal clásico), y el hecho de que Re-Pair la descubra automáticamente por compresión estadística sugiere que la estructura motívica de una pieza bien construida es, en cierto sentido, información redundante comprimible.

grammar_transformer.py — Transformación estructural de MIDI mediante gramática Re-Pair (3.181 líneas)

Propósito. Extiende la idea de `grammar_composer.py` (extracción de gramática Re-Pair) hacia la **transformación** del MIDI original: en vez de solo aprender la gramática, sustituye elementos de una longitud dada por otros elementos "compatibles" de la misma gramática, muestreados con una temperatura configurable — produciendo variaciones estructuralmente coherentes de la pieza que preservan su jerarquía motívica pero alteran su contenido concreto.

Estructuras de datos. Cada nota se trata como un token hoja (pitch, duración en beats); el algoritmo Re-Pair construye reglas jerárquicas sobre esa secuencia de tokens, y cada regla resultante se **vectoriza a 47 dimensiones** (presumiblemente estadísticas del fragmento musical que representa: perfil de alturas, ritmo, contorno, etc.), permitiendo comparar elementos de la gramática entre sí en un espacio vectorial common.

Algoritmos. El pipeline completo: (1) extracción de notas a secuencia de tokens; (2) aprendizaje de reglas jerárquicas Re-Pair (idéntico a `grammar_composer.py` en modo `learn`); (3) vectorización de cada regla a 47 dimensiones; (4) construcción de una **matriz de compatibilidad por coseno**, agrupada por longitud de elemento (solo se comparan elementos de la misma duración, para que la sustitución preserve el "hueco" rítmico exacto); (5) sustitución de los elementos de la longitud objetivo por otros elementos compatibles, muestreados con temperatura T (T=0: siempre el más compatible por similitud coseno, sustitución "greedy" de mínima variación; T más alto: más exploración/sorpresa); (6) reconstrucción de la secuencia plana de notas y escritura del MIDI de salida.

Conceptos musicales. Sustituir un motivo por otro "compatible" en similitud vectorial —en vez de aleatoriamente— formaliza la práctica compositiva de la variación motívica controlada: cambiar el contenido concreto de una célula temática manteniendo su función estructural y su encaje rítmico dentro de la jerarquía formal de la pieza, similar en espíritu al principio de "desarrollo temático" beethoveniano pero automatizado mediante compresión gramatical y similitud vectorial.

harmonic_analyzer.py — Análisis armónico funcional didáctico (numerales romanos con explicación)

Propósito. El "compañero armónico" de `metric_analyzer.py` (que cubre el eje rítmico): toma un MIDI y devuelve, compás a compás, el acorde detectado, su cifrado, su numeral romano, su función armónica (Tónica/Subdominante/Dominante) y las cadencias presentes, con una explicación en lenguaje llano del **por qué** de cada análisis — a diferencia de `reharmonizer.py` o `voice_leader.py`, que transforman la armonía, esta herramienta solo **analiza y explica** lo que ya está escrito, con fines didácticos explícitos.

Estructuras de datos. El pipeline segmenta las notas por compás (o por una ventana en beats configurable), detecta el acorde y el bajo por comparación con plantillas de intervalos características de cada calidad de acorde, y produce como salida una tabla compás a compás con acorde/numeral/función/explicación, exportable opcionalmente como JSON adjunto (*sidecar*).

Algoritmos. La estimación de tonalidad global usa **Krumhansl-Schmuckler**. La novedad de la v1.1 es la detección de **dominantes secundarios**: cuando aparece una tríada mayor o séptima de dominante "sorprendente" —que no encajaría diatónicamente en el grado donde aparece— el sistema comprueba si el acorde siguiente resuelve exactamente una quinta justa por debajo; si resuelve, se etiqueta como V/x o V7/x (dominante secundario de x, función D con tonicización temporal) en vez de forzar una etiqueta diatónica engañosa; si no resuelve de esa manera específica, se mantiene el etiquetado cromático genérico anterior (acorde de mediante, préstamo modal). Esta distinción —etiquetar según la función *resuelta* en vez de según la clasificación puramente vertical del acorde— es la diferencia entre un análisis armónico superficial y uno funcional genuino.

Conceptos musicales. El fichero implementa, paso a paso y con explicación textual, el flujo estándar de un curso universitario de armonía funcional: identificación de acordes por análisis vertical, determinación de tonalidad, cifrado en numerales romanos relativos a esa tonalidad, clasificación funcional (T/S/D) de cada acorde, y reconocimiento de cadencias — con el añadido de tonicización temporal vía dominantes secundarios, el primer paso hacia el análisis de música cromática más allá de la práctica común estrictamente diatónica.

harmonic_gnn_analyzer.py — Análisis armónico mediante grafo de notas (redes neuronales de grafos)

Propósito. Complementa a `harmonic_analyzer.py`, que analiza compás a compás sobre un histograma de alturas "plano" (una secuencia lineal de ventanas temporales independientes). Este fichero modela en cambio la pieza completa como un **grafo de notas** con dos tipos de arista —simultaneidad (verticalidad: "esto es un acorde") y sucesión (horizontalidad: "esto es una línea melódica" dentro de la misma voz)— y propaga información entre notas vecinas mediante varias rondas de *message passing*, la operación fundamental de las **redes neuronales de grafos (GNN)**.

Estructuras de datos. El grafo tiene un nodo por nota (con atributos de pitch, tiempo de inicio/fin) y dos familias de aristas: simultaneidad (entre notas que suenan a la vez) y sucesión (entre notas consecutivas de la misma voz), exportable directamente a JSON con el modo `graph`.

Algoritmos. El argumento central, explícito en el propio código, es que un histograma "plano" por compás **no distingue** "cuatro notas simultáneas = un acorde en bloque" de "cuatro notas en sucesión rápida = un arpegio o una floritura melódica que pasa por esas alturas sin que ellas constituyan el acorde real" — ambos casos producen el mismo histograma de clases de altura si se analizan sin estructura temporal fina. El grafo de notas, en cambio, sí distingue estos casos: al propagar información entre vecinos mediante **rondas de message passing** (equivalente conceptual a una capa de convolución de grafo: cada nodo actualiza su representación combinando la de sus vecinos según el tipo de arista), el contexto armónico de cada nota emerge de combinar tanto lo que suena simultáneamente con ella como lo que la rodea temporalmente en su propia voz, en vez de depender de una ventana temporal arbitraria fija. Esto habilita una capacidad ausente en `harmonic_analyzer.py`: clasificar **cada nota individual** (no solo cada compás) como "estructural" (tono del acorde) u "ornamental" (nota de paso, bordadura, apoyatura) — útil como entrada directa para `schenkerian_reducer.py`, que necesita precisamente esa clasificación nota a nota para construir su jerarquía de reducción.

Conceptos musicales. El fichero formaliza, mediante una estructura de grafo y propagación de mensajes en vez de reglas explícitas de contrapunto, la distinción perceptual entre "nota estructural del acorde" y "nota de adorno melódico" — la misma distinción que un analista humano hace intuitivamente al distinguir un arpegio de un acorde en bloque, aquí resuelta mediante una arquitectura de aprendizaje relacional sobre el grafo temporal de la pieza.

harmony_checker.py — Corrector pedagógico de conducción de voces SATB

Propósito. Complementario inverso de `voice_leader.py` (que genera un voicing SATB correcto): toma un intento de armonización a cuatro voces (Soprano/Alto/Tenor/Bajo) ya escrito y señala sus errores con explicación pedagógica, "como un profesor corrigiendo deberes de armonía" — no reescribe nada, solo marca y explica.

Estructuras de datos. Acepta tanto cuatro pistas separadas (ordenadas automáticamente por altura) como una única pista con acordes de cuatro notas por tiempo, que se separan algorítmicamente en las cuatro voces.

Algoritmos. Detecta siete categorías de error mediante comparación directa entre pares de voces en instantes consecutivos: quintas y octavas **paralelas** (dos voces se mueven en el mismo intervalo perfecto de un acorde al siguiente — la infracción más fundamental del contrapunto de Fux/Rameau, porque diluye la independencia perceptual de las voces), quintas **directas** (llegada a una quinta/octava por movimiento directo —ambas voces en la misma dirección— entre las voces extremas), **cruce de voces** (una voz teóricamente inferior sube por encima de su vecina superior), **solapamiento** (una voz invade el registro que acaba de dejar su vecina, aun sin cruzarse estrictamente), **espaciado** excesivo (hueco mayor de una octava entre voces superiores adyacentes — regla clásica de coralística), **sensible sin resolver** (la nota sensible de la tonalidad, en el bajo o la soprano, no asciende a la tónica en la cadencia) y **salto melódico excesivo** (más de una sexta en una sola voz, señalado como aviso, no como error duro). Cada detección viene acompañada de una explicación en lenguaje natural del principio de conducción de voces infringido.

Conceptos musicales. El catálogo de errores es exactamente el temario estándar de un curso universitario de armonía a cuatro voces al estilo coral, formalizado como comprobaciones deterministas sobre pares de acordes MIDI consecutivos — convirtiendo una tarea tradicionalmente corregida a mano por un profesor de conservatorio en un proceso automatizado con feedback inmediato.

harvester.py — Extracción de fragmentos musicales reutilizables desde MIDIs

Propósito. Segmenta MIDIs en fragmentos reutilizables (secciones formales, motivos, frases entre cadencias, segmentos de textura homogénea) usando criterios exclusivamente musicales, no arbitrariamente temporales, sirviendo de proveedor de material a otras herramientas del ecosistema (`completer.py`, `leitmotif_tracker.py`).

Algoritmos. Combina cinco criterios de detección de fronteras, ponderados conjuntamente: (1) **curva de novedad** derivada de una **matriz de auto-similitud (SSM)** con patrón de "tablero de ajedrez" (*checkerboard kernel*) — la técnica estándar de segmentación estructural en recuperación de información musical (MIR), donde se detecta un cambio de sección buscando, a lo largo de la diagonal de la SSM, los puntos donde un kernel en forma de tablero de ajedrez (positivo en dos cuadrantes diagonales, negativo en los otros dos) tiene máxima correlación — matemáticamente, detecta transiciones de "un bloque homogéneo de similitud a otro"; (2) detección de **cadencias** (auténtica, plagal, semicadencia, engañosa) como fronteras formales naturales; (3) **repetición de motivos** vía Re-Pair (compartido con `grammar_composer.py`); (4) silencios o pausas largas como frontera natural; (5) combinación ponderada de todos los criterios anteriores en una puntuación de frontera única. La salida se organiza en categorías con sufijos semánticos explícitos (`.frag_A/B`: secciones formales, `.motif_NN`: motivos repetidos, `.cadence_*`: fragmento previo a cada tipo de cadencia, `.phrase_NN`: frases delimitadas entre cadencias, `.texture_NN`: segmentos de textura rítmica homogénea).

Conceptos musicales. El uso combinado de la SSM tipo *checkerboard* (una técnica tomada directamente de la literatura de MIR para segmentación estructural de audio, aquí aplicada a datos simbólicos) junto con criterios explícitamente musicales (cadencias, motivos, silencio) refleja que la segmentación formal "correcta" de una pieza rara vez coincide con un único criterio: combina evidencia estadística (auto-similitud, repetición) con evidencia teórica (cadencias tonales) para una segmentación más robusta que cualquiera de los dos criterios por separado.

instrument_cloner.py — Clonación neuronal de timbres a partir de ~16 s de audio (DDSP)

Propósito. Puerto a PyTorch puro (sin TensorFlow ni la librería `ddsp` de Google) de *Neural Instrument Cloning* (Jonason & Ericsson, DAFx 2022): clona el timbre de un instrumento real a partir de apenas ~16 segundos de audio de referencia, sin reentrenar la red completa para cada instrumento nuevo.

Estructuras de datos. La arquitectura central es un **decoder GRU compartido** entre todos los instrumentos vistos durante el entrenamiento, más un **embedding z pequeño y libre por instrumento**: clonar un instrumento nuevo no reentrena el decoder compartido (que permanece congelado) sino que solo ajusta los pocos parámetros libres específicos de ese instrumento — el embedding z, la respuesta de reverb aprendida y la mezcla seco/húmedo — sobre los pocos segundos de audio de referencia disponibles. Esta es la misma filosofía de **meta-aprendizaje por parámetros compartidos + parámetros específicos rápidos de ajustar** que sustenta técnicas como el *fine-tuning* de adaptadores ligeros en otros dominios del aprendizaje profundo.

Algoritmos. El *pipeline* replica el modelo fuente-filtro DDSP: `(f0, loudness, z[instrumento])` se procesan con una GRU más MLPs para producir una distribución armónica, una amplitud global y magnitudes de ruido, que alimentan la misma **síntesis armónica + ruido filtrado** al estilo DDSP descrita en `ddsp_synth.py`, seguida de un **reverb con respuesta al impulso (IR) aprendida por instrumento** y una mezcla húmedo/seco también aprendida. La extracción de f0 y sonoridad es completamente propia (algoritmo **YIN** para detección de altura por autocorrelación normalizada con corrección de submúltiplos, más ponderación tipo A para la sonoridad perceptual), evitando dependencias externas como CREPE (detector de f0 basado en redes neuronales) o librosa.

Conceptos musicales. El hecho de que solo ~16 segundos de audio basten para "clonar" un timbre nuevo depende críticamente de que el decoder compartido ya haya aprendido, durante el entrenamiento multi-instrumento, la estructura general de "cómo suena un instrumento acústico" (mapeo de f0/sonoridad a espectro armónico plausible); el embedding z específico solo necesita capturar las particularidades tímbricas de ese instrumento concreto dentro de ese espacio ya aprendido — un ejemplo claro de transferencia de aprendizaje aplicada a modelado de timbre.

latent_composer.py — Composición end-to-end mediante VAE multi-rol

Propósito. El más temprano (v0.1) de los cuatro enfoques generativos de "composición end-to-end" del ecosistema (junto a `latentscore/diffusion/flow_composer`), basado en un **autoencoder variacional (VAE)** sobre piano rolls multi-rol.

Estructuras de datos. El espacio latente se organiza en `z_style` (código de estilo), extraíble de cualquier MIDI de referencia vía `encode` y serializable a JSON. Los modos de composición (`compose`) incluyen `reconstruct` (decodificar directamente un `z` dado), `z-noise` (perturbar un `z` de referencia con ruido gaussiano antes de decodificar — la forma más simple de generar variaciones), `blend` (interpolar entre dos `z` de estilo) y `sweep` (recorrido continuo de un parámetro, opcionalmente suavizado con scipy).

Algoritmos. Un VAE estándar: encoder que produce los parámetros (media, log-varianza) de una distribución gaussiana aproximada al posterior, muestreo por el **truco de reparametrización** (necesario para que el muestreo sea diferenciable y el gradiente pueda propagarse a través de él durante el entrenamiento) y decoder que reconstruye el piano roll. El entrenamiento maximiza el **ELBO** (*Evidence Lower Bound*: reconstrucción menos divergencia KL respecto al prior gaussiano), con `--beta` y `--beta-warmup` controlando el peso de la divergencia KL a lo largo del entrenamiento — la técnica estándar de "beta-annealing" de β-VAE, que evita el colapso posterior (el fenómeno donde el decoder ignora completamente el código latente) al introducir gradualmente la presión regularizadora KL en vez de aplicarla a plena fuerza desde el inicio del entrenamiento. La opción `--retrieval nnr` (vecino más cercano en el espacio latente) ofrece una alternativa no paramétrica a la decodificación neuronal: en vez de generar directamente desde el decoder, recupera el MIDI real del corpus de entrenamiento cuyo `z` esté más cerca del punto consultado.

Conceptos musicales. Comparado con la difusión (que requiere un proceso iterativo de muestreo) y los flujos (biyección exacta pero arquitectónicamente más restringida), el VAE ofrece el espacio latente más simple y de menor dimensión de los cuatro enfoques, a costa de una calidad de reconstrucción típicamente más "promediada"/borrosa — el compromiso clásico de los VAE frente a modelos generativos más recientes.

latent_lab.py — Codec neuronal de audio y puentes espectrograma↔latente (3.876 líneas)

Propósito. Laboratorio extenso de experimentación con **codecs de audio neuronales** propios (entrenados desde cero, no reutilizando EnCodec/SoundStream) y con puentes bidireccionales entre representaciones espectrales (STFT, compartidas con `audio_lab.py`) y espacios latentes aprendidos, incluyendo un flujo completo de edición mediante imágenes PNG.

Estructuras de datos. El **codec wav↔latente** propio (`train-coder`) aprende una representación latente comprimida del audio crudo. El puente **espectrograma↔latente** se puede resolver por varios métodos intercambiables (`--method {auto,pix2pix,greedy,viterbi,causal,causal-adv}`), cada uno representando una estrategia algorítmica distinta para el mismo problema de traducción entre dos representaciones. El flujo `spectrogram-to-latent-train_step1/step2` reutiliza el mismo puente PNG-editable que `audio_lab.py`, permitiendo editar visualmente espectrogramas y aprender el mapeo correspondiente en el espacio latente. Se incluye también **reducción de dimensionalidad por PCA** sobre representaciones latentes o espectrales, con conversión bidireccional entre coordenadas PCA de baja dimensión y la representación intermedia completa — permitiendo visualizar y editar directamente en un espacio de 2-3 dimensiones interpretable.

Algoritmos. Los métodos de traducción espectrograma↔latente cubren un espectro de complejidad: `greedy` (decisión local óptima paso a paso, sin considerar el futuro), `viterbi` (**programación dinámica** que encuentra la secuencia globalmente óptima de estados latentes discretos dado un modelo de transición y emisión — el algoritmo clásico usado en HMMs para decodificación de secuencias), `pix2pix` (traducción de imagen a imagen mediante GAN condicional, tratando el espectrograma y el latente como dos "imágenes" a traducir entre sí, en la línea del paper original de Isola et al.), `causal` y `causal-adv` (variantes que respetan una restricción de causalidad temporal — el latente en el instante t solo depende del espectrograma hasta t, necesario para aplicaciones de baja latencia o *streaming*, con la variante `-adv` añadiendo entrenamiento adversarial).

Conceptos musicales. El puente PNG editable extiende a espacios latentes aprendidos la misma filosofía de edición visual creativa que `audio_lab.py` aplica a espectrogramas STFT crudos, permitiendo manipular directamente representaciones internas de un modelo neuronal con herramientas gráficas convencionales, sin necesidad de escribir código.

latentscore_composer.py — Síntesis procedimental de audio ambiente desde texto

Propósito. Integración directa del proyecto `latentscore` (github.com/beatsbyprabal/latentscore): un motor de síntesis 100% NumPy/SciPy (sin modelos neuronales de por medio) que genera texturas de audio ambiente a partir de descripciones textuales libres ("lluvia en ciudad", "jazz cafe") o de configuraciones JSON explícitas, con soporte de *morphing* entre dos ambientes y encadenamiento de varias "vibes" en una secuencia con transiciones.

Estructuras de datos. Cada descripción textual se traduce (presumiblemente mediante un diccionario/mapa semántico interno, dado que no depende de ningún LLM) a un conjunto de parámetros de síntesis procedimental —tempo, brillo espectral y demás "patches" ajustables explícitamente vía `--patch tempo=slow brightness=dark`—, que a su vez controlan generadores de textura sonora (ruido coloreado, capas rítmicas, resonancias).

Algoritmos. El comando `morph` realiza una **transición paramétrica suave** entre los parámetros de dos configuraciones (dos "vibes"), interpolando el vector de parámetros de síntesis a lo largo de varios pasos discretos en vez de mezclar directamente las señales de audio resultantes — una interpolación en el espacio de *parámetros* generativos, más coherente perceptualmente que un simple *crossfade* de las señales. El comando `chain` encadena varias vibes en una secuencia con fundidos (`--fade`) entre transiciones consecutivas, generando una pista ambiente continua de larga duración a partir de una lista de descripciones o de un fichero de playlist.

Conceptos musicales. El enfoque de "síntesis procedimental" (parámetros generativos determinados por reglas explícitas, sin aprendizaje estadístico) para música ambiental/generativa sigue la tradición de la "música generativa" popularizada por Brian Eno: sistemas basados en reglas que producen texturas sonoras continuas y variables sin una partitura fija, aquí con el añadido de control por lenguaje natural sobre los parámetros del sistema.

leadsheet_realizer.py — De cifrado de acordes a acompañamiento pianístico tocable

Propósito. El puente práctico que falta entre la armonía abstracta (`chord_progression_generator.py`, `reharmonizer.py`, que trabajan la armonía en abstracto) y el coral SATB de `voice_leader.py`: convierte un cifrado de acordes de jazz/pop estándar (`C Am7 F G`) en un acompañamiento de piano legible y tocable por una persona, con patrones de *comping* idiomáticos y *voicings* ergonómicos.

Estructuras de datos. El parser de cifrado reconoce la notación estándar de acordes de jazz (tríadas, séptimas de todas las calidades —mayor, menor, disminuida, semidisminuida—, sextas, novenas añadidas, suspendidas, y **acordes con bajo invertido** mediante notación de barra, `C/E`, `G/B`), con duración opcional por acorde expresada en compases.

Algoritmos. Seis patrones de "comping" (acompañamiento pianístico idiomático) están codificados explícitamente: `block` (acorde sostenido más bajo, el patrón más simple), `ballad` (acorde en tiempos 1 y 3, bajo en raíz-quinta), `arpeggio` (arpegio ascendente de las notas del acorde), `alberti` (bajo Alberti clásico: grave-agudo-medio-agudo), `waltz` (patrón de vals 3/4: bajo en el primer tiempo, acorde en los tiempos 2 y 3) y `pop` (bajo en la mano izquierda con acordes a contratiempo en la derecha — el patrón sincopado característico del acompañamiento pop). Cada patrón resuelve el *voicing* concreto de las notas del acorde a una disposición cómoda para la mano, respetando el bajo invertido cuando se especifica con notación de barra.

Conceptos musicales. Los seis patrones catalogados son, cada uno, el idioma de acompañamiento estándar de un género o contexto distinto —balada, jazz, vals, pop—, formalizando la práctica de "realizar" un cifrado (una habilidad central del pianista de acompañamiento, tradicionalmente aprendida por imitación e improvisación) como una función determinista del cifrado y el patrón elegido.

leitmotif_analyst.py — Identificación de leitmotifs y sus transformaciones

Propósito. Dado un MIDI, identifica automáticamente sus **leitmotifs** (células musicales breves y recurrentes asociadas a una idea, en el sentido wagneriano del término) y rastrea cada aparición a lo largo de la obra, detectando qué **transformación** compositiva se le aplicó en cada reaparición respecto a su forma original.

Estructuras de datos. Los motivos candidatos se extraen con el mismo algoritmo **Re-Pair** usado en `harvester.py` y `grammar_composer.py`, y se agrupan por significancia (frecuencia de aparición y longitud) para determinar cuáles constituyen leitmotifs genuinos de la obra frente a repeticiones triviales.

Algoritmos. Para cada reaparición del motivo, el sistema comprueba, contra el catálogo de transformaciones de `variation_engine.py`, cuál (si alguna) explica la diferencia entre la ocurrencia original y la nueva: **inversión** del contorno melódico (cada intervalo asciende donde antes descendía y viceversa — una de las transformaciones seriales clásicas, popularizada por la técnica dodecafónica pero presente ya en la fuga barroca), **retrógrado** (la secuencia temporal se invierte — se toca "hacia atrás"), **inversión retrógrada** (ambas transformaciones combinadas), **aumentación** y **diminución** rítmicas (todas las duraciones se multiplican o dividen por un factor, típicamente 2), **transposición** (mismo contorno exacto, otra altura) y **transformación modal** (cambio de modo mayor a menor o viceversa, manteniendo el contorno). El resultado se presenta como una tabla de apariciones por compás, un **mapa ASCII** de presencia de cada leitmotif a lo largo de la obra completa (una visualización compacta y textual de la "arquitectura temática" de la pieza) y exportación de cada leitmotif como MIDI independiente.

Conceptos musicales. El catálogo de transformaciones (inversión, retrógrado, inversión retrógrada, aumentación, diminución, transposición) es precisamente el conjunto canónico de operaciones sobre un motivo temático en la tradición contrapuntística occidental desde la fuga barroca hasta la técnica dodecafónica de la Segunda Escuela de Viena, aquí usado no para *generar* variaciones (como en `variation_engine.py`) sino para **reconocer** cuál de ellas explica una reaparición observada en una obra ya compuesta — un problema de clasificación inversa al de generación.

leitmotif_tracker.py — Gestión, siembra y rastreo de leitmotifs en composición dirigida

Propósito. Complementa a `leitmotif_analyst.py` (que *detecta* leitmotifs en una obra ya existente): este gestiona un **banco de leitmotifs** definidos explícitamente por el usuario o importados de otras herramientas, y los siembra estratégicamente a lo largo de una obra en construcción, transformándolos según el contexto emocional de cada sección.

Estructuras de datos. Un banco de motivos registrados (`register`), cada uno con su "*fingerprint*" (huella identificable) y un registro de sus apariciones. El comando `plan` opera sobre un `obra_plan.json` producido por `narrator.py`, leyendo las pistas de leitmotif (`leitmotif_hints`) que ese plan narrativo asocia a cada sección.

Algoritmos. El flujo cubre todo el ciclo de vida de un leitmotif en una composición dirigida: registro manual o importado (`from-theorist`, que importa sugerencias de `theorist.json`; `from-harvest`, que importa motivos ya extraídos por `harvester.py`), planificación de dónde sembrar cada motivo según el plan narrativo de la obra, un modo `suggest` exploratorio que propone activamente ubicaciones de inyección, `inject` para materializar la siembra según un *schedule* JSON explícito, `trace` para detectar apariciones en una obra ya compuesta (reutilizando la lógica de `leitmotif_analyst.py`), un informe combinado (mapa ASCII/JSON) y `propagate`, que aplica una **mutación semántica** a todos los motivos del banco simultáneamente (p. ej. "todos los motivos se oscurecen" al entrar en una sección dramática) delegando la transformación concreta en `variation_engine.py`. La integración con `mscz2vec.py` permite **reconocimiento por similitud vectorial** — detectar reapariciones de un motivo incluso si ha sido transformado lo suficiente como para no coincidir literalmente con su forma original.

Conceptos musicales. El sistema formaliza el uso wagneriano/straussiano del leitmotif como técnica narrativa: un motivo no es una entidad fija sino que se transforma sistemáticamente según su contexto dramático (una idea musical puede "oscurecerse" armónicamente al asociarse a un giro trágico de la trama), y este fichero automatiza tanto la asignación estratégica de motivos a secciones como su transformación contextual coherente.

lyrics_melody.py — Melodía condicionada por el contenido emocional del texto (v3.1)

Propósito. Aprende la relación entre el contenido semántico/emocional de cada verso de una canción y la melodía completa que lo acompaña, a partir de un corpus de ficheros ABC con letra alineada (campo `w:`), generando después melodías nuevas condicionadas en un texto dado, con fluidez explícita entre frases consecutivas.

Estructuras de datos. Cada verso se codifica con un **embedding de sentence-embeddings preentrenado y multilingüe** (MiniLM), inyectado como *segment embedding*: cada posición de la secuencia melódica recibe el embedding del verso al que pertenece, acoplando texto y melodía a nivel de representación vectorial continua en vez de mediante reglas discretas. La secuencia melódica completa de la canción (todas las frases concatenadas, separadas por un token especial `<phrase>`) se modela con un **Transformer causal con atención relativa** al estilo Music Transformer, de modo que cada nota se condiciona en todo el historial melódico anterior — no solo en la frase actual — dando continuidad y fluidez natural en las transiciones entre versos.

Algoritmos. Tres mejoras de consistencia melódica añadidas en la v3.1, aplicables sin reentrenar el modelo (post-procesado sobre la salida del muestreo): (A) **suavizado de octava**, que tras decodificar cada token corrige la octava elegida a la más próxima a la nota anterior (minimizando el salto en semitonos), eliminando saltos de registro abruptos que el modelo entrenado podía producir entre notas o versos consecutivos; (B) **máscara de escala**, que antes de muestrear pone a $-\infty$ los logits de los tokens de altura cuya clase de altura no pertenece a la escala/modo pedidos (diez modos disponibles, de mayor y menor a pentatónicas, blues y cromática), garantizando que la melodía resultante quede estrictamente dentro de la tonalidad solicitada sin alterar los pesos del modelo entrenado — una forma de **decodificación restringida** (*constrained decoding*) equivalente en espíritu a enmascarar tokens inválidos en generación de texto estructurado; (C) **sesgo de intervalo por perfil emocional**, un término aditivo en el espacio logarítmico de los logits proporcional a los pesos de un perfil (p. ej. semitonos reforzados para un perfil "melancólico", quintas para uno "heroico"), que refuerza suavemente —sin forzar categóricamente— los intervalos típicamente asociados a ese carácter emocional.

Conceptos musicales. La combinación de máscara de escala (restricción dura, garantiza corrección tonal) con sesgo de intervalo por perfil (preferencia blanda, orienta el carácter expresivo) ilustra un patrón de diseño recurrente en generación musical controlable: separar las restricciones que deben cumplirse *siempre* (corrección teórica) de las preferencias que deben *orientar sin forzar* (expresividad), aplicadas en momentos distintos del proceso de muestreo (antes vs. durante el cálculo de probabilidades).

lyrics_writer.py — Generación automática de letras para melodías ABC (LLM)

Propósito. Dado un fichero de notación ABC sin letra y una temática general, genera automáticamente las líneas de letra (`w:`) correspondientes usando un LLM (API de Anthropic u OpenAI), de modo que el número de sílabas de cada línea coincida exactamente con el número de notas del compás correspondiente — el requisito estructural que espera `lyrics_melody.py` para su corpus de entrenamiento (una sílaba por nota).

Algoritmos. El flujo procesa compás a compás, en orden: para cada compás, calcula el número de notas (= número de sílabas objetivo), y llama al LLM pidiendo un fragmento de letra en español con exactamente ese número de sílabas, coherente temáticamente y narrativamente con los versos ya generados (se pasa el **historial completo** de versos previos en cada llamada, para mantener coherencia narrativa a lo largo de toda la canción — un uso directo del contexto de ventana larga de un LLM para generación secuencial con memoria completa). El conteo de sílabas se verifica con una **heurística silábica para español** tras cada respuesta del modelo; si no coincide con el objetivo, se **reintenta** (hasta `--max-retries` veces) indicándole explícitamente al modelo el número de sílabas obtenido frente al deseado — un bucle de verificación-y-corrección (*self-correction loop*) donde la heurística de conteo actúa como una función de validación externa al LLM, similar en espíritu a los bucles de ejecución-y-corrección usados en generación de código.

Conceptos musicales. La restricción de “una sílaba por nota” formaliza la práctica clásica de escritura de canciones silábica (frente a la escritura melismática, donde una sílaba se extiende sobre varias notas), y el hecho de que se verifique mecánicamente compás a compás en vez de dejarlo a la intuición del LLM refleja que el conteo silábico es una restricción dura y verificable objetivamente, no una cuestión de estilo negociable.

melody_adapter.py — Adaptación de melodías a progresiones de acordes externas

Propósito. Dado un MIDI de melodía y una progresión de acordes distinta (en texto o en MIDI), ajusta las notas de la melodía para que sean compatibles con la nueva armonía, respetando el contorno melódico original y distinguiendo entre notas estructurales (del acorde) y notas de paso/ornamentales, que se tratan con criterios distintos.

Algoritmos. Cuatro modos de adaptación con niveles crecientes de intervención: `strict` (solo desplaza las llamadas *avoid notes* —notas que forman una disonancia especialmente chocante contra el nuevo acorde, típicamente una segunda menor sobre la fundamental o la tercera— y solo en tiempos métricamente fuertes, para mínima intervención); `smooth` (además de `strict`, añade notas de paso para suavizar los saltos melódicos que la propia adaptación introduce, dando un resultado más cantábile/fluido); `jazz` (trata las tensiones 9ª, 11ª y 13ª como notas válidas incluso sin preparación explícita —el vocabulario armónico ampliado del jazz—, desplazando únicamente las *avoid notes* más severamente disonantes: la 9ª bemol, la 13ª bemol, u 11ª natural sobre un acorde mayor); `rewrite` (reescribe tanto el ritmo como las alturas manteniendo solo el contorno general, para máxima compatibilidad armónica a costa de fidelidad literal a la melodía original). El modo `--report` cuantifica el resultado: proporción de notas desplazadas sobre el total, un *score* de compatibilidad armónica antes y después de la adaptación (compartiendo la métrica con `reharmonizer.py`), y una lista explícita de colisiones resueltas y sin resolver.

Conceptos musicales. La jerarquía de intervención (strict → smooth → jazz → rewrite) formaliza una escala continua entre dos objetivos en tensión — fidelidad a la melodía original vs. corrección armónica frente a la nueva progresión —, con el criterio jazzístico de qué constituye una "*avoid note*" (dependiente del idioma armónico, no absoluto: una 11ª natural es tensión válida en un acorde menor pero disonancia a evitar en uno mayor) codificado explícitamente como parte central del modo `jazz`.

La familia “melody_conditioned”: generación de melodía condicionada a acordes (v1.1 → v5.0)

Cinco versiones sucesivas del mismo problema —generar una línea melódica cuya cada nota “sabe” en todo momento qué acorde suena y decide en consecuencia— documentan una evolución algorítmica particularmente clara del ecosistema, añadiendo motores de generación en cada iteración sin eliminar los anteriores.

v1.1 — condicionamiento determinista y LLM. Dos motores: `chord_guided` (determinista, combina una cadena de Markov condicionada por tensión, un contorno melódico “emocional” y una tabla `EMOTIONAL_CHORD_WEIGHTS` que pondera intervalos según el estado afectivo deseado, sin necesitar entrenamiento) y `llm` (usa Claude o ChatGPT como motor generativo mediante *prompting* estructurado, en modo `direct` —descripción simbólica del contexto— o `fewshot` —ejemplos reales del corpus como demostración en el prompt).

v2.0 — cuatro motores algorítmicos clásicos. Añade `chord_guided` como Markov de **segundo orden** (entrenable opcionalmente desde un corpus MIDI real), `grammar` (elaboración schenkeriana en tres capas ancladas al acorde activo: chord tones estructurales → notas de paso → ornamentos, determinista), `search` (*beam search* tipo A* con función de coste combinando compatibilidad armónica, contorno y suavidad, con garantías locales por nota) y `genetic` (población de melodías evaluada por *fitness* multicriterio, incluyendo clímax situado en la proporción áurea).

v3.0 — motor `lstm`. Añade un quinto motor: una LSTM entrenada sobre corpus propio, situando el sistema en el espectro completo desde reglas puramente deterministas hasta aprendizaje profundo end-to-end sobre los mismos datos de entrada.

v4.0 — motivo semilla, Kneser-Ney, arco de tensión, interpolación. Cuatro motores nuevos: `motif` (aplica el catálogo completo de transformaciones contrapuntísticas —repetición, secuencia, inversión, retrogradación, aumentación/disminución, variación ornamental— a un motivo semilla de 2-4 notas, dado por el usuario o generado automáticamente, produciendo coherencia temática interna); `bayes` (n-grama con **suavizado de Kneser-Ney** — el suavizado estándar de modelado de lenguaje estadístico, superior al add-k simple porque redistribuye probabilidad según la *versatilidad* de un contexto, no solo su frecuencia, funcionando sin GPU ni redes neuronales); `tension_arc` (planificación **de atrás hacia adelante**: decide primero la nota de resolución de cada frase, después la nota de máxima tensión, y rellena algorítmicamente el camino entre ambas — invirtiendo el orden habitual de generación secuencial hacia adelante, para garantizar que la frase tenga un destino audible planificado desde el principio); `interpolate` (interpolación lineal entre dos perfiles emocionales completos, con `--alpha` recorriendo continuamente de un perfil a otro, p. ej. de “flamenco” a “tanguero”).

v5.0 — nueve motores, 16 perfiles, compositor formal. Consolida los motores anteriores (nueve en total) con 16 perfiles emocionales, post-procesado armónico añadido tras la generación, y un **compositor de estructuras formales completas** que encadena múltiples llamadas de generación de frase en una arquitectura formal mayor (no solo genera una frase suelta sino una sección estructurada).

Conceptos musicales comunes. El motivo estructural detrás de toda la familia es el espectro completo de paradigmas de generación de secuencias musicales condicionadas — reglas deterministas, cadenas de Markov, búsqueda heurística, algoritmos evolutivos, aprendizaje profundo (LSTM) y modelos estadísticos con suavizado — todos resolviendo el mismo problema (melodía condicionada armónicamente), permitiendo comparar directamente sus resultados sobre exactamente el mismo espacio de entrada/salida.

melody_critic.py — Crítica constructiva pedagógica de una melodía

Propósito. A diferencia de `quality_scorer.py` (que puntúa una obra pero no enseña), analiza una melodía y devuelve **comentario pedagógico** explicando qué está bien y qué mejorar, pensado explícitamente para quien está aprendiendo a componer, no para clasificar/rankear obras.

Algoritmos. Siete métricas de análisis melódico clásico, cada una con su propio criterio de "buena práctica" pedagógica: **ámbito** (distancia entre la nota más grave y más aguda; un ámbito moderado es considerado ideal para líneas cantábiles); **proporción grados/saltos** (movimiento conjunto —por segundas— frente a saltos mayores de una tercera, un indicador clásico de "cantabilidad" melódica); **saltos sueltos** (saltos grandes que *no* están compensados por un movimiento posterior en dirección contraria — la regla de compensación de saltos del contrapunto tradicional: un salto grande debería seguirse, idealmente, de movimiento por grado en sentido opuesto); **posición del clímax** (dónde cae la nota más aguda de la frase; el ideal pedagógico situado hacia el 75% de la frase, coherente con la proporción áurea usada en otras herramientas del ecosistema); **cambios de dirección** (sinuosidad del contorno: ni una línea completamente recta —aburrida— ni errática —incoherente); **notas repetidas** (proporción de repeticiones consecutivas, indicador de estatismo melódico); y **equilibrio de frases** (segmentación por silencios y balance entre la frase antecedente y consecuente, la estructura de pregunta-respuesta clásica).

Conceptos musicales. El fichero codifica, con umbrales explícitos, un conjunto de heurísticas de "buena escritura melódica" que forman parte del canon pedagógico de composición desde el contrapunto especies hasta los manuales modernos de melodía: cantabilidad (grados vs. saltos), dirección del clímax, variedad de contorno y equilibrio formal — la misma información que un profesor de composición señalaría al revisar el ejercicio de un estudiante.

melody_generator.py — Generación de melodías originales desde cero (“Bloque 0” melódico)

Propósito. Construye una melodía completa *ex nihilo* a partir de parámetros musicales o de lenguaje natural, sin ningún MIDI de entrada — el equivalente melódico exacto de `chord_progression_generator.py` para la voz principal, el “bloque 0” del pipeline compositivo del ecosistema.

Algoritmos. Cinco motores intercambiables: `markov` (cadena de Markov de segundo orden sobre pares intervalo-duración, con estadísticas aprendidas del corpus si se proporciona o tablas estilísticas internas por defecto); `grammar` (gramática generativa basada en reglas schenkerianas: elabora progresivamente desde una estructura de capa profunda —típicamente un arpegio de tónica— añadiendo notas de paso y ornamentos en capas sucesivas, el mismo principio estructural usado en `combined_simplifier.py` pero en dirección generativa en vez de reductiva); `search` (búsqueda heurística A* que explora el espacio de alturas guiada por una función de coste de consonancia, contorno y tensión objetivo, con garantías tanto locales — nota a nota— como globales sobre la frase completa); `genetic` (población de frases evaluadas por *fitness* multicriterio, evolucionada por cruce y mutación); y `hybrid`, que combina `grammar` (para la estructura profunda) con `markov` (para la ornamentación de superficie), descrito como el motor que produce los resultados “más musicales” por defecto — una síntesis explícita de los dos paradigmas (reglas estructurales + estadística de corpus) en vez de depender de uno solo.

Conceptos musicales. El motor `hybrid` formaliza una intuición compositiva real: la estructura profunda de una buena melodía suele seguir principios composicionales relativamente universales (arpegios de acorde, movimiento hacia un clímax), mientras que su superficie concreta (qué notas de paso exactas, qué ornamentos) se beneficia de la variedad estadística observada en ejemplos reales — combinar ambos aporta tanto coherencia estructural como naturalidad de superficie.

melody_harmonizer.py — Adaptación de melodía a una armonía dada, con ornamentación (v2.0)

Propósito. Toma un MIDI de armonía (acordes) y otro de melodía, y reescribe la melodía para que sea musicalmente correcta sobre la armonía dada — el problema complementario e inverso de `chord_progression_generator.py/reharmonizer.py` (que arman una progresión para una melodía existente): aquí la armonía es el dato fijo y la melodía la que se adapta.

Estructuras de datos. La **arquitectura de estrategias** es explícitamente extensible: cada modo de adaptación (`--adapt-mode`) es una función registrada de forma independiente (patrón *registry*, con `--list-strategies` para enumerarlas), de modo que añadir o quitar una estrategia no requiere tocar las demás — el mismo patrón de diseño usado en `melody_harmonizer.py`'s propia familia de herramientas y en `chord_pattern_voicer.py`.

Algoritmos. Se mencionan explícitamente modos como `dp_global`, sugiriendo que al menos una de las estrategias resuelve la adaptación mediante **programación dinámica global** sobre toda la melodía (en vez de decisiones locales nota a nota), buscando la secuencia de ajustes de altura globalmente óptima según una función de coste que penaliza tanto la disonancia armónica como el alejamiento del contorno original — análogo en estructura al algoritmo de alineación de secuencias (Needleman-Wunsch) pero aplicado a compatibilidad armónica en vez de similitud de secuencias biológicas. Tres tipos de ornamento post-adaptación, aplicables selectivamente: **apoyatura** (nota disonante acentuada en tiempo fuerte que resuelve por grado), **nota de paso** (rellena el intervalo entre dos tonos del acorde por movimiento conjunto) y **bordadura** (nota auxiliar superior o inferior que se aparta y vuelve a la nota original) — el catálogo estándar de disonancias no armónicas preparadas y resueltas del contrapunto tonal.

Conceptos musicales. Comparte con `melody_adapter.py` el problema general de compatibilizar melodía y armonía, pero con la dirección de la restricción invertida (armonía fija, melodía flexible) y con énfasis explícito en la ornamentación post-adaptación como recurso expresivo adicional, no solo en la corrección de colisiones.

meter_adapter.py — Adaptación métrica: reescritura rítmica entre compases distintos

Propósito. Implementa directamente las guías del método "Adapting Music from one Meter to Another" (Active Analysis): reescribe una pieza en un compás distinto del original (p. ej. de 4/4 a 3/4), añadiendo o quitando pulsos, mientras preserva en la medida de lo posible el material asociado a los tiempos fuertes (incluyendo anacrusas), la textura rítmica (sin exceder la subdivisión más rápida del original) y el contorno melódico.

Algoritmos. Dos estrategias simétricas según la dirección del cambio: al **añadir pulsos** (compás destino más largo), si ya existe silencio de cola disponible en la unidad rítmica se usa ese margen sin tocar ninguna nota (equivalente a "separar más los motivos"); si la unidad está saturada (sin holgura), se alarga la nota del tiempo más débil, o —si el hueco es grande— se le añade una repetición melismática (`--melisma`) en vez de un único valor de duración desproporcionadamente largo, preservando la sensación de pulso interno. Al **quitar pulsos** (compás destino más corto), si ya hay silencio de cola suficiente simplemente se acorta el contenedor sin tocar notas; si la unidad está saturada, se eliminan notas siguiendo un **orden de prioridad explícito**: primero las repetidas, después las de paso (movimiento conjunto, menos estructurales), y solo como último recurso las de tiempo débil — siempre preservando la primera y la última nota de la unidad, que anclan el material a los tiempos fuertes de entrada y salida.

Conceptos musicales. El orden de prioridad de eliminación de notas (repetidas > de paso > tiempo débil, nunca las notas de los extremos) codifica una jerarquía de importancia estructural muy similar en espíritu a la reducción schenkeriana usada en `combined_simplifier.py`: las notas menos estructurales (repetición literal, ornamentación de paso) se sacrifican primero, preservando el "esqueleto" armónico-melódico de la frase incluso bajo una restricción métrica muy distinta a la original.

metric_analyzer.py — Análisis de estructura métrica y síncopa

Propósito. Cubre explícitamente un hueco identificado en la suite de análisis del ecosistema: mientras otras herramientas (`analyzer`, `quality_scorer`, `section_score`, `arc_supervisor`, `playability_auditor`) miden altura, armonía, emoción, forma y tocabilidad, ninguna diagnosticaba específicamente el **eje rítmico-métrico** — cuánto sincopa la obra, dónde "pelea" contra el pulso, cómo se agrupa en hipercompases, y si el ritmo es plano o variado.

Algoritmos. La métrica de síncopa implementa directamente el modelo de **Longuet-Higgins & Lee** (LHL, 1984): una nota se considera sincopada cuando cae en una posición métricamente débil pero se sostiene sobre una posición métricamente fuerte inmediatamente posterior (la nota "ocupa" tiempo fuerte sin haber empezado en él), ponderada por el peso métrico relativo de la posición implicada — el modelo canónico de la literatura de cognición musical para cuantificar síncopa perceptual. La **densidad** se mide en onsets por compás; la **fuerza del downbeat** como proporción de onsets que caen exactamente en el primer tiempo. El **nPVI** (*normalized Pairwise Variability Index*) mide la variabilidad rítmica de los *IOI* (Inter-Onset Intervals, intervalos entre ataques consecutivos) — una métrica tomada de la lingüística (originalmente usada para cuantificar el ritmo silábico de distintos idiomas, distinguiendo lenguas de ritmo acentual de las de ritmo silábico) y adoptada en musicología computacional para distinguir ritmo "plano" (regular) de "irregular" (variado). El **hipermetro** se estima buscando periodicidad en la acentuación agrupada en unidades de 2, 4 u 8 compases (grupos hipermétricos), y la **estabilidad** cuantifica cuántos cambios de compás/tempo ocurren a lo largo de la obra.

Conceptos musicales. El modelo LHL de síncopa y el nPVI de variabilidad rítmica son ambos herramientas tomadas directamente de la literatura académica de cognición musical y lingüística computacional respectivamente, aplicadas aquí como métricas cuantitativas objetivas sobre datos MIDI crudos, allí donde tradicionalmente un analista humano describiría estas propiedades de forma cualitativa ("esta pieza sincopa mucho", "el ritmo es muy regular").

midi2yaml.py — Conversión de fragmentos MIDI a sintaxis de partitura.yaml

Propósito. Herramienta de "ida y vuelta" para el sistema declarativo de `generar_midi.py`: lee un MIDI producido por cualquier herramienta del ecosistema y produce bloques YAML listos para insertar de vuelta en `partitura.yaml` (melodía, armonía y/o marcato), en la sintaxis `[compás, beat, duración, nota, velocidad]` que ese sistema espera.

Algoritmos. El proceso es esencialmente una serialización estructurada: convierte eventos MIDI absolutos a la representación relativa (compás, beat) usada por el YAML declarativo, con parámetros de redondeo rítmico (`--redondear`, para cuantizar a una rejilla legible en vez de valores de tiempo exactos con ruido de interpretación humana) y un umbral de silencio (`--umbral-silencio`) para decidir cuándo un hueco entre notas se considera una pausa explícita frente a simplemente el final de una nota.

Conceptos musicales. Esta herramienta cierra el ciclo del flujo de trabajo compositivo de Pablo para su obra orquestal a medida: permite generar material con cualquier herramienta algorítmica del ecosistema (que produce MIDI) y luego **incorporarlo de vuelta** a la fuente de verdad declarativa (`partitura.yaml`) editada a mano, en vez de que ambos flujos de trabajo —algorítmico y declarativo manual— permanezcan separados e incompatibles.

midi_align.py — Alineación temporal de MIDIs mediante puntos de anclaje

Propósito. Recibe un MIDI y una lista de pares de tiempos (origen→destino) y reescala cada segmento del MIDI para que sus bordes temporales coincidan con los tiempos de destino especificados — pensado explícitamente para alinear *stems* exportados de un generador de audio externo (Suno, mencionado explícitamente) con el MIDI original de referencia que los originó, cuando ambos han derivado temporalmente por razones de renderizado.

Algoritmos. Cada "anclaje" es un par de instantes en formato minuto:segundo.milisegundo; el primer anclaje debe ser necesariamente 0:00.000→0:00.000, y el último segmento se estira hasta el final del MIDI de referencia. Dos modos de reescalado por segmento: `tempo` (cambia el BPM efectivo de cada segmento entre dos anclajes consecutivos, preservando el ritmo interno relativo —solo cambia la "velocidad" global del segmento, como un *time-stretch* uniforme) y `linear` (redistribuye las notas proporcionalmente en el tiempo dentro del segmento, útil cuando los desajustes internos del segmento son irregulares y un simple cambio de tempo uniforme no bastaría para realinear cada nota individual con precisión).

Conceptos musicales. El problema que resuelve es el de **alineación temporal no uniforme** entre dos representaciones del mismo contenido musical que han derivado en el tiempo — un problema estructuralmente análogo al de alineación de audio a partitura (*score-following*) en recuperación de información musical, aquí resuelto de forma simplificada mediante interpolación por tramos entre puntos de anclaje dados manualmente en vez de mediante alineación automática (p. ej. DTW).

midi_concat.py y midi_merge.py — Composición temporal y espacial de MIDIs

Propósito. Dos utilidades complementarias de bajo nivel: `midi_concat.py` (v2.0) encadena N MIDIs **en serie** (uno reproducido a continuación del anterior), útil para unir frases, variaciones o secciones generadas por separado en una única pieza continua; `midi_merge.py` (v1.0) los combina **en paralelo**, convirtiendo cada MIDI de entrada (stem) en una pista distinta de un único MIDI multi-pista simultáneo.

Estructuras de datos. `midi_concat.py` ofrece un **selector de pistas y canal** mediante una sintaxis compacta tras `@` en el nombre de fichero (`archivo.mid@0-2,5` para pistas concretas, o `all` por defecto), permitiendo componer selectivamente solo ciertas voces de cada fuente. `midi_merge.py` gestiona automáticamente la **reasignación de canales MIDI** para evitar colisiones entre stems (reservando explícitamente el canal 9, el estándar General MIDI para percusión, que nunca se reasigna) y normaliza el **TPB** (ticks por beat, la resolución temporal del fichero) de todos los stems de entrada al valor máximo encontrado (o a uno especificado), reescalando proporcionalmente los tiempos de cada stem para preservar su ritmo relativo exacto bajo la nueva resolución.

Conceptos musicales. Ambas herramientas formalizan las dos operaciones básicas de composición de material musical discreto: concatenación temporal (forma A-B-C, secciones sucesivas) y superposición simultánea (textura polifónica, varias voces a la vez) — los dos ejes ortogonales sobre los que se construye cualquier estructura musical compleja a partir de fragmentos más simples.

midi_dna_unified.py — Mezclador unificado de “ADN” musical (v1.4, fusión de tres generaciones)

Propósito. El motor central de generación estilística del ecosistema, descrito explícitamente como la fusión de “lo mejor de tres generaciones” de mezcladores MIDI anteriores: dados MIDIs fuente, extrae su “ADN” musical (ritmo, emoción, forma, armonía, ornamentación, groove) y lo recombina para producir nuevo material estilísticamente fiel.

Estructuras de datos y algoritmos (catálogo de 17 capacidades, A-Q). Entre las más distintivas: **extracción rítmica avanzada** mediante histograma de 16 subdivisiones (una rejilla de semicorchea por compás en 4/4) con detección de síncopa y pesos de acento; **curvas emocionales completas** que combinan varios modelos de la literatura de cognición musical — el modelo de **tensión melódica de Lerdahl** (basado en distancia en el espacio tonal jerárquico de su teoría generativa de la música tonal), el **espacio Tonnetz** (la red toroidal de relaciones armónicas por terceras y quintas, originaria de Euler y retomada en la teoría neo-riemanniana), y ejes de **valencia/activación** (el modelo circumplejo de emoción de Russell, adaptado a música); **segmentación formal automática** en secciones A/B/C; Markov de segundo orden para melodía; **mosaic splicing** (fragmentos reales del MIDI fuente, transpuestos y reinsertados, en vez de generados nota a nota — un enfoque de “collage” o *concatenative synthesis* simbólico); **contrapunto multi-especie** seleccionado dinámicamente según la densidad emocional del momento; **groove map** (patrón de *timing* humanizado — microdesviaciones respecto a la rejilla exacta— extraído estadísticamente del MIDI fuente y reaplicado); gramática **Sequitur** para clasificar frases generadas por calidad musical; **voice-leading riguroso** sin paralelas y movimiento mínimo entre voces; y **modulación real por sección** (al relativo en la sección B, a la dominante en otras, siguiendo convenciones formales clásicas de modulación tonal entre secciones de una obra).

Conceptos musicales. El propio nombre “ADN musical” es la metáfora organizadora de toda la herramienta: cada MIDI fuente se descompone en un conjunto de “genes” estilísticos independientes (ritmo, emoción, forma, ornamentación, groove) que pueden combinarse, mutarse y heredarse de forma modular — la misma metáfora que sustenta a `evolver.py` y `counterexample.py`, que consumen directamente el sistema de *scoring* de esta herramienta.

midi_pianoroll_analyzer.py — Análisis multi-pista con visualización HTML interactiva

Propósito. Toma un MIDI multi-pista, identifica automáticamente el rol de cada pista (melodía, contra-melodía, armonía, bajo, acompañamiento) y genera un **informe HTML interactivo** con piano roll por instrumento, fila de acordes con función armónica, fila de sección global y métricas por compás, todo alineado en el mismo eje temporal — una especie de "panel de control" visual que integra en una sola vista información que otras herramientas del ecosistema producen por separado (harmonic_analyzer, arc_supervisor, grammar_composer).

Algoritmos. El piano roll de cada pista se colorea por frase según la **gramática Re-Pair individual de esa pista** (cada instrumento tiene su propia segmentación motívica, no una compartida), los acordes se etiquetan con función armónica (Tónica/Predominante/Dominante/Dominante secundario), la sección global se detecta por SSM y curva de novedad (la misma técnica de `harvester.py`), y las métricas por compás (tensión, densidad, velocidad, contorno) se calculan con las mismas fórmulas usadas en otras herramientas de análisis del ecosistema, unificadas aquí en una sola visualización temporal sincronizada.

Conceptos musicales. El diseño de "todo alineado en el mismo eje temporal" refleja la práctica de análisis de partitura orquestal, donde un analista examina simultáneamente varias capas de información (qué instrumento toca qué, qué armonía suena, en qué sección formal estamos) sincronizadas visualmente por compás — aquí automatizado y hecho interactivo en un navegador.

midi_split_harmony.py — Separación automática de melodía y acordes

Propósito. Dado un MIDI de entrada (monofónico o polifónico, con armonía limpia o mal transcrita — p. ej. proveniente de una transcripción automática de audio ruidosa), separa la voz melódica principal de la progresión armónica subyacente, produciendo un MIDI de melodía limpio y continuo y otro de acordes cuantizado y corregido.

Algoritmos. Pipeline de cinco etapas: (1) pre-análisis (tempo, compás, tonalidad, canales); (2) separación de melodía mediante un algoritmo **skyline mejorado** —no la simple regla "nota más aguda gana" del skyline clásico, sino ponderado por continuidad temporal y por **verosimilitud interválica** (penaliza saltos melódicos implausibles, prefiriendo una línea que se mueva de forma musicalmente coherente aunque eso signifique no elegir siempre la nota más aguda); (3) extracción de acordes mediante **chroma por ventanas solapadas** combinado con *template matching* (comparación contra plantillas de intervalos de cada calidad de acorde) y un **HMM (modelo oculto de Markov) de funciones armónicas**, que impone coherencia temporal entre acordes consecutivos vía las probabilidades de transición del HMM — evitando que el reconocimiento acorde-a-acorde produzca secuencias armónicamente incoherentes que un análisis puramente instantáneo (sin memoria de estado) sí podría producir; (4) post-limpieza (cuantización al pulso, fusión de acordes de duración anormalmente corta, corrección por conducción de voces, verificación de coherencia tonal global); (5) exportación. El modo `corpus` complementa el modo `deterministic` (basado puramente en teoría musical) aprendiendo matrices de transición y emisión del HMM directamente de un directorio de MIDIs de referencia, combinándolas con las reglas teóricas.

Conceptos musicales. El uso de un HMM para el reconocimiento de acordes es una técnica bien establecida en *Music Information Retrieval*: modela la progresión armónica como una cadena de estados ocultos (los acordes reales) que emiten observaciones ruidosas (el chroma medido), y el algoritmo de Viterbi (o similar) encuentra la secuencia de estados más probable dado el chroma observado completo, aprovechando el contexto temporal completo en vez de decisiones instantáneas independientes.

midi_test_generators.py — Generadores de MIDIs sintéticos multi-pista para pruebas

Propósito. Genera diez MIDIs sintéticos de referencia (cinco pistas cada uno: melodía, contra-melodía, armonía, bajo, acompañamiento), cubriendo diez estilos y formas musicales distintas, específicamente diseñados como conjunto de pruebas para validar `midi_pianoroll_analyzer.py` y, por extensión, cualquier otra herramienta de análisis del ecosistema.

Contenido. El catálogo cubre deliberadamente una amplia variedad formal y estilística para ejercitar distintos algoritmos de análisis: **rondó** A-B-A-C-A en Re menor (para probar segmentación formal con retorno de sección), **blues** de doce compases con tres *choruses* en La menor (para probar detección de forma cíclica y armonía de blues), **vals** 3/4 con forma A-B-A'-Coda en La bemol mayor (compás ternario, variación de la reexposición), **fuga** a cuatro voces en Re menor con sujeto y respuesta (para probar detección de imitación contrapuntística y entradas de voces), **tango** con ritmo de habanera en La menor (síncopa característica), y **jazz modal** en re dórico con forma AABA y tres *choruses* (armonía modal no funcional, forma de canción estándar de jazz).

Conceptos musicales. El propio diseño del conjunto de pruebas es una lección implícita de formas musicales: cada MIDI sintético está elegido para ejercitar una estructura formal o un rasgo armónico/rítmico específico y bien documentado de la práctica compositiva occidental (rondó, blues de doce compases, fuga, forma AABA de jazz), garantizando que cualquier algoritmo de análisis del ecosistema se valide contra ejemplos representativos de la diversidad real de la música que se espera que analice.

midinet_composer.py — Generación de melodía condicionada en acordes mediante GAN bar-a-bar (MidiNet)

Propósito. Reimplementación de **MidiNet** (Yang, Chou & Yang, 2017): una GAN condicional que genera melodía **compás a compás**, condicionada tanto en el acorde activo de ese compás como en el compás anterior ya generado, produciendo continuidad entre compases consecutivos sin necesidad de un modelo recurrente completo.

Estructuras de datos. El generador convolucional recibe tres entradas concatenadas: ruido `z` (100 dimensiones), un vector de acorde `y` de 13 dimensiones (12 clases cromáticas + 1 bit binario mayor/menor — una codificación deliberadamente simple de la calidad armónica) y el compás anterior `prev_x` ya generado (el mecanismo de "condicionamiento en el pasado" propio de MidiNet, que sustituye la recurrencia explícita de una RNN por condicionamiento convolucional directo del compás previo). La salida es un **piano roll binario** de forma (1, 16, 128): 16 pasos temporales (semicorcheas dentro del compás) por 128 alturas MIDI posibles.

Algoritmos. El discriminador es **condicional** (evalúa la coherencia del piano roll generado tanto con el acorde como con el compás anterior, no solo su realismo absoluto). La función de pérdida del generador combina tres términos: pérdida **adversarial** estándar, **feature matching** (en vez de solo intentar engañar al discriminador en su salida final, el generador también se entrena para que las activaciones internas del discriminador ante ejemplos generados coincidan estadísticamente con las de ejemplos reales — una técnica de estabilización de entrenamiento GAN bien documentada, que reduce el colapso de modo) y un término de "matching de medias" adicional. La pérdida del discriminador usa **entropía cruzada binaria (BCE) con suavizado de etiquetas** (*label smoothing*, 0.9 en vez de 1.0 para ejemplos reales) — otra técnica estándar de estabilización que evita que el discriminador se vuelva excesivamente confiado, mitigando gradientes que se desvanecen para el generador.

Conceptos musicales. El condicionamiento explícito en el compás anterior (en vez de en toda la historia mediante una RNN) refleja una simplificación deliberada del original MidiNet: apostar a que la coherencia melódica de compás a compás (continuidad local) captura la mayor parte de la naturalidad percibida, sin necesitar memoria de largo alcance — una hipótesis razonable para melodías populares de estructura relativamente repetitiva.

mix_evolver.py — Algoritmo genético para encontrar la mejor cadena de efectos de mezcla

Propósito. Busca, mediante algoritmo genético, la mejor `Chain` de efectos de `audio_effects.py` para un WAV concreto, puntuada por `audio_reference_scorer.py` contra un corpus de referencia — sin implementar ningún efecto DSP propio: importa directamente `apply_chain` de `audio_effects.py` para renderizar cada candidato y `load_manifest` para saber qué genes (parámetros de efecto) son válidos al mutar/inicializar, ilustrando el patrón de composición modular característico del ecosistema (herramientas de "búsqueda"/"evolución" que reutilizan motores de "ejecución" ya construidos por otras herramientas).

Algoritmos. Cada individuo de la población genética es una cadena completa de efectos (una lista ordenada de `EffectGene`, la misma estructura de `audio_effects.py`); el *fitness* de cada individuo se calcula renderizando el WAV con esa cadena y puntuándolo con la distancia de Mahalanobis de `audio_reference_scorer.py` contra el corpus de referencia (y opcionalmente contra un "floor" de calibración). El algoritmo genético opera sobre este espacio de cadenas de efectos con los operadores estándar (selección, cruce, mutación, elitismo), explorando automáticamente el espacio combinatorio de qué efectos aplicar, en qué orden y con qué parámetros, para maximizar la afinidad tímbrica con el corpus de referencia.

Conceptos musicales/de producción. Automatiza una tarea que un ingeniero de mezcla resolvería por prueba y error guiada por la experiencia (qué cadena de EQ+compresor+reverb hace que esta mezcla suene "como" mis referencias favoritas), convirtiendo la búsqueda en un proceso de optimización exploratoria del espacio combinatorio completo de configuraciones de efectos, en vez de ajuste manual iterativo.

ml_architect.py — Composición estructural dirigida por gramáticas aprendidas de corpus (v8.0)

Propósito. Aprende, de un corpus de MIDIs reales, la gramática estructural típica de ese corpus (secciones, roles instrumentales, transiciones) y genera obras completas nuevas siguiendo esa gramática aprendida, con capacidad de actualización incremental del modelo y exploración sistemática del espacio de resultados.

Algoritmos. `segment` detecta las secciones formales de un MIDI de entrada (compartiendo la técnica de segmentación estructural de `harvester.py/midi_pianoroll_analyzer.py`). `train/update` construyen y actualizan **incrementalmente** un modelo sobre un corpus (añadir MIDIs nuevos sin reentrenar desde cero — importante quando el corpus de referencia crece con el tiempo). `variants` (novedad v8) genera N variantes de una obra, las puntúa automáticamente y las ordena por *score*, presentando al usuario ya solo las mejores en vez de una única salida no comparada. `explore` (novedad v8) realiza un **barrido sistemático** del espacio bidimensional arco-emocional × variedad estructural, con ranking automático de los resultados — permitiendo explorar deliberadamente el *trade-off* entre una obra muy predecible en su arco emocional pero poco variada, y una muy variada pero potencialmente sin arco coherente, en vez de fijar arbitrariamente un único punto de ese espacio.

Conceptos musicales. El barrido arco-vs-variedad formaliza una tensión compositiva real y bien reconocida: la coherencia narrativa de una obra (un arco emocional claro y predecible) compite parcialmente con su interés como material (variedad y sorpresa), y presentar explícitamente ese *trade-off* como un espacio explorable en vez de una única salida "óptima" reconoce que la elección entre ambos extremos es, en última instancia, una decisión estética del usuario.

ml_expander.py — Expansión orquestal de bocetos pianísticos mediante ML entrenado por reducción sintética

Propósito. Aprende la transformación piano→orquesta a partir de partituras orquestales reales, mediante un truco de generación de datos de entrenamiento: **colapsa** (reduce sintéticamente) obras orquestales reales a una versión de piano, generando así pares de entrenamiento (piano_reducido, orquesta_original) sin necesitar un corpus emparejado piano↔orquesta ya existente (que sería muy escaso) — un ejemplo de **generación sintética de datos de entrenamiento por proceso inverso**, análogo en espíritu a como se generan pares de entrenamiento para súper-resolución de imagen degradando artificialmente imágenes de alta resolución.

Estructuras de datos. Tres modelos PyTorch **MLP** independientes (entrenables en CPU, deliberadamente ligeros): `TextureNet` (decide la textura orquestal a partir de *features* de sección/arquetipo y tensión), `InstrumentNet` (decide, dado un rol instrumental, la secuencia concreta de eventos MIDI para ese instrumento) e `ExpressionNet` (decide las curvas de expresión CC1/CC11 —modulación y expresión, los controladores MIDI continuos estándar— a partir de la tensión y el instrumento). El *pipeline* de *features* extrae **31 dimensiones con semántica musical explícita**, reutilizando funciones ya existentes en otras herramientas del ecosistema: `analyze_bars` de `orchestrator.py` (16 métricas por compás con sus gradientes), `score_bass_line` de `bass_line_composer.py` (6 criterios de calidad del bajo), `detect_key` de `piano_expander.py`, más posición relativa, *gap ratio* e histograma de intervalos comprimido.

Algoritmos. Tras la predicción de los tres modelos ML, se aplica una **corrección idiomática post-ML basada en reglas** (no en aprendizaje): reutiliza `chord_tone_category`/`admissible_pcs` de `melody_harmonizer.py`, `_best_candidate`/`_motion_type` de `counterpoint.py`, `classify_articulation` de `orchestrator.py`, y rangos idiomáticos por instrumento de `piano_expander.py` — un patrón híbrido explícito donde el ML decide el contenido general y las reglas deterministas garantizan que el resultado final respete restricciones idiomáticas duras (rango del instrumento, tonos de acorde válidos) que el ML por sí solo no puede garantizar de forma fiable.

Conceptos musicales. El patrón "ML genera, reglas corrigen" refleja una estrategia de diseño pragmática y recurrente en sistemas de composición asistida: el aprendizaje automático es bueno capturando patrones estadísticos complejos (qué textura orquestal es plausible dado un contexto), pero las restricciones duras de viabilidad instrumental (un violín no puede tocar por debajo de su cuerda más grave) se garantizan mejor con reglas explícitas verificables, no dejadas a la esperanza de que el modelo las haya aprendido implícitamente.

ml_inpainter.py — Relleno inteligente de compases vacíos (v2.0)

Propósito. Detecta compases vacíos o de densidad anormalmente baja en una obra MIDI multi-pista y los rellena usando una combinación de análisis de estilo por pista, cadenas de Markov **bidireccionales** y, opcionalmente, un modelo PyTorch autorregresivo entrenado sobre corpus, coordinando múltiples pistas simultáneamente mediante una restricción armónica compartida entre ellas.

Algoritmos. El uso de cadenas de Markov **bidireccionales** (en vez de solo hacia adelante) es la elección algorítmica central: al rellenar un hueco, el sistema tiene información tanto del material que lo precede como del que lo sigue (a diferencia de una generación puramente autorregresiva hacia adelante, que solo puede condicionar en el pasado) — un problema estructuralmente más parecido al *inpainting* de imágenes (rellenar una región dado su contexto completo circundante) que a la generación secuencial estándar, de ahí el nombre de la herramienta tomado directamente de esa analogía. `variants` genera N versiones completas de la pieza rellenada para comparación, en vez de comprometerse con una única solución. El **constraint armónico entre pistas** asegura que, al rellenar un hueco en la pista de bajo mientras la pista de armonía ya tiene contenido en ese compás (o viceversa), el material generado sea consistente armónicamente con lo que ya suena en las demás pistas en ese mismo instante — coordinación entre modelos independientes de pista mediante una restricción compartida, en vez de generar cada pista de forma completamente aislada.

Conceptos musicales. El planteamiento como "inpainting" (tomado directamente de visión por computador) formaliza correctamente la diferencia entre "continuar" una pieza (solo pasado disponible) y "rellenar un hueco interno" (pasado y futuro disponibles como contexto) — dos problemas de generación condicionada con información disponible estructuralmente distinta, que requieren, como aquí, algoritmos de condicionamiento diferentes (autorregresivo unidireccional vs. bidireccional).

modulator.py — Modulación continua y musicalmente informada de parámetros MIDI (v2.0)

Propósito. Aplica curvas de modulación continua (crescendo, arcos de tensión, cambios de densidad, etc.) a parámetros musicales de un MIDI existente —densidad, tensión, dinámica, ritmo, registro, articulación, dispersión (*spread*), contorno, tempo, brillo— de forma "musicalmente informada": consciente de la armonía, la separación de voces y la estructura formal, en vez de aplicar transformaciones puramente matemáticas ciegas al contenido musical.

Algoritmos. Las mejoras de la v2.0 sobre v1.0 son todas instancias del mismo principio general —sustituir una transformación matemática ciega por una musicalmente consciente—: el análisis armónico por compás (tonalidad local vía perfiles Krumhansl) permite que la modulación de un parámetro no rompa la coherencia tonal local; la **separación de voces** (melodía/bajo/armonía tratadas individualmente) permite aplicar la misma curva de modulación con distinta intensidad o fase a cada capa textural; la modulación de **densidad** respeta una rejilla rítmica coherente al añadir/quitar notas de paso (no las coloca en posiciones métricamente arbitrarias); la modulación de **tensión** con conducción de voces planifica de antemano la resolución de las disonancias introducidas (novenas, etc.), en vez de simplemente insertarlas sin resolución; el **scope por voz con offset configurable** permite escalonar (*stagger*) la misma curva entre voces distintas (p. ej. que el crescendo llegue ligeramente antes en la voz superior); la **detección de estructura formal** hace que la modulación respete cadencias y cierres de frase, evitando que una curva ciega corte una frase musical a mitad de su cadencia; y el **groove map real** amplifica micro-*timings* ya presentes en el MIDI fuente en vez de imponer una humanización genérica. Se ofrecen doce formas de curva (`linear`, `exponential`, `logarithmic`, `sigmoid`, `arc`, `arc_down`, `s_curve`, `pulse`, `plateau`, `wave`, `random_walk`, `custom`), cada una con una forma temporal característica reutilizable para cualquiera de los nueve parámetros modulables.

Conceptos musicales. La lista de nueve parámetros modulables junto con doce formas de curva define, en efecto, un pequeño lenguaje declarativo de automatización musical (similar en espíritu a la automatización de parámetros en un DAW, pero operando sobre contenido MIDI simbólico con consciencia armónica en vez de sobre audio o control MIDI CC crudo), donde cada combinación parámetro×curva codifica un gesto expresivo distinto (p. ej. densidad con curva `arc` = crescendo-decrescendo de textura).

morpher.py — Morphing gradual entre dos obras MIDI

Propósito. Dado A.mid y B.mid, genera N pasos intermedios donde el "ADN" musical de A se transforma gradualmente en B — a diferencia de una transferencia de estilo puntual (un salto directo A→B), produce una secuencia continua de estados intermedios explorables.

Algoritmos. Seis dimensiones de morphing, controlables independientemente: contorno melódico y secuencia de notas, patrón rítmico/groove, progresión y complejidad armónica, curvas emocionales (tensión/activación/valencia), envolvente dinámica de velocidades, y tempo — permitiendo, por ejemplo, morphear la armonía rápidamente mientras el ritmo cambia gradualmente. Cuatro modos de interpolación temporal disponibles para cada dimensión: `linear` (tasa de cambio constante), `sigmoid` (cambio acelerado en el centro del recorrido, suave en los extremos — la forma de curva más perceptualmente "natural" para una transición, evitando el salto brusco al inicio/fin de una interpolación lineal), `exponential` (cambio lento al principio, rápido al final) y `sinusoidal` (oscilación periódica en vez de monótona).

Conceptos musicales. Separar el morphing en seis ejes independientes con curvas temporales propias por eje permite composiciones de transición mucho más sofisticadas que una simple mezcla lineal de dos piezas: p. ej. una transición donde el ritmo cambia de forma abrupta (sigmoide muy pronunciada) pero la armonía evoluciona con una exponencial lenta, simulando una modulación tonal gradual bajo un cambio de textura rítmica repentino.

mosaic_composer.py — Collage y ensamblaje dramático de fragmentos musicales (v2.0)

Propósito. Segmenta un corpus completo de MIDIs en fragmentos etiquetados, los indexa por función dramática, tensión, estilo y compatibilidad de "costura", y los reensambla creativamente siguiendo un arco narrativo explícito — con el resultado deliberadamente **irreconocible** respecto a las fuentes originales, no una simple concatenación de citas.

Algoritmos. Pipeline de siete etapas orquestando varias otras herramientas del ecosistema: (1) **HARVEST**, segmentación del corpus (con `harvester.py` o internamente); (2) **INDEX**, etiquetado de cada fragmento por tensión, rol dramático, tipo de cadencia y estilo; (3) **PLAN**, diseño del arco dramático completo como una secuencia de "slots" con función y tensión objetivo; (4) **SELECT**, para cada slot se eligen **dos** fragmentos de fuentes distintas (no uno); (5) **BLEND**, fusión genuina de sus "ADNs" (des-identificación del material original + injerto + mezcla vía `midi_dna_unified.py`) — melodía de uno, ritmo del otro, por ejemplo, en vez de alternar bloques completos de cada fuente; (6) **STITCH**, unión de los slots consecutivos mediante puentes de `morpher.py` o transiciones tonales explícitas; (7) **POLISH**, reharmonización final y/o unificación de estilo mediante `style_transfer`.

Conceptos musicales. La técnica de fusionar deliberadamente "melodía de A + ritmo de B" en vez de alternar bloques enteros de material citado formaliza el concepto de *collage compositivo genuino* (frente al *pastiche* por yuxtaposición): el objetivo explícito es que el resultado sea nuevo material derivado, no una simple recombinación reconocible de sus fuentes — un principio estético distinto del muestreo/sampling literal.

motif_table_v2.py — Catálogo curado de 100 motivos y frases melódicas (v2.0)

Propósito. Complemento melódico de `chord_table.py` y `rhythm_table.py`: en vez de progresiones de acordes o patrones rítmicos, ofrece un catálogo curado de 100 gestos melódicos (motivos cortos de 2-5 notas, y frases más largas de 6-9 notas con arco antecedente-consecuente), descritos por grados de escala y etiquetados con tensión (escala 1-10), contorno y emoción, buscables y exportables.

Estructuras de datos. Tres tipos de entrada catalogados con semántica distinta: **motivo** (célula corta, material de construcción), **frase** (gesto largo con arco pregunta-respuesta completo, lista para insertarse directamente), y **escala** (recorrido modal completo, material de referencia/estudio sin forma retórica propia, a diferencia de una frase). Ocho modos (mayor, menor, dórico, frigio, frigio dominante, lidio, mixolidio, armónico menor) y siete tipos de contorno explícito (ascendente, descendente, arco, ondulante, zigzag, salto, estático) categorizan cada entrada.

Conceptos musicales. La distinción explícita entre “motivo” (material bruto de construcción, para desarrollarse), “frase” (unidad retórica completa lista para usarse) y “escala” (material de estudio sin función retórica) formaliza tres niveles distintos de organización melódica reconocidos en la teoría de la forma musical: la célula, la frase, y el material escalar puramente técnico.

mscz2vec.py — Vectorización perceptual de partituras (7.472 líneas)

Propósito. El motor de vectorización musical central del ecosistema (invocado como subproceso por `analyzer.py`, `counterexample.py` y otras herramientas): analiza una partitura (MusicXML, MSCZ, MIDI) y extrae vectores de características a múltiples niveles perceptuales simultáneamente.

Estructuras de datos. Doce familias de vectores disponibles, cada una con su propia dimensionalidad y semántica: `unified` (vector 6D perceptual unificado, el resumen por defecto usado por `analyzer.py`), `melodic` (histograma de intervalos + estadísticas de pitch), `harmonic` (funciones armónicas y progresiones), `rhythmic` (densidad, patrones, n-gramas rítmicos), `instrumental` (familias de instrumentos y orquestación), `motif` (motivos y estructura formal vía Sequitur, compartiendo el algoritmo con `harvester.py`/`grammar_composer.py`), `form` (vector de estructura formal), `emotion` (vector semántico-emocional completo), `tension` (perfil de tensión armónica), `stability` (perfil de estabilidad tonal), `novelty` (vector de novedad estructural, relacionado con la curva de novedad SSM de `harvester.py`) y `entropy` (entropía y sorpresa informacional de la secuencia).

Conceptos musicales. El fichero es, en efecto, la "API" de traducción entre música simbólica y espacio vectorial que consume el resto del ecosistema: cualquier herramienta que necesite comparar, agrupar o medir similitud entre obras (analyzer, counterexample, evolver a través de midi_dna_unified) delega en última instancia en las representaciones vectoriales que produce este fichero, haciendo de él la pieza de infraestructura compartida más reutilizada de todo el proyecto.

muscriptor_transcribe.py — Transcripción de audio a MIDI con transformer decoder-only (MuScriptor)

Propósito. Puerto de un único fichero del modelo **MuScriptor** (Kyutai × Mirelo, CC BY-NC 4.0): transcribe una grabación de audio real a notas MIDI por instrumento, usando un transformer decoder-only que decodifica un flujo de tokens al estilo **MT3** (Multi-Task Multitrack Music Transcription, Google Magenta), con soporte de *streaming* por fragmentos (*chunks*) mediante un mecanismo de "tie-prologue" entre ellos.

Estructuras de datos. Tres tamaños de modelo disponibles (small: 103M parámetros/14 capas, medium: 307M/24 capas —por defecto—, large: 1.4B/48 capas, que requiere GPU), todos con la misma arquitectura decoder-only pero escala creciente. El "tie-prologue" es el mecanismo que permite procesar audio largo en fragmentos manejables sin perder continuidad de notas que cruzan la frontera entre dos *chunks* consecutivos: el prólogo de un fragmento incluye contexto del final del fragmento anterior, de modo que una nota que empezó justo antes del corte se reconoce correctamente como la misma nota sostenida y no como dos eventos independientes.

Algoritmos. La adaptación deliberadamente **reduce el alcance** del proyecto original: el repositorio de MuScriptor completo incluye también exportación a partitura vía MuseScore y un servidor FastAPI con visualización de piano roll en vivo, ambos **excluidos explícitamente** en esta adaptación "por no encajar con el estilo mutopia" (fichero único, CLI simple, sin servidor) — un ejemplo explícito de la disciplina arquitectónica que se autoimpone el ecosistema incluso al adaptar proyectos externos con más funcionalidad disponible.

Conceptos musicales. La tarea de transcripción automática de audio polifónico a notas discretas por instrumento (piano transcription, más generalmente *multitrack music transcription*) es uno de los problemas centrales y más difíciles del MIR (Music Information Retrieval): requiere separar simultáneamente qué instrumento suena, qué nota, cuándo empieza y cuándo termina, a partir de una única señal de audio mezclada.

musegan.py — Generación multi-pista mediante GAN sobre piano rolls binarios (BinaryMuseGAN, PyTorch)

Propósito. Reimplementación de **BinaryMuseGAN** (Dong et al., ISMIR 2018): genera música multi-pista (batería, bajo, guitarra, cuerdas, piano) directamente como piano rolls binarios de cinco dimensiones, mediante una GAN entrenada con la pérdida de Wasserstein.

Estructuras de datos. El piano roll es un tensor 5D: (lote, compases, pasos-de-tiempo, alturas, pistas), con valores por defecto de 4 compases, 48 pasos temporales por compás, 84 alturas (rango C1-B7) y 5 pistas fijas (batería, bajo, guitarra, cuerdas, piano) — una representación tensorial explícita del piano roll multi-instrumento como "imagen" 3D+canal, análoga a un volumen de vídeo con una dimensión de "instrumento" adicional a las de tiempo y altura.

Algoritmos. El generador comparte una **deconvolución 3D** (*transposed convolution*) inicial entre todas las pistas, seguida de ramas separadas por pista con dos orientaciones de procesamiento complementarias —"pitch-time" y "time-pitch"— que capturan patrones tanto a lo largo del eje temporal como del eje de alturas para cada instrumento individualmente. El discriminador usa ramas simétricas por pista más dos *streams* adicionales especializados: uno de **chroma** (resumen armónico agregado sobre las 12 clases de altura, ignorando la octava, para juzgar coherencia armónica global) y uno de **onset/offset** (patrón de ataques y finales de nota, para juzgar coherencia rítmica). El entrenamiento usa la pérdida **WGAN-GP** (Wasserstein GAN con penalización de gradiente): sustituye la divergencia JS del GAN original (que puede saturarse y no dar gradiente útil cuando las distribuciones no se solapan) por la distancia de Wasserstein, estimada mediante un discriminador (llamado "crítico" en la formulación Wasserstein) restringido a ser 1-Lipschitz, aquí impuesto mediante penalización del gradiente en vez del recorte de pesos del WGAN original — la variante de entrenamiento GAN más estable de la literatura para datos estructurados discretos/binarios como un piano roll.

Conceptos musicales. Los dos *streams* discriminadores especializados (chroma y onset/offset) formalizan explícitamente que la "coherencia musical" de un piano roll multi-pista generado depende tanto de su coherencia armónica global (qué notas suenan juntas, independientemente de su ritmo exacto) como de su coherencia rítmica (cuándo atacan y terminan las notas, independientemente de su altura exacta) — dos aspectos parcialmente independientes que un único discriminador genérico podría no aprender a evaluar por separado con la misma eficacia.

music_transformer.py — Generación con estructura a largo plazo mediante atención relativa (Huang et al. 2019)

Propósito. Reimplementación del **Music Transformer** original de Google Magenta (Huang et al., 2019): un Transformer autorregresivo decoder-only capaz de generar música con coherencia estructural a largo plazo (hasta 2048 eventos MIDI), gracias a su mecanismo de **atención relativa**.

Estructuras de datos. La representación es *event-based* (no una rejilla temporal fija), con un vocabulario de 391 tokens: 128 para note-on, 128 para note-off, 32 niveles cuantizados de *velocity*, 100 para incrementos de tiempo, más tres tokens especiales (padding, inicio y fin de secuencia) — el esquema de tokenización canónico introducido por el paper original de Performance RNN/Music Transformer, ampliamente reutilizado (con variaciones) en otras herramientas del ecosistema.

Algoritmos. La innovación central del paper original, replicada aquí, es la **atención relativa con el "truco de skewing"** (*relative attention with skewing trick*): la atención estándar de un Transformer calcula compatibilidad entre posiciones absolutas, pero para música (donde los patrones se repiten en relaciones *relativas* de posición, no en posiciones absolutas concretas) es más útil una atención que dependa de la **distancia** entre dos posiciones, no de sus índices absolutos. Calcular esto de la forma ingenua requeriría una matriz de sesgos de tamaño $O(N^2D)$ (N=longitud de secuencia, D=dimensión), prohibitivamente costosa para secuencias largas; el truco de *skewing* consiste en una reordenación matricial ingeniosa (desplazar cada fila de una matriz intermedia un número de posiciones distinto, después recortar) que permite calcular exactamente el mismo resultado con complejidad $O(ND)$ — una reducción de un orden de magnitud en la dimensión de secuencia, precisamente lo que permite escalar a las 2048 posiciones de contexto que dan nombre a la capacidad "a largo plazo" del modelo.

Conceptos musicales. La atención relativa captura directamente una propiedad estructural fundamental de la música: la repetición y variación de patrones a intervalos temporales relativos (un motivo que reaparece "4 compases después", independientemente de en qué compás absoluto se encuentre la pieza) es mucho más común y perceptualmente relevante que relaciones de posición absoluta, y el mecanismo de atención relativa modela esto de forma nativa en vez de tener que aprenderlo indirectamente de datos como tendría que hacer un Transformer con codificación posicional absoluta estándar.

musiclm.py — Generación de música a partir de texto (MusicLM, pipeline completo)

Propósito. Implementación de **MusicLM** (Agostinelli et al., Google, 2023): combina **MuLaN** (aprendizaje contrastivo audio-texto al estilo CLIP) con la arquitectura en cascada de **AudioLM** (ya presente como `audiolm.py` en el ecosistema) para generar música a partir de descripciones en lenguaje natural libre.

Estructuras de datos. MuLaN aprende un **espacio de embedding compartido** entre audio y texto mediante aprendizaje contrastivo (pares audio-texto correspondientes se acercan en el espacio, pares no correspondientes se alejan — exactamente la misma técnica que CLIP para imagen-texto, aquí aplicada al dominio audio-texto). Un **cuantizador vectorial residual (RVQ)** se ajusta posteriormente sobre los embeddings de audio de MuLaN ya entrenado, discretizándolos en tokens — la misma técnica RVQ usada en el códec de `audiolm.py`, aquí aplicada no al audio crudo sino directamente al espacio semántico aprendido por MuLaN.

Algoritmos. El flujo completo encadena seis etapas: entrenamiento contrastivo de MuLaN sobre pares (audio, texto); ajuste del cuantizador residual sobre los embeddings de audio de MuLaN entrenado; extracción de embeddings (de audio o de texto — ambos viven en el mismo espacio tras el entrenamiento contrastivo, permitiendo usar el embedding de un texto como si fuera el embedding de un audio "objetivo" nunca escuchado); generación de música condicionada en la descripción de texto, delegando en la arquitectura en cascada de tres transformers de AudioLM (semántico→grueso→fino) pero condicionando cada etapa en los tokens MuLaN derivados del texto en vez de (o además de) en un audio de referencia; y un modo de diagnóstico *round-trip* que mide la similitud audio↔embed sin generar nada, para verificar que el espacio contrastivo aprendido efectivamente relaciona sonidos con sus descripciones textuales.

Conceptos musicales. El principio de MusicLM es que un espacio de embedding compartido audio-texto (MuLaN), entrenado sobre una tarea puramente discriminativa (¿este audio y este texto corresponden?), puede reutilizarse después como **condicionamiento** para una tarea completamente distinta (generación), simplemente porque el texto y el audio correspondiente ya viven cerca en ese espacio — una técnica de condicionamiento por espacio compartido mucho más flexible que entrenar un modelo texto→audio de extremo a extremo directamente sobre pares etiquetados (que serían mucho más escasos que pares audio-texto naturales de internet).

mutator.py — Mutación semántica de MIDIs guiada por lenguaje natural

Propósito. Traduce instrucciones libres en lenguaje natural (“más oscuro”, “añade urgencia en el clímax”, “que suene como lluvia”) en parámetros concretos del pipeline `midi_dna_unified.py` y las aplica sobre un MIDI existente, con verificación automática de que el resultado efectivamente se movió en la dirección semántica pedida.

Algoritmos. El flujo de seis etapas es notable por su **bucle de verificación cerrado**: (1) lee el MIDI fuente y calcula su “*fingerprint*” (estado vectorial actual); (2) parsea la instrucción combinando un **diccionario semántico** interno (mapeo directo de frases comunes a deltas de parámetros) con un **LLM como fallback/enriquecimiento** para instrucciones no cubiertas por el diccionario; (3) resuelve los deltas de parámetro como cambios **relativos** al estado actual (no valores absolutos — “más oscuro” desplaza el estado actual hacia lo oscuro, no lo fija a un valor de oscuridad predeterminado); (4) aplica los parámetros resultantes delegando en `variation_engine.py`/`morpher.py`/herramientas de tensión, reutilizando la maquinaria de transformación ya construida en vez de reimplementarla; (5) **verifica con `mscz2vec.py`** que el resultado se movió efectivamente en la dirección semántica pedida en el espacio vectorial (un chequeo de que la instrucción se cumplió, no solo que se ejecutó sin error); (6) guarda un **historial de mutaciones en forma de árbol**, reversible — permitiendo deshacer una mutación concreta o explorar ramas alternativas de edición desde cualquier punto del historial, como un sistema de control de versiones aplicado a la edición semántica de una obra.

Conceptos musicales. El paso de verificación (4→5) es conceptualmente el más interesante: convierte una instrucción subjetiva en lenguaje natural en una hipótesis vectorial verificable (“más oscuro” implica un desplazamiento medible en ciertas dimensiones del vector perceptual de `mscz2vec`), cerrando el bucle entre intención expresada en lenguaje y resultado medible, en vez de confiar ciegamente en que la transformación aplicada logró el efecto pedido.

narrator.py — Diseñador de arquitectura narrativa para obras musicales

Propósito. Define la estructura dramática completa de una obra **antes** de generar ningún MIDI, actuando como "director de orquesta" de todo el pipeline: convierte una narrativa emocional (en lenguaje natural o mediante GUI) en forma musical concreta (secciones A/B/C, duraciones, cadencias), curvas exportables a `tension_designer.py`, un plan de ejecución completo (`plan.yaml`, consumido por `runner.py`) y un mapa tonal explícito.

Estructuras de datos. Siete **arcos narrativos predefinidos**, cada uno una plantilla de estructura dramática ya asociada a una forma musical clásica: `hero` (llamada → prueba → clímax → resolución, el monomito campbelliano aplicado a estructura musical), `tragedy` (ascenso → clímax → caída abrupta), `romance` (presentación → tensión → unión → reafirmación), `mystery` (ambigüedad → revelaciones → crisis → resolución), `meditation` (plano → ondulaciones sutiles → retorno al reposo, sin arco dramático fuerte), `rondo` (tema A reiterado con episodios contrastantes — mapeado directamente a la forma rondó clásica) y `sonata` (exposición → desarrollo → reexposición, la forma sonata clásica completa).

Algoritmos. El fichero funciona como capa de planificación de más alto nivel del ecosistema: en vez de generar contenido musical directamente, produce las **especificaciones** (curvas de tensión, mapa tonal, estructura formal, `plan.yaml`) que consumen otras herramientas generativas (`tension_designer.py`, `cadence_designer.py`, `runner.py`), separando explícitamente la decisión de "qué forma dramática tendrá la obra" de "cómo se genera el contenido concreto para cada sección" — una separación de responsabilidades entre planificación de alto nivel y ejecución de bajo nivel.

Conceptos musicales. El mapeo directo entre arcos narrativos literarios (el viaje del héroe, la tragedia, el misterio) y formas musicales clásicas (rondó, sonata) formaliza la intuición, común en la crítica musical, de que las formas musicales tradicionales codifican en sí mismas arquetipos narrativos — la forma sonata como un "viaje" con partida (exposición), conflicto (desarrollo) y retorno transformado (reexposición).

openmusic.py — Text-to-Music con QA-MDT (Quality-Aware Masked Diffusion Transformer)

Propósito. Genera audio musical de alta calidad directamente a partir de una descripción textual, mediante una reimplementación autocontenida de **QA-MDT** (Quality-Aware MDT), sin depender de ningún otro script del ecosistema.

Estructuras de datos/Arquitectura. El pipeline encadena un **VAE de espectrograma Mel** (comprime el audio a un espacio latente de menor dimensión, igual que en `audio_diffusion_cli.py`), un modelo de **difusión latente** sobre ese espacio usando la arquitectura **MDT/DiT** (*Diffusion Transformer*, 28 capas — sustituye la U-Net convolucional habitual en difusión de imagen por un Transformer puro, la tendencia arquitectónica más reciente en modelos de difusión de gran escala) y un **vocoder HiFi-GAN** para convertir el espectrograma final de vuelta a forma de onda de audio. El condicionamiento textual usa **Flan-T5-large** (vía atención cruzada, el mismo patrón de condicionamiento textual que Stable Diffusion usa con CLIP/T5) más un **token de calidad MOS** explícito (Mean Opinion Score, la métrica estándar de evaluación subjetiva de calidad de audio) inyectado como condición adicional — permitiendo, en teoría, pedir explícitamente al modelo que genere con una calidad percibida objetivo, aprovechando que el corpus de entrenamiento típicamente mezcla grabaciones de calidad heterogénea y el modelo puede aprender a asociar el token MOS con las características que distinguen grabaciones de alta vs. baja fidelidad.

Algoritmos. El comando `describe` usa un LLM o heurísticas para traducir una "intención libre" del usuario (lenguaje coloquial, posiblemente vago) a un **prompt técnico** más estructurado y efectivo para el modelo de difusión — reconociendo que los modelos de difusión texto-audio, como los de texto-imagen, suelen responder mejor a prompts con vocabulario técnico específico (instrumentación, género, tempo) que a descripciones puramente coloquiales.

Conceptos musicales. El **token de calidad MOS** es la contribución distintiva de QA-MDT frente a modelos de difusión texto-audio más simples: permite que el modelo aprenda de un corpus de calidad heterogénea (mezclas profesionales junto a grabaciones amateur) sin que la generación resultante "regrese a la media" de calidad del corpus completo, porque el modelo puede condicionar explícitamente en "quiero calidad alta" en el momento de generar.

orchestral_colorist.py — Color tímbrico emocional para obras MIDI

Propósito. Mientras `orchestrator.py` asigna instrumentos de forma mecánica/idiomática (según rango y textura), este fichero lo hace por **semántica emocional**: cada emoción define una paleta orquestal completa (instrumento por rol, registro objetivo, rango dinámico) extraída de convenciones reales de la práctica orquestal — melancolía → violonchelo en registro medio, fragilidad → flauta sola *pianissimo* con arpa, angustia → registros extremos con trémolo de cuerda, triunfo → metales *fortissimo*.

Algoritmos. Pipeline de cuatro etapas: (1) detección de rol por pista (melodía/bajo/armonía/contrapunto/per —canal 10, el estándar General MIDI para batería); (2) selección de la paleta —instrumento General MIDI, registro y dinámica— según la emoción solicitada; (3) aplicación mediante *program change* MIDI, **transposición de octavas hacia el registro objetivo mientras se permanece dentro del rango físico real del instrumento** (validado contra el catálogo de rangos idiomáticos de `playability_auditor.py`, evitando asignar a un instrumento notas fuera de su tesitura físicamente ejecutable) y reescalado de las velocidades de nota al rango dinámico de la paleta; (4) informe tabular de las asignaciones realizadas.

Conceptos musicales. El catálogo de asociaciones emoción→timbre codifica convenciones orquestales bien establecidas en la práctica compositiva romántica y de música de cine (el violonchelo como voz de melancolía introspectiva, el trémolo de cuerda grave como recurso estándar de tensión/angustia, los metales en *fortissimo* como signo convencional de triunfo), formalizando explícitamente un vocabulario semántico-tímbrico que un orquestador humano aplicaría por convención estilística aprendida, no por regla mecánica de rango instrumental.

orchestrator.py — Orquestación automática y percusión para mocks orquestales

Propósito. Lee el MIDI de `stitcher.py` junto con sus *fingerprints* JSON y genera un MIDI multi-pista listo para importar en FL Studio con distribución de material a una plantilla orquestal completa (cuerdas, maderas, metales, percusión), *keyswitches*, automatización de expresión y un informe de sesión.

Algoritmos. La distribución de material por sección orquestal se basa en el rol detectado de cada línea (melodía, contramelodía, bajo, armonía). Los **keyswitches** —el mecanismo estándar de las librerías de samples orquestales virtuales para cambiar de técnica de ejecución (legato, staccato, pizzicato…) mediante notas MIDI fuera de rango dedicadas a ese propósito— se seleccionan automáticamente según la duración y el contexto de cada nota (`classify_articulation`, reutilizada por `ml_expander.py`). El **CC1** (control de dinámica de *crossfade* entre capas de sample) se interpola directamente desde la curva de tensión del *fingerprint* de la obra, y el **CC11** (expresión) recibe inflexiones automáticas de frase — ambos automatismos MIDI estándar de producción orquestal virtual, generados aquí algorítmicamente en vez de dibujarse a mano en un DAW. El registro de cada línea se ajusta idiomáticamente (transposición de octavas cuando cae fuera del rango físico real del instrumento).

Conceptos musicales. El fichero automatiza el trabajo típicamente manual y tedioso de un compositor de música para medios preparando un *mock* (maqueta orquestal con samples) para escucha previa: asignación de instrumentos, articulaciones y automatización de expresión — tareas que consumen gran parte del tiempo de producción en composición para cine/videojuegos y que aquí se derivan directamente del contenido musical y de su curva de tensión ya calculada.

part_extractor.py — Extracción de partes instrumentales individuales con transposición

Propósito. Cubre un hueco explícito del ecosistema: mientras varias herramientas generan MIDIs orquestales multi-pista, ninguna producía las **partes individuales** que necesita cada intérprete real para leer su música. Separa cada pista en su propio MIDI autónomo, aplicando la transposición del instrumento transpositor correspondiente, y reporta rango y compases de silencio (*tacet*).

Algoritmos. Es la operación inversa exacta de `piano_reducer.py` (que fusiona voces en dos pentagramas) y de `midi_merge.py` (que combina stems en uno): aquí se **divide** una partitura multi-pista en partes de lectura individual. Las transposiciones estándar de instrumentos transpositores están codificadas explícitamente (trompeta/clarinete/saxofón en si♭: escrito = sonante + 2 semitonos; trompa/corno inglés en fa: +7 semitonos; flautín y contrabajo con transposición de octava), siguiendo la convención universal de notación orquestal donde el intérprete lee una nota distinta de la que realmente suena, para que la digitación sea uniforme entre instrumentos de la misma familia afinados en tonalidades distintas.

Conceptos musicales. La transposición de instrumentos transpositores es una convención centenaria de la notación orquestal occidental, motivada históricamente por la construcción física de los instrumentos (el largo del tubo determina su tonalidad fundamental) y mantenida por tradición pedagógica (una misma digitación produce el mismo patrón de lectura en toda la familia de instrumentos, independientemente de su afinación de transposición concreta) — automatizarla correctamente es indispensable para que una parte impresa sea utilizable por un músico real.

performer.py — Renderizado expresivo de interpretación para MIDI cuantizado

Propósito. Los compositores algorítmicos del ecosistema producen MIDI cuantizado y de velocidad uniforme; esta herramienta reconoce explícitamente que gran parte de la emoción percibida en una interpretación musical no está en *qué* notas suenan sino en *cómo* suenan, aplicando reglas interpretativas locales inspiradas en las **reglas KTH / Director Musices** (el sistema de reglas de interpretación expresiva desarrollado en el KTH Royal Institute of Technology de Estocolmo, una de las referencias académicas más consolidadas de la investigación en interpretación musical computacional) como último paso del pipeline antes del renderizado a audio.

Algoritmos. Siete reglas aplicadas de forma consciente de la estructura de frase (no globalmente uniforme): **rubato** por frase (arco agógico: arranque contenido, aceleración hacia el pico, *ritardando* en el cierre, más un *ritardando* final de la pieza completa); **agógica** (alargamiento temporal deliberado de la nota que constituye el pico melódico de cada frase — un recurso interpretativo estándar para señalar el punto culminante); **dinámica** (crescendo hacia el pico de frase, combinado con una jerarquía métrica que refuerza los tiempos fuertes y atenúa los contratiempos, más un *jitter* aleatorio de pequeña magnitud para evitar mecanicidad perfecta); **lead** (la melodía se adelanta unos 15 ms respecto al acompañamiento — el fenómeno bien documentado de que los intérpretes humanos, especialmente pianistas, tienden a anticipar ligeramente la voz melódica sobre el acompañamiento, perceptualmente deseable e imperceptible como "error"); **humanize** (microdesincronización aleatoria de los ataques, evitando la precisión robótica de un cuantizado perfecto); **pedal** (control MIDI 64, con retoma automática en cada cambio de armonía — la técnica pianística estándar de cambiar el pedal de sustain para evitar que armonías sucesivas se mezclen sonoramente); y **swell** (arco de expresión CC11 dentro de notas largas sostenidas, más automatización de CC1).

Conceptos musicales. El fichero es una implementación computacional directa de un modelo académico consolidado de interpretación musical expresiva, formalizando precisamente los micro-ajustes de tiempo, dinámica y articulación que distinguen una interpretación "humana" convincente de una ejecución mecánicamente literal de la partitura — el problema central de la síntesis de interpretación expresiva en informática musical.

phrase_builder.py — Motor de sintaxis melódica: antecedente/consecuente

Propósito. Toma un motivo corto (2-4 compases, idealmente extraído por `harvester.py`) y lo expande en frases completas de 4, 8 o 16 compases aplicando la lógica sintáctica clásica de la frase musical occidental: antecedente→consecuente, pregunta→respuesta, semicadencia→cadencia auténtica — el propio docstring señala que el resultado suena "inevitablemente correcto" porque sigue la gramática con la que el oído occidental está entrenado desde la infancia musical.

Algoritmos. El concepto central formaliza la frase musical como un **par estructurado**: el antecedente (A) presenta la idea y termina en tensión (semicadencia, sobre V); el consecuente (C) responde a esa idea y termina en reposo (cadencia auténtica, sobre I). Cuatro tipos de relación A→C, cada uno asociado a una práctica compositiva histórica concreta: `parallel` (C es casi idéntico a A salvo la cadencia final — el modelo bachiano de "pregunta y respuesta casi idéntica"), `contrasting` (C introduce material nuevo pero comparte el mismo final — el modelo clásico vienés de Haydn/Mozart), `sequential` (C transpone literalmente el material de A a otro grado — el recurso de la secuencia armónica) y `developmental` (C desarrolla motívicamente el material de A en vez de repetirlo o transponerlo literalmente — el modelo beethoveniano de desarrollo temático inmediato).

Conceptos musicales. La taxonomía de cuatro tipos de relación antecedente-consecuente es, en efecto, una breve historia comprimida de la evolución de la técnica de frase en la música tonal occidental —de la simetría casi literal del barroco a la elaboración desarrollante del clasicismo tardío— formalizada aquí como cuatro estrategias generativas intercambiables sobre el mismo par estructural A→C.

piano_autocomplete.py — Autocompletado de interpretaciones de piano en tiempo real

Propósito. Inspirado en el proyecto "Training a 125M-parameter Model to Autocomplete Piano": un Transformer decoder-only pequeño que continúa una interpretación pianística nota a nota en tiempo real, aprendida sobre datos de interpretación reales (no partitura cuantizada), con post-entrenamiento por preferencias.

Estructuras de datos. La representación fundamental es `NOTE(pitch, delta_onset, duration, velocity)`: **una nota completa es un único paso autorregresivo**, no un token por campo como en la tokenización event-based de `music_transformer.py` — cada nota tiene cinco campos categóricos (tipo de evento, altura, delta de tiempo desde la nota anterior, duración, velocidad), cada uno con su propio *embedding*, y el token final de entrada a la red es la **suma** de los cinco embeddings. Un pequeño **"nested decoder"** predice los cinco campos **en cadena** dentro del mismo paso autorregresivo (evento→altura→delta→duración→velocidad, cada campo condicionado explícitamente en los ya predichos de esa misma nota) — una factorización autorregresiva de grano fino *dentro* de cada paso autorregresivo de grano grueso, que permite modelar dependencias entre campos de la misma nota (p. ej., la velocidad probable depende de qué altura se acaba de decidir) sin necesitar una secuencia de tokens separados por campo. El **pedal de sustain** deliberadamente **no** se modela como un evento MIDI aparte: se "hornea" directamente en la duración de la nota durante el preprocesado (la nota se extiende hasta que el pedal se suelta o el mismo *pitch* se vuelve a tocar), simplificando la representación a costa de perder la información explícita de cuándo se pisó/soltó el pedal físicamente.

Algoritmos. La arquitectura del Transformer usa **RMSNorm** (normalización más simple y barata que LayerNorm, que prescinde de restar la media, usada en arquitecturas modernas como LLaMA), **atención causal** estándar y **SwiGLU** como función de activación de las capas *feed-forward* (una variante *gated* de GLU con SiLU, también popularizada por LLaMA/PaLM, generalmente superior a ReLU/GELU en modelos de lenguaje modernos) — la misma receta arquitectónica de los LLMs de texto contemporáneos, aplicada aquí a secuencias de notas de piano. El post-entrenamiento usa **DPO** (*Direct Preference Optimization*): en vez de entrenar un modelo de recompensa separado y optimizar contra él vía RL (como en RLHF clásico), DPO optimiza directamente la política sobre pares de continuaciones comparadas, usando métricas heurísticas locales como **juez automático** — un sustituto sin conexión externa del "Gemini pairwise judge" que el proyecto original habría usado, reemplazado aquí por heurísticas propias para mantener el fichero autocontenido.

Conceptos musicales. Modelar el pedal implícitamente en la duración de la nota (en vez de como evento explícito) es una simplificación de representación que sacrifica fidelidad física (el pedal afecta también a notas no directamente pulsadas por resonancia simpática) a cambio de una tokenización mucho más compacta y directamente aprovechable por un modelo de secuencia estándar.

La familia “piano_duo_analyzer”: visualizador HTML de piano a dos manos con análisis armónico (v2.0 → v5)

Cuatro versiones (base v2.0, v3, v4, v5) del mismo visualizador: toma un MIDI de dos pistas (mano izquierda/derecha) y genera un informe HTML interactivo con tres filas sincronizadas — piano roll de la mano derecha coloreado según la posición de cada nota dentro del acorde detectado en la mano izquierda en ese instante (azul: nota fundamental/más grave del acorde; amarillo, verde, blanco: resto de notas del acorde; rojo: nota que no pertenece al acorde, es decir, disonancia u ornamento), piano roll de la mano izquierda, y una fila de análisis armónico funcional.

Evolución de v2 a v5. Las firmas de función capturadas muestran que `build_chord_timeline` gana dos parámetros nuevos entre la versión base y la v5 — `arp_fusion` (probablemente fusión de notas de un acorde arpegiado disperso en el tiempo en un único evento de acorde lógico, para que un arpegio en la mano izquierda se reconozca como “el mismo acorde” en vez de como notas aisladas sin contexto armónico) y `both_hands_bars` (extensión del análisis para considerar ambas manos conjuntamente en ciertos compases, útil cuando la textura no se divide limpiamente por mano) — y `chord_position_for_note` gana un parámetro `voicing_order` (permitiendo que la clasificación de color de cada nota tenga en cuenta el orden/disposición vertical concreto del *voicing* del acorde, no solo su pertenencia abstracta al conjunto de clases de altura del acorde). Estas adiciones sugieren una evolución centrada en mejorar la robustez del reconocimiento armónico frente a texturas pianísticas reales (arpegios, voicings dispersos) más que un cambio de arquitectura.

Conceptos musicales. El esquema de color —fundamental/tonos del acorde vs. notas ajenas al acorde— es una codificación visual directa e inmediata del análisis armónico funcional aplicado nota a nota, pensada explícitamente como herramienta didáctica para que un estudiante de piano visualice de un vistazo qué notas de la mano derecha son “estructurales” (parte del acorde) y cuáles son ornamentales/de paso en relación con la armonía que sostiene la mano izquierda.

piano_expander.py — Expansión orquestal de bocetos pianísticos (v2.0, línea por línea)

Propósito. Lee un MIDI de piano y genera un MIDI multi-pista con orquestación idiomática, donde cada instrumento genera su **propia** línea musical independiente de principio a fin — no una copia transpuesta mecánica del piano— siguiendo arquetipos rítmicos y de textura específicos de cada familia orquestal.

Algoritmos. La mejora central de la v2.0 es la **arquitectura línea-por-línea**: cada instrumento se genera de forma coherente e independiente a lo largo de toda la pieza (en vez de derivarse compás a compás de forma local, lo que produciría inconsistencia estilística entre secciones), con **generadores rítmicos específicos por familia y arquetipo** — las cuerdas generan figuraciones de arpegio, las maderas generan "colchones" armónicos sostenidos, los metales entran estratégicamente en los clímax formales, reflejando convenciones orquestales bien establecidas de qué familia instrumental cumple qué función textural. El **contrapunto real con mirada hacia adelante** (*lookahead*) significa que la contramelodía generada para acompañar a la melodía principal se planifica conociendo su propio arco expresivo y clímax por adelantado, evitando quintas paralelas mediante verificación explícita contra el motivo principal en cada paso, en vez de generarse nota a nota sin conocimiento del futuro inmediato de la melodía que acompaña. Una **curva de tensión global** controla qué instrumentos entran y cuándo, reservando deliberadamente los metales para el clímax formal de la pieza — la misma lógica de "reserva de recursos tímbricos para el punto culminante" que usa `climax_engineer.py` en su etapa de refuerzo.

Conceptos musicales. La reserva estratégica de los metales para el clímax formal es una convención orquestal muy consolidada (los metales tienen el mayor poder dinámico bruto de la orquesta y "gastarlos" pronto reduce el contraste disponible para el punto culminante), y las funciones texturales asignadas por familia (cuerdas=arpegio/movimiento, maderas=colchón armónico, metales=momento climático) reflejan convenciones de orquestación estándar enseñadas en cualquier tratado de orquestación desde Rimsky-Korsakov en adelante.

piano_reducer.py — Reducción pianística de obras orquestales

Propósito. Lee un MIDI multi-pista (o un `partitura.yaml`) y genera una reducción pianística idiomática a dos pentagramas (mano derecha/izquierda), lista para evaluar en FL Studio o imprimir con MuseScore/LilyPond — la operación inversa exacta de `piano_expander.py`.

Algoritmos. Cinco estrategias de asignación de material a cada mano: `smart` (asignación inteligente según el rol musical detectado de cada pista, por defecto), `register` (mano derecha = notas por encima de un umbral de altura configurable, mano izquierda = por debajo — la partición más simple), `layer` (capas impares a la mano derecha, pares a la izquierda), `priority` (lista explícita de pistas prioritarias dada por el usuario) y `melody+bass` (la melodía principal siempre a la mano derecha, bajo y armonía combinados en la izquierda). Sobre la asignación de material se aplican técnicas pianísticas explícitas de reducción: reducción de notas simultáneas al límite polifónico ejecutable por una mano; **octavación idiomática** (notas fuera del rango cómodo se transponen una octava); **bajo Alberti/arpegios** como sustituto de acordes en bloque de más de tres voces en la mano izquierda (la misma figuración que genera `bass_line_composer.py` con estilo “classical”, aquí aplicada como recurso de reducción); y **compresión armónica** (acordes de más de cinco notas se reducen a las notas más representativas, priorizando fundamental, tercera, séptima y voces extremas).

Conceptos musicales. El repertorio de técnicas (octavación, Alberti, compresión armónica) es exactamente el vocabulario estándar del “reductor” profesional que adapta una partitura orquestal a una versión de piano tocable por un único intérprete — una habilidad tradicionalmente aprendida por estudio de reducciones publicadas de repertorio orquestal y operístico.

piano_synth.py — Síntesis física de piano y teclados, con optimización espectral (5.096 líneas)

Propósito. Herramienta consolidada de fichero único (fusión de varios scripts previos `generators/*.py`) para síntesis basada en física de piano de cola (síntesis modal), Rhodes (síntesis FM), un sintetizador propio llamado "Prism", comparación con DDSP, extracción de respuesta al impulso (IR) de referencia, y optimizadores de gradiente/evolución diferencial para ajustar parámetros de síntesis contra grabaciones reales.

Algoritmos. La **síntesis modal** del piano de cola modela cada cuerda percutida como una superposición de **modos de vibración** (armónicos con frecuencia, amortiguación y fase propios, derivados de un modelo físico simplificado de cuerda golpeada por un martillo, capturando fenómenos como la ligera inarmonicidad de las cuerdas de piano reales debida a su rigidez) — un enfoque de síntesis basada en física (*physical modeling*) más costoso computacionalmente pero más fiel al comportamiento acústico real que la síntesis aditiva pura. El Rhodes usa **síntesis FM** (modulación de frecuencia, el mecanismo histórico del Yamaha DX7 y del propio Rhodes eléctrico real, cuyo timbre metálico característico surge de la modulación de un oscilador portador por otro modulador a una frecuencia relacionada). La comparación con **DDSP** sitúa esta síntesis basada en física junto a la síntesis diferenciable de `ddsp_synth.py` como dos paradigmas alternativos para el mismo problema (reproducir el timbre de un piano), permitiendo comparar objetivamente sus resultados. Los **optimizadores de gradiente y de evolución diferencial** (DE, un algoritmo evolutivo sin gradiente especialmente efectivo en espacios de parámetros continuos de dimensión moderada) ajustan los parámetros del modelo físico contra grabaciones de referencia reales extraídas de bibliotecas de samples (Salamander, mencionada en el propio fichero) mediante comparación espectral.

Conceptos musicales. El contraste explícito entre síntesis modal basada en física (parámetros con significado físico directo: rigidez de cuerda, amortiguación) y síntesis DDSP (parámetros aprendidos, sin interpretación física directa necesaria) ilustra dos filosofías completas de modelado del sonido de un instrumento acústico: modelar las leyes físicas que lo producen, frente a aprender estadísticamente su comportamiento a partir de ejemplos.

piano_to_orchestra.py — Piano MIDI → orquesta completa, autónomo (v2.3, 4.007 líneas)

Propósito. Recibe un MIDI de piano (melodía + armonía, 1 o 2 pistas) y genera directamente un MIDI orquestal completo con instrumentos reales, *keyswitches*, automatización CC1/CC11 y percusión — sin requerir `orchestrator.py` ni ningún otro módulo externo, deliberadamente autocontenido.

Estructuras de datos. Pipeline interno con componentes claramente separados por responsabilidad: `MIDILoader` (carga notas/tempo/resolución con *hash* MD5 del fichero fuente, para detectar reprocesamiento innecesario), `MelodyExtractor` (separa melodía, bajo y voces interiores del piano de entrada), `HarmonicAnalyzer` (tonalidad, secciones, acordes, tensión) y `VoicingEngine` (genera cuatro voces SATB por sección).

Algoritmos. El diseño en componentes encadenados —carga → extracción melódica → análisis armónico → generación de voces SATB → orquestación con timbre— reproduce, dentro de un único fichero autocontenido, el mismo pipeline conceptual que en el resto del ecosistema está distribuido entre varias herramientas independientes (`midi_split_harmony`, `harmonic_analyzer`, `voice_leader`, `orchestrator/piano_expander`), una decisión de diseño explícita para que esta herramienta funcione de forma completamente autónoma sin depender de la presencia del resto del ecosistema.

Conceptos musicales. Al integrar en un solo flujo la separación melodía/bajo, el análisis armónico funcional y la generación de un voicing SATB completo antes de orquestar, formaliza el proceso compositivo clásico de “armonización a cuatro voces primero, orquestación después” que se enseña en los cursos de orquestación tradicional, donde el arreglista parte de una reducción armónica correcta antes de distribuir las voces entre los instrumentos reales.

pitch_transcriber.py — Adaptación de Basic Pitch (Spotify): audio → MIDI

Propósito. Adaptación al estilo mutopia de **Basic Pitch** (Spotify Research): un modelo ligero de transcripción polifónica audio→MIDI, ejecutado vía `onnxruntime` (sin dependencia de TensorFlow), validado explícitamente por equivalencia numérica exacta frente a la implementación original.

Algoritmos. El modelo produce tres **posteriogramas** (mapas de probabilidad tiempo-frecuencia): uno de *onsets* (ataques de nota), uno de *frames* activos (presencia continua de nota) y uno de contorno de altura fino (para pitch bends). `output_to_notes_polyphonic` convierte estos posteriogramas en eventos de nota discretos mediante umbralización de *onset*/*frame* con un parámetro de duración mínima de nota, e incluye el "**melodia trick**" — una heurística tomada directamente del algoritmo de extracción de melodía predominante Melodia (Salamon & Gómez), que favorece la continuidad de una línea melódica sobre detecciones aisladas espurias. `get_pitch_bends` extrae **micro-desviaciones de afinación continuas** del contorno de altura fino y las traduce a eventos de *pitch bend* MIDI, capturando inflexiones expresivas (vibrato, portamento) que una cuantización estricta a semitono perdería. El *ventaneo* del audio de entrada con solape (`overlap_len`) y la reconstrucción posterior (`unwrap_output`) procesan el fichero completo en fragmentos manejables sin discontinuidades en las fronteras entre ventanas.

Conceptos musicales. La distinción entre *onset*, *frame* y contorno de altura fino refleja tres aspectos perceptualmente distintos de una nota tocada: cuándo empieza exactamente, si sigue sonando, y con qué microvariaciones de afinación — la extracción de *pitch bends* explícitos es lo que permite que la transcripción capture matices expresivos de un instrumento real (una guitarra con bends, una voz con vibrato) que una transcripción puramente cuantizada a semitono descartaría por completo.

playability_auditor.py — Auditoría de tocabilidad idiomática instrumental

Propósito. Cubre un hueco explícito: el ecosistema genera abundante material orquestal y evalúa su calidad musical (`quality_scorer`, `section_score`, `voice_aligner`), pero ninguna herramienta verificaba si un **intérprete humano real** podría físicamente tocar cada parte generada. Audita cada pista contra el perfil físico y técnico del instrumento asignado, y opcionalmente corrige lo corregible.

Algoritmos. Seis categorías de comprobación, cada una codificando una restricción física o técnica real de ejecución instrumental: **rango** (notas fuera del rango físico absoluto del instrumento —error duro— o fuera de su tesitura "cómoda"/idiomática —aviso, técnicamente posible pero forzado—); **polifonía** (acordes en instrumentos monofónicos por naturaleza —un instrumento de viento no puede tocar dos notas a la vez—, o dobles/triples cuerdas no idiomáticas en instrumentos de cuerda frotada, que solo pueden ejecutar ciertos intervalos dobles cómodamente según las cuerdas al aire disponibles); **respiración** (frases continuas sin pausa para respirar, aplicable a vientos y voz —un instrumentista de viento o cantante necesita puntos de respiración regulares que una línea puramente algorítmica podría no dejar); **velocidad** (densidad de notas por segundo por encima del límite técnico razonable del instrumento y del intérprete); **repetición** (la misma nota repetida más rápido de lo que la articulación disponible —picado, staccato— permite ejecutar de forma limpia); y comprobaciones adicionales de salto (intervalos melódicos poco idiomáticos para el instrumento en cuestión).

Conceptos musicales. El catálogo de restricciones formaliza el conocimiento práctico de orquestación instrumental que un compositor aprende tradicionalmente estudiando tratados de orquestación e interactuando con intérpretes reales: cada familia instrumental tiene limitaciones físicas específicas (el fuelle/aliento de los vientos, la polifonía natural limitada de la cuerda frotada, la velocidad de articulación de cada técnica) que una generación puramente algorítmica, sin conocimiento explícito de estas restricciones, violaría sistemáticamente.

practice_tutor.py — Graduador de dificultad y generador de plan de estudio

Propósito. Completa el trío `playability_auditor` (¿es tocable?) + `fingering` (¿con qué dedos?) + este fichero (¿qué tan difícil es, y cómo practicarlo?): estima el nivel de dificultad de una pieza de piano, localiza los compases más difíciles y genera material de estudio estructurado — manos separadas, versión lenta (delegando en `tempo_designer.py`) y bucles de práctica de los compases más exigentes.

Algoritmos. El comando `grade` estima una nota de dificultad (escala 1-8, compartida con la familia `combined_simplifier`) descomponiéndola en factores explícitos: **velocidad** exigida (notas por segundo, resultado de tempo × densidad), **extensión** (amplitud de acordes/intervalos dentro de una sola mano), **polifonía** (número de notas simultáneas por mano) y **saltos** (distancia melódica grande dentro de una mano, que exige reposicionamiento rápido) — el mismo vocabulario de factores de dificultad que usa `combined_simplifier.py` para medir el grado resultante de una simplificación, aquí aplicado como herramienta de diagnóstico en vez de como función objetivo de una reducción. El comando `plan` genera automáticamente material de práctica estructurado: extracción de cada mano por separado (la técnica pedagógica estándar de practicar manos separadas antes de juntarlas), una versión a tempo reducido (reutilizando `tempo_designer.py`), y bucles aislados específicamente de los compases identificados como más difíciles.

Conceptos musicales. El pipeline "graduar dificultad → practicar manos separadas → practicar lento → hacer bucles de los pasajes difíciles" es exactamente la metodología de estudio pianístico estándar enseñada en cualquier conservatorio, automatizada aquí a partir del análisis objetivo de la partitura en vez de depender del criterio subjetivo del estudiante o profesor para decidir qué compases requieren atención especial.

preference_trainer.py — Aprendizaje de preferencias musicales personales (v1.2)

Propósito. Pipeline autónomo de siete comandos (`index → train → eval → rank → set → validate → info`) que aprende las preferencias musicales del usuario a partir de una colección de MIDIs indexada, y después evalúa/rankea obras nuevas según cuán bien encajan con ese perfil aprendido — uno de los cuatro `EvalProvider` mencionados en `audio_reference_scorer.py`, aquí en su forma nativa sobre MIDI simbólico.

Algoritmos. `index` vectoriza el corpus completo (con el mismo patrón de "una sola vez, cacheado" visto en `corpus_explorer.py` y `analyzer.py`: 100.000 MIDIs en 1-3 horas), con *features* opcionalmente extendidas o específicamente melódicas según el flag elegido. `train` ajusta un modelo (presumiblemente un clasificador o regresor sobre el espacio vectorial indexado, dado el flujo posterior de `eval`/`rank`) que aprende a distinguir obras "preferidas" de "no preferidas" a partir de las etiquetas que el usuario proporciona durante el entrenamiento. `eval` puntúa un MIDI nuevo contra ese modelo aprendido; `rank` ordena varios candidatos por esa puntuación; `set` gestiona conjuntos de preferencias (posiblemente múltiples perfiles); `validate` comprueba la consistencia del modelo entrenado (posiblemente con validación cruzada sobre las etiquetas dadas).

Conceptos musicales. A diferencia de `quality_scorer.py` (que mide calidad musical según criterios teóricos generales) o `audio_reference_scorer.py` (que mide afinidad a un corpus de referencia externo fijo), este fichero aprende específicamente el **gusto individual** del usuario concreto —una función de preferencia personalizada, no una noción universal de "buena música"— lo cual lo convierte en el componente de evaluación más directamente subjetivo y personalizado del ecosistema, apropiado como señal de recompensa en bucles de refinamiento dirigidos por el gusto de una persona específica en vez de por un estándar teórico general.

quality_scorer.py — Evaluación multidimensional de calidad musical (v1.2)

Propósito. Evalúa una o varias obras MIDI en cinco dimensiones de calidad —formal (corrección de conducción de voces e intervalos), coherencia (homogeneidad interna entre secciones), arco (estructura del arco emocional/tensión), melódica (interés y variedad) y corpus (similitud estilística a una colección de referencia)— con un *score* final **relativo** (percentil dentro de un corpus dado) y un *fallback* absoluto (0.0-1.0) cuando no hay corpus disponible.

Algoritmos. La combinación de las cinco dimensiones en un *score* único no usa pesos fijos: los **pesos efectivos se modulan automáticamente por la confianza de cada dimensión** — si, por ejemplo, la dimensión "corpus" no tiene suficientes referencias fiables para producir una estimación confiable, su peso en el *score* combinado se reduce automáticamente en favor de las dimensiones que sí tienen una estimación sólida, en vez de contribuir con un número potencialmente ruidoso al mismo peso nominal que las demás — una forma de **ponderación bayesiana implícita por incertidumbre**, evitando que una dimensión mal estimada arrastre el resultado combinado tanto como una bien estimada.

Conceptos musicales. Las cinco dimensiones cubren sistemáticamente los ejes de evaluación musical más citados en la literatura de composición asistida por IA (corrección técnica, coherencia estructural, arco dramático, interés melódico, fidelidad estilística), ofreciendo tanto un *score* absoluto interpretable (para obras aisladas) como uno relativo (percentil, más informativo cuando se dispone de un corpus de comparación amplio y representativo del estilo objetivo).

reharmonizer.py — Reharmonización guiada de melodías con progresiones alternativas (v1.1)

Propósito. Toma una melodía fija y genera N versiones con progresiones armónicas alternativas, aplicando técnicas de reharmonización clásica, jazzística y modal, validando en cada caso la compatibilidad de cada nota melódica con la nueva armonía propuesta.

Algoritmos. Once estrategias de reharmonización, cada una un procedimiento armónico histórico o técnico bien documentado: sustitución diatónica simple; **sustitución de tritono** (bII7 en vez de V7 — ambos acordes comparten el mismo tritono característico entre tercera y séptima, la sustitución armónica más emblemática del jazz moderno); **dominantes secundarios** (V/ii, V/IV, V/V…, tonicización temporal de grados no tónicos); **préstamo modal** (tomar acordes del modo paralelo —bVII, bIII, iv en mayor— un recurso extensamente usado en pop/rock y romanticismo); **mediante cromática** (III, bVI, bII usados como mediantes cromáticas en vez de diatónicas, un recurso central del lenguaje armónico romántico tardío); **ciclos de Coltrane** (progresión por terceras mayores equidistantes que divide la octava en tres partes iguales — la firma armónica de “Giant Steps” de John Coltrane, un recorrido tonal simétrico ajeno a la práctica común tonal); acorde **napolitano** (bII en primera inversión, sustituyendo o precediendo a la subdominante, un color cromático clásico desde el barroco); **pedal** armónico con acordes cambiantes sobre una nota sostenida; reharmonización en **modos menores** (dórico, frigio, etc.); progresiones funcionales **barrocas**; y progresiones **impresionistas** por tonos enteros y cuartas apiladas (evitando la tríada tradicional).

Conceptos musicales. El catálogo de once estrategias constituye, en efecto, un pequeño compendio histórico de técnicas de reharmonización occidentales desde el barroco (napolitana) hasta el jazz moderno más avanzado armónicamente (ciclos de Coltrane), cada una con su propia lógica de sustitución y su propio “sabor” estilístico reconocible, validadas todas contra la restricción común de que la melodía original siga siendo consonante o al menos plausible sobre la nueva armonía propuesta.

remi.py — Pop Music Transformer (REMI): generación y continuación de piano pop

Propósito. Implementación del paper "Pop Music Transformer: Beat-Based Modeling and Generation of Expressive Pop Piano Compositions" (Huang & Yang, ACM MM 2020): genera y continúa piezas de piano pop expresivo usando la representación **REMI** y un **Transformer-XL**.

Estructuras de datos. La tokenización **REMI** (REvamped MIDI-derived events) es la contribución central del paper original: en vez de una tokenización puramente basada en eventos con incrementos de tiempo continuos (como en `music_transformer.py`), REMI ancla explícitamente cada evento a una **rejilla métrica** — tokens de `Bar` (nuevo compás), `Position` (posición discreta dentro del compás), opcionalmente `Chord` (acorde detectado en esa posición), `Tempo Class`/`Tempo Value`, y finalmente `Note Velocity`/`Note On`/`Note Duration` por nota — dando al modelo una noción explícita y directa de métrica (compás y posición dentro de él) en vez de tener que inferirla implícitamente de una secuencia de deltas de tiempo continuos, lo cual el paper original demostró que produce ritmo perceptualmente más "en el pulso" y coherente métricamente.

Algoritmos. La arquitectura es un **Transformer-XL** (12 capas, d_model=512, 8 cabezas): a diferencia de un Transformer estándar con contexto fijo, Transformer-XL mantiene una **memoria recurrente** de los últimos 512 tokens de segmentos anteriores, reutilizada (sin recalcular) como contexto adicional al procesar el segmento actual — extendiendo efectivamente el contexto disponible mucho más allá de la ventana de atención nominal del segmento actual, y evitando el problema de fragmentación de contexto que sufriría un Transformer estándar al procesar una pieza larga en trozos independientes sin memoria entre ellos.

Conceptos musicales. El anclaje explícito a la rejilla métrica (Bar/Position) en vez de a deltas de tiempo continuos formaliza directamente la intuición de que la música popular occidental tiene una estructura métrica muy regular y perceptualmente central (el pulso, el compás), y que darle al modelo esa estructura como información explícita en el vocabulario de tokens —en vez de esperar que la infiera puramente de los datos— mejora significativamente la coherencia rítmica percibida de la generación resultante.

La familia “remi_diffusion”: modelo de lenguaje de difusión enmascarada sobre tokens REMI (v1 y v2)

Dos versiones aplican **modelos de lenguaje de difusión** (en vez de generación autorregresiva) al mismo vocabulario de tokens REMI de `remi.py`, siguiendo directamente la guía técnica del grupo de investigación de Kuleshov (“How to Build a Diffusion Language Model”).

v1 — difusión enmascarada por bloques. Sustituye la generación autorregresiva izquierda→derecha de `remi.py` por **difusión enmascarada** (MDLM, *Masked Diffusion Language Model*) aplicada en bloques de un compás cada uno (“*block diffusion*”): el proceso de ruido hacia adelante reemplaza progresivamente tokens por un token especial `[MASK]`, y el modelo aprende a invertir ese proceso. Incluye **guidance sin clasificador** (D-CFG, *Diffusion Classifier-Free Guidance* — la técnica estándar de guiado en modelos de difusión de imagen, adaptada aquí a texto/música discreta) condicionado en progresiones de acordes/tonalidad deseadas, y un *sampler* con **remasking** (ReMDM): durante el muestreo, el modelo puede volver a enmascarar posiciones ya “decididas” si resultan inconsistentes con el resto, corrigiendo errores durante la propia generación — una capacidad que la generación autorregresiva pura no tiene, porque nunca puede “reconsiderar” un token ya emitido.

v2 — encoder-decoder, KV-cache real, difusión de estado uniforme. Tres mejoras sustanciales: (1) **arquitectura encoder-decoder** (en vez de un único Transformer bidireccional): un encoder “pesado” procesa la historia (compases ya generados, siempre limpios) con atención **block-causal** (bidireccional dentro de cada compás, causal entre compases), mientras un decoder “ligero” hace *cross-attention* a esa memoria para des-ruidar el compás actual — el mismo patrón arquitectónico “encoder pesado, decoder ligero” de Gemma Diffusion; (2) **KV-cache real entre compases**: en vez de reprocesar toda la historia en cada paso de *denoising* de cada compás (extremadamente ineficiente), el encoder cachea claves/valores por capa y solo calcula los tokens del compás nuevo al añadirlo, con una ventana deslizante configurable que expulsa los compases más antiguos del caché al superar el límite — la misma optimización de KV-cache que hace viable la inferencia autorregresiva eficiente, aquí adaptada a un esquema por bloques en vez de por token; (3) **difusión de estado uniforme** (UDLM, seleccionable junto al esquema de máscara de v1): en vez de reemplazar tokens por `[MASK]`, el proceso de ruido hacia adelante los reemplaza por tokens **aleatorios** del vocabulario (nunca hay un símbolo `[MASK]` explícito), y el muestreo mantiene un estado “comprometido/no comprometido” por posición, pudiendo “re-ruidar” (re-aleatorizar) posiciones ya comprometidas para corregir errores — una familia de difusión discreta alternativa a MDLM con propiedades de corrección de errores distintas.

Conceptos musicales. La capacidad de “remasking”/“re-ruido” durante el muestreo —ausente en cualquier generación puramente autorregresiva— es la ventaja conceptual central de aplicar difusión a la generación musical simbólica: permite que el modelo revise y corrija decisiones armónicas o melódicas ya tomadas si resultan inconsistentes con material generado después, un tipo de “edición durante la composición” que ningún modelo estrictamente izquierda→derecha puede realizar sin regenerar desde el punto de inconsistencia.

rhythm_table.py — Catálogo curado de patrones rítmicos con tensión y emoción

Propósito. Complemento rítmico de `chord_table.py`: catálogo curado de patrones rítmicos —tanto células de una sola línea de *onsets* como *grooves* multi-voz completos— con etiquetas de tensión (1-10) y emoción, buscables y exportables; un patrón puede usarse como base percusiva independiente o como "ritmo armónico" (puntos de cambio de acorde) aplicado sobre una progresión de `chord_table.py`.

Estructuras de datos. Dos tipos catalogados: **célula** (una única línea de *onsets*, el patrón rítmico más simple) y **groove** (varias voces simultáneas, como un patrón completo de batería con bombo/caja/hi-hat). El catálogo cubre 13 estilos geográfica y estilísticamente muy diversos: afrocubano, tango, habanera, balcánico, flamenco, jazz, blues, bossa nova, baião (ritmo brasileño del nordeste), rock, hip-hop, reggaetón y *drum and bass*.

Conceptos musicales. El uso dual de un mismo patrón rítmico —como base percusiva directa, o como "ritmo armónico" que determina en qué instantes cambia el acorde de una progresión— formaliza una idea poco frecuente pero musicalmente rica: el ritmo de cambio armónico (*harmonic rhythm*) de una pieza es en sí mismo un patrón rítmico con su propia identidad estilística, tan característico de un género como su patrón de batería literal — el ritmo armónico sincopado de un tango no es casual, comparte la misma "sensación" rítmica que su patrón de habanera percusivo.

runner.py — Ejecutor de pipelines de composición

Propósito. Lee un plan `.yaml` producido por `theorist.py` o `narrator.py` y ejecuta cada paso del pipeline compositivo en orden, con *logging*, gestión de errores, puntos de pausa interactivos y capacidad de reanudar pipelines interrumpidos — el "orquestador de orquestadores" que convierte un plan declarativo de alto nivel en una secuencia real de invocaciones a las demás herramientas del ecosistema.

Algoritmos. Cinco etapas: (1) **carga**, detecta automáticamente si el YAML proviene del formato de `theorist.py` o de `narrator.py` (formatos de plan potencialmente distintos, manejados de forma transparente); (2) **validación**, verifica que todos los scripts referenciados en el plan existen realmente en el sistema de ficheros y que sus *inputs* están disponibles antes de lanzar nada, evitando fallos a mitad de una ejecución larga por un error trivial de configuración detectable de antemano; (3) **ejecución**, lanza cada paso como proceso, registrando *stdout*/*stderr* y midiendo duración; (4) **checkpoint**, guarda el estado tras cada paso completado, de modo que un pipeline interrumpido (por error, por elección del usuario, o por los puntos de pausa explícitos con `--pause-after`) pueda **reanudarse** exactamente donde se quedó (`--from`) en vez de reejecutar desde el principio; (5) **informe** final con tiempos y ficheros generados por cada paso.

Conceptos musicales. Aunque el contenido es de infraestructura de ejecución más que de teoría musical, refleja la naturaleza intrínsecamente secuencial y dependiente del proceso compositivo modelado por el ecosistema: un plan narrativo determina la estructura formal, que determina las curvas de tensión, que determinan la armonía, que determina la melodía — una cadena de dependencias donde cada paso consume la salida del anterior, exactamente el tipo de flujo de trabajo que un ejecutor de pipeline con *checkpointing* está diseñado para gestionar de forma robusta.

schenker_composer.py y schenkerian_reducer.py — Elaboración generativa y reducción analítica schenkeriana

Propósito. Un par complementario que aborda la reducción schenkeriana en direcciones opuestas: `schenkerian_reducer.py` (v1.0) parte de una obra ya compuesta (superficie / *Vordergrund*) y produce reducciones progresivas hacia una estructura fundamental candidata (*Ursatz*: una línea melódica descendente por grados conjuntos —*Urlinie*, con anclaje típico en el tercer grado— sobre un bajo I-V-I, *Bassbrechung*, el arpegio de bajo fundamental de la teoría schenkeriana); `schenker_composer.py` (v1.0) hace el camino **inverso**: entrena una red que aprende a "elaborar" —añadir ornamentación— desde un nivel de reducción n hacia el nivel n+1 más cercano a la superficie, construyendo su corpus de entrenamiento reduciendo automáticamente MIDIs reales con un analizador schenkeriano interno propio (inspirado en `schenkerian_reducer.py` y en la Teoría Generativa de la Música Tonal de Lerdahl & Jackendoff, pero reimplementado de forma autocontenida).

Honestidad epistemológica explícita. Ambos ficheros incluyen una advertencia notable, poco común en software de este tipo: el análisis schenkeriano automático **no es determinista ni "la verdad"** — distintos analistas humanos proponen reducciones razonables distintas de la misma pieza, y el propio Schenker nunca formalizó reglas computables completas. En consecuencia, `schenkerian_reducer.py` no pretende ofrecer una única reducción "correcta": genera **N candidatos puntuados** por un *score* de bondad de forma y presenta el mejor junto con las alternativas —el mismo principio de "puntuar una población de candidatos" que usa `melody_conditioned.py --engine genetic`—, y `schenker_composer.py` aclara que la red aprende el **estilo** de elaboración presente en su corpus de entrenamiento concreto, no reglas schenkerianas universales válidas para cualquier estilo.

Algoritmos. La reducción se basa en una heurística de puntuación combinada de peso métrico, peso armónico (pertenencia al acorde activo) y duración de cada nota, usada para decidir qué notas pertenecen a cada nivel jerárquico de reducción — la aproximación computacionalmente tratable a un proceso que en la práctica analítica humana depende de juicio experto no formalizable completamente. `schenker_composer.py` entrena un Transformer sobre pares (nivel n, nivel n+1) tokenizados, con la convención de niveles explícitamente invertida respecto a un reductor clásico (nivel 0 = fondo más reducido, nivel máximo = superficie original), de modo que la red aprende literalmente el paso de "elaboración hacia adelante" en vez de "reducción hacia atrás".

Conceptos musicales. El par de herramientas formaliza los dos sentidos del análisis schenkeriano clásico —análisis (reducir una superficie a su estructura profunda) y, especulativamente, síntesis (elaborar una estructura profunda hacia una superficie plausible)— con la honestidad epistemológica explícita de que ambos procesos son heurísticos, no algorítmicamente exactos, reflejando fielmente el estatus real de la teoría schenkeriana en la musicología académica: una herramienta analítica poderosa e influyente pero no un sistema formal completamente mecanizable.

section_score.py — Detección de secciones musicalmente incoherentes

Propósito. Segmenta una obra MIDI, construye un perfil global de la pieza y puntúa cada segmento en cuatro ejes de coherencia musical, permitiendo detectar automáticamente qué secciones "no encajan" con el resto de la obra — útil para auditar material generado algorítmicamente que puede producir incoherencias locales invisibles a una inspección superficial.

Algoritmos. Cuatro ejes de coherencia, cada uno capturando un tipo distinto de "encaje": **tonal** (compatibilidad de la melodía/armonía del segmento con la tonalidad global estimada de toda la obra — un segmento que modula sin resolución a una tonalidad lejana puntuaría bajo aquí); **estructural** (distancia vectorial del segmento respecto al perfil vectorial medio de toda la obra — detecta segmentos "atípicos" en un sentido estadístico general, no solo tonal); **vecinal** (incompatibilidad específicamente con los segmentos inmediatamente adyacentes — un segmento puede encajar bien con el perfil global de la obra pero sonar como una transición abrupta respecto a sus vecinos directos, un tipo de incoherencia que la métrica estructural por sí sola no detectaría); y **rítmico** (desviación del patrón rítmico dominante de la pieza). Cinco estrategias de segmentación intercambiables (`--granularity`): ventanas fijas de N compases, fronteras detectadas por SSM+novedad (reutilizando `harvester.py`), fronteras por cambio de textura, fronteras en cadencias detectadas con `music21`, o una combinación híbrida de forma+textura. Los pesos relativos de los cuatro ejes son configurables (`--weights`), con perfiles predefinidos como "balanced" (30/25/25/20%) o "harmonic" (previsiblemente ponderando más el eje tonal).

Conceptos musicales. La distinción entre coherencia "estructural" (respecto al perfil global) y "vecinal" (respecto a los vecinos inmediatos) formaliza dos nociones distintas y a veces contradictorias de "encajar" en una obra: una sección puede ser estadísticamente típica de la pieza en su conjunto pero sonar como una transición brusca en su contexto local inmediato, o viceversa —típica de sus vecinos pero atípica del carácter general de la obra.

select.py — Selección interactiva de MIDIs por clustering UMAP + KMeans

Propósito. Vectoriza una colección completa de MIDIs, los agrupa por similitud musical mediante clustering, y presenta muestras representativas de cada grupo para escucha y selección manual — una herramienta de curación asistida para explorar grandes colecciones (p. ej. la salida masiva de un generador algorítmico) sin tener que escuchar cada fichero individualmente.

Algoritmos. El flujo combina dos técnicas complementarias de análisis multivariante no supervisado: **KMeans** particiona el espacio vectorial de la colección en N grupos (clusters) según similitud, mientras **UMAP** (la misma técnica de reducción de dimensionalidad no lineal usada en `analyzer.py`) proyecta ese mismo espacio de alta dimensión a 2D, coloreado por cluster o por *score* emocional, para dar una visión de conjunto navegable de toda la colección. El bucle interactivo reproduce muestras de audio de cada cluster (vía `pygame`) y registra la decisión del usuario nota a nota —mantener (S), descartar (N), continuar sin decidir (C)— exportando al final la lista curada junto con comentarios opcionales por fichero.

Conceptos musicales. El clustering musical automático antes de la escucha manual reduce drásticamente el esfuerzo de curación: en vez de escuchar cada uno de, por ejemplo, cientos de fragmentos generados por `mosaic_composer.py` o `evolver.py`, el usuario escucha solo unas pocas muestras representativas de cada grupo estilísticamente coherente, confiando en que el resto de miembros de ese cluster son musicalmente similares a las muestras ya evaluadas.

silence_designer.py — El silencio como recurso dramático deliberado

Propósito. Observación de diseño explícita: los compositores automáticos del ecosistema tienden a "rellenar" —textura continua, sin respiraciones— y el silencio es el recurso emotivo más barato y menos aprovechado en generación algorítmica. Esta herramienta audita la saturación textural de una obra y abre espacio donde la narrativa lo pide, con la restricción explícita de **nunca insertar tiempo** (los huecos se crean acortando notas existentes, preservando la métrica original intacta).

Algoritmos. Cuatro operaciones, cada una un recurso retórico del silencio distinto: **luftpause** (respiraciones breves en las fronteras de sección, detectadas por novedad armónica combinada con cambio de densidad — el mismo tipo de detección de frontera usado por `harvester.py`, aquí aplicado para decidir *dónde* respirar en vez de dónde segmentar); **pausa** (silencio dramático general antes de un compás clave, especificado manualmente o colocado automáticamente antes del pico de energía de la obra); **reducción** (adelgazamiento deliberado de la textura a una sola voz —la melodía— en los compases previos al pico, generando intimidad antes del golpe dramático, la misma técnica de "vaciado" que usa `climax_engineer.py`); y **fermata** (calderón final: el último acorde se prolonga y se deja resonar — la **única** de las cuatro operaciones que efectivamente extiende la duración total, ya que todas las demás operan exclusivamente acortando material existente).

Conceptos musicales. La restricción de "nunca insertar tiempo, solo acortar" es una decisión de diseño deliberada que preserva la forma métrica global de la obra intacta mientras crea la sensación perceptual de espacio y aire —el silencio como elemento expresivo activo, no como ausencia de contenido—, un recurso retórico central en la práctica compositiva madura (el silencio antes de una entrada dramática, la respiración de frase) que un generador puramente aditivo tiende sistemáticamente a pasar por alto.

song_architect.py — De frases sueltas a estructura de canción popular completa (v2.0)

Propósito. Toma un número reducido de frases musicales (fragmentos MIDI) y genera una obra completa con estructura de canción popular estándar —Intro, Verso, Pre-estribillo, Estribillo, Verso 2, Puente, Solo, Outro— con coherencia musical genuina entre secciones, no una simple yuxtaposición de las frases dadas.

Algoritmos. La coherencia entre secciones se garantiza mediante varios mecanismos combinados: tonalidad y modo unificados para toda la obra; **conducción de voces** entre secciones y entre acordes consecutivos (evitando saltos armónicos abruptos en las fronteras de sección); curvas de tensión dramática asignadas por sección (siguiendo la función retórica esperada de cada una: un estribillo típicamente en mayor tensión/energía que un verso); **transformaciones motívicas coherentes** (variación, inversión, aumentación —el mismo catálogo contrapuntístico de `leitmotif_analyst.py`/`variation_engine.py`— aplicadas a las frases fuente para generar material derivado pero relacionado en cada sección nueva); progresiones armónicas adaptadas al **rol retórico** de cada sección (un puente suele modular o usar armonía más inestable que un estribillo, que suele ser armónicamente más simple y repetitivo); selección inteligente de qué fragmentos fuente usar en cada sección según su interés melódico relativo; y manejo robusto de MIDIs multi-pista con deduplicación de notas superpuestas y detección automática de compás.

Conceptos musicales. La plantilla formal (Intro-Verso-Pre-Estribillo-Estribillo-Verso2-Puente-Solo-Outro) es la estructura canónica de la canción popular anglosajona contemporánea, y asignar una función retórica y un nivel de tensión característico a cada etiqueta formal (el estribillo como punto de máxima memorabilidad/energía, el puente como sección de contraste armónico) formaliza convenciones de *songwriting* ampliamente documentadas en manuales de composición de canciones populares.

spectro_midi.py — Transcripción de espectrogramas NPZ a MIDI (CNN + Transformer, agnóstico de instrumento)

Propósito. Transcribe espectrogramas (formato `.npz` compartido con `audio_lab.py`) o audio WAV directamente a MIDI, mediante un modelo entrenado CNN+Transformer agnóstico del instrumento fuente, con un **bucle autónomo** (`autotrain`) que genera datos, entrena, evalúa e itera sin supervisión constante — además de dos modos deterministas sin aprendizaje automático (`melody`/`harmony`, por pico espectral puro) como alternativa ligera.

Algoritmos. La arquitectura combina una **CNN** (para extraer *features* espaciales locales del espectrograma, análogo a su uso estándar en visión por computador aplicado aquí a la "imagen" tiempo-frecuencia) con un **Transformer** (para modelar dependencias de más largo alcance sobre la secuencia de *features* extraídas por la CNN) — un patrón arquitectónico híbrido CNN+Transformer común en transcripción de audio, donde la CNN captura patrones espectrales locales (la forma de un armónico, un ataque) y el Transformer captura la estructura temporal global (cuándo empiezan y terminan las notas en relación unas con otras). El modo `autotrain` cierra un **bucle de auto-mejora**: genera datos de entrenamiento (sintéticos o de un corpus MIDI, renderizados a espectrograma), entrena el modelo sobre ellos, evalúa contra el MIDI de referencia conocido con métricas estándar de detección (F1, precisión, *recall* sobre notas correctamente transcritas), y repite el ciclo — un patrón de auto-entrenamiento iterativo sin supervisión humana constante entre iteraciones. Los modos `melody`/`harmony` sin ML son deliberadamente simples: detección de **picos espectrales** directos sobre el espectrograma, sin ningún modelo entrenado, ofreciendo una alternativa rápida y sin dependencia de un modelo previamente entrenado cuando la señal es limpia y monofónica (o de pocas voces).

Conceptos musicales. Ofrecer explícitamente tanto un camino de aprendizaje profundo (preciso pero requiere entrenamiento) como un camino puramente de procesamiento de señal (`melody`/`harmony`, sin ML, inmediato) para el mismo problema de transcripción refleja un patrón recurrente y deliberado del ecosistema: proporcionar siempre un *fallback* determinista y sin dependencias pesadas junto a la opción neuronal más potente.

stem_separator.py — Separación de fuentes de audio (HT-Demucs, sin la librería demucs)

Propósito. Reimplementación autocontenida (un único fichero, dependencias mínimas) de **Hybrid Transformer Demucs v4** (Rouard et al., Meta AI): separa un WAV mezclado en cuatro *stems* (batería, bajo, resto, voces) cargando el checkpoint preentrenado oficial "htdemucs", sin depender del paquete `demucs` completo ni de su cadena de dependencias (dora, omegaconf, einops…).

Algoritmos. La arquitectura híbrida procesa la mezcla por **dos ramas paralelas**: una rama de **frecuencia** que opera sobre la representación STFT (tratando la parte real e imaginaria del espectro complejo como canales separados, "complex-as-channels") mediante una U-Net, y una rama de **forma de onda cruda** (dominio temporal directo) mediante otra U-Net paralela — combinando así las ventajas de ambos dominios de representación, ya que ciertos tipos de separación (p. ej. transitorios percusivos nítidos) se benefician de la resolución temporal directa mientras otros (separación tonal) se benefician de la estructura espectral. Un **Transformer cross-dominio** en el cuello de botella de la red fusiona la información de ambas ramas antes de la decodificación, que también es dual: máscara + iSTFT (transformada de Fourier inversa) para reconstruir desde la rama de frecuencia, y forma de onda directa desde la rama de tiempo, sumadas para producir los cuatro *stems* finales.

Conceptos musicales. La combinación híbrida frecuencia+tiempo refleja que ningún dominio de representación único es óptimo para todos los tipos de fuente de audio a separar: la batería (contenido transitorio, ataques nítidos) se beneficia de la resolución temporal directa, mientras que la voz o instrumentos armónicos sostenidos se benefician de la estructura espectral clara del dominio de frecuencia — combinar ambas ramas con un mecanismo de fusión (el Transformer del cuello de botella) permite que el modelo aproveche lo mejor de cada representación según el tipo de fuente en cada momento.

stitcher.py — Orquestador de fragmentos MIDI con coherencia armónica (“Coherence Stitcher”)

Propósito. Usa los *fingerprints* (`.fingerprint.json`) generados por `midi_dna_unified.py` para tomar un conjunto de fragmentos MIDI dispersos, analizarlos, ordenarlos en la secuencia más coherente posible (o respetar un orden manual), puntuar cada transición posible entre fragmentos, generar **puentes automáticos** entre fragmentos incompatibles, y ensamblar el MIDI final.

Algoritmos. El “puntuado” de cada transición posible entre dos fragmentos combina compatibilidad armónica, de registro, de tempo y de tensión — es decir, el problema de encontrar el mejor **orden** de una colección de fragmentos para ensamblarlos coherentemente es estructuralmente un problema de **camino de coste mínimo** sobre un grafo completo donde los nodos son fragmentos y las aristas están ponderadas por el coste (incompatibilidad) de la transición entre cada par — análogo en estructura al problema del viajante (TSP) aunque probablemente resuelto aquí con una heurística voraz o de inserción en vez de una solución exacta, dada la naturaleza del dominio. Cuando dos fragmentos consecutivos en la secuencia elegida son incompatibles por debajo de un umbral, se genera automáticamente un **puente** invocando `midi_dna_unified.py` con los parámetros armónicos necesarios para suavizar la transición (en vez de simplemente descartar la combinación o forzar una yuxtaposición abrupta).

Conceptos musicales. El fichero formaliza el trabajo de un editor de “collage” musical experimentado: decidir en qué orden suenan mejor juntos varios fragmentos independientes, y cuándo un fragmento necesita un puente de transición explícito (una modulación, un enlace armónico) en vez de encajar directamente con el siguiente — un problema combinatorio con una lectura musical clara de compatibilidad tonal y de tensión.

style_blender.py — Fusión dinámica de dos estilos a lo largo de una obra

Propósito. A diferencia de `style_transfer.py` (un único estilo aplicado a toda la obra con una intensidad fija), mezcla **dos** fuentes de estilo simultáneamente usando una curva de mezcla (*blend*) que cambia compás a compás — permitiendo fusiones progresivas como "pop→tango" o "clásico→jazz", un viaje continuo entre dos mundos sonoros en vez de un punto fijo intermedio.

Estructuras de datos. El parámetro `blend` recorre [0,1]: 0.0 = estilo A puro, 0.5 = fusión equitativa, 1.0 = estilo B puro, definible de tres formas — puntos explícitos compás:valor (`"0:0.0, 16:0.5, 32:1.0"`), importado directamente de una curva de `tension_designer.py`, o mediante un preset con forma predefinida.

Conceptos musicales. Que la curva de mezcla pueda importarse de `tension_designer.py` conecta directamente el eje puramente estilístico (qué "suena a" cada fuente) con el eje dramático-emocional del resto del ecosistema: una transición de estilo pop→tango podría, por ejemplo, coincidir deliberadamente con una curva de tensión creciente, de modo que el cambio estilístico refuerce perceptualmente el arco dramático de la obra en vez de ser un efecto puramente decorativo independiente de la narrativa musical.

style_transfer.py — Transferencia de estilo: aplica el ADN de un MIDI a otro

Propósito. Dado un MIDI de “contenido” (la idea melódica del usuario) y un MIDI de “estilo”, preserva la esencia melódica del primero mientras aplica el ADN rítmico, armónico, emocional y de textura del segundo — un “filtro de estilo” musical explícitamente diseñado para mantener la voz creativa original del usuario, no para sustituirla.

Algoritmos. Seis dimensiones de transferencia controlables independientemente (ritmo/groove, armonía, emoción, timbre/textura de acompañamiento, dinámica, tonalidad — con transposición opcional del contenido a la tonalidad del estilo) permiten decidir exactamente cuánto de cada aspecto del estilo se importa, dejando el resto fiel al contenido original. Dos estrategias de preservación melódica explícitas: **preservar contorno** (mantiene la dirección ascendente/descendente de la melodía original, pero permite que los intervalos exactos se ajusten para encajar con la nueva armonía del estilo) frente a **preservar intervalos** (mantiene los saltos melódicos exactos del original, aceptando potencialmente más fricción con la nueva armonía) — una elección explícita entre dos nociones distintas y a veces incompatibles de “fidelidad melódica al original”.

Conceptos musicales. La distinción preservar-contorno vs. preservar-intervalos formaliza una tensión real en cualquier reharmonización o transferencia de estilo: el contorno (forma general de la línea) y los intervalos exactos (su contenido preciso) son ambos candidatos razonables a “lo esencial” de una melodía, y cuál se prioriza depende de si se valora más la silueta reconocible de la idea o su contenido armónico literal.

synth_forge.py — Motor de síntesis multi-paradigma (seis técnicas clásicas en Python puro)

Propósito. Implementa en NumPy puro (sin Csound, SuperCollider ni Faust externos) seis paradigmas clásicos de síntesis sonora, cada uno accesible como subcomando independiente con su propio conjunto de parámetros.

Algoritmos. Los seis paradigmas: **sample** (síntesis por muestras con *keymap* y capas de velocidad, al estilo del formato ligero SFZ — la técnica estándar de sampler profesional: distintas grabaciones reales disparadas según rango de altura y de velocidad de la nota tocada); **waveguide** (modelado físico por guía de onda, una extensión del algoritmo **Karplus-Strong** —simulación de una cuerda pulsada mediante una línea de retardo realimentada con filtrado— con modos adicionales *plucked* (pulsada) y *bowed* (frotada, modelando explícitamente la **fricción stick-slip** entre el arco y la cuerda, el mecanismo físico real por el que un arco de violín excita la vibración de la cuerda mediante ciclos alternos de adherencia y deslizamiento); **additive** (síntesis aditiva pura, suma ponderada de armónicos, la misma técnica que `audio_lab.py`); **fm** (síntesis FM de dos operadores, con razón armónica o inarmónica entre portadora y moduladora, la técnica histórica del Yamaha DX7); **brass** (modelado físico específico de instrumento de viento-metal: labios no lineales como excitador —el mecanismo físico real de vibración labial que excita la columna de aire— acoplados a un banco de resonadores modales tipo **Klank**, el modelo estándar de resonador modal de Csound); y **granular** (síntesis granular: una nube de "granos" cortos de sonido, cada uno con su propia ventana de amplitud tipo Hann, superpuestos con densidades y posiciones variables — la técnica que permite, entre otras cosas, cambiar la duración de un sonido sin alterar su altura, o crear texturas nebulosas a partir de fuentes granuladas).

Conceptos musicales/acústicos. El catálogo cubre sistemáticamente las grandes familias históricas de síntesis de sonido —muestreo, modelado físico, aditiva, modulación de frecuencia, y granular—, cada una con un modelo matemático y una intuición física o perceptual distinta de cómo se genera o simula el timbre de un instrumento, ofreciendo una comparación directa de las mismas notas MIDI renderizadas bajo paradigmas de síntesis radicalmente distintos.

tempo_designer.py — Diseño de la arquitectura de tempo (agógica global)

Propósito. Mientras `performer.py` aplica rubato **local** por frase mediante reglas automáticas, este fichero diseña la **agógica global** de toda la obra y la "hornea" directamente en el mapa de tempo del MIDI (meta-evento 0x51, el estándar MIDI para cambios de tempo) — el equivalente temporal de `tension_designer.py` (curva de tensión) y `climax_engineer.py` (arquitectura de energía): aquí se diseña específicamente la arquitectura del **tiempo**.

Algoritmos. El modelo de tempo se define por puntos de control (pares compás:BPM), interpolados según tres formas seleccionables: `step` (terrazas discretas, cambios abruptos de tempo entre secciones, sin transición gradual), `linear` (rampas de accelerando/ritardando a tasa constante entre puntos de control) y `smooth` (interpolación coseno, con aceleración/desaceleración suave en los extremos de cada tramo, evitando el cambio de velocidad perceptualmente brusco de una rampa puramente lineal en sus fronteras). Se añaden calderones (`--fermata`, ralentización local puntual de un instante concreto) y un ritardando final configurable como desaceleración progresiva en los últimos compases.

Conceptos musicales. El tempo global de una obra es en sí mismo un parámetro expresivo con su propia arquitectura narrativa —terrazas de tempo entre secciones contrastantes, aceleraciones hacia un clímax, el ritardando final convencional de cierre— tratado aquí con el mismo rigor de diseño paramétrico que el ecosistema aplica a la tensión armónica o a la energía dinámica, en vez de dejarlo como un valor constante o como resultado incidental de otras decisiones compositivas.

tension_designer.py — Diseñador visual interactivo de curvas de tensión

Propósito. Permite dibujar a mano, de forma interactiva, la curva de tensión (y otras curvas emocionales relacionadas) de una obra completa **antes** de generar ningún MIDI; las curvas resultantes se convierten directamente en *flags* `--mt-*` para `midi_dna_unified.py`, cerrando el ciclo entre diseño visual y generación paramétrica.

Estructuras de datos. Cinco curvas editables independientemente, cada una en el rango [0,1]: tensión armónica principal, actividad/densidad rítmica, registro melódico, complejidad armónica y swing, seleccionables mediante atajos de teclado dedicados durante la sesión de dibujo interactivo.

Conceptos musicales. Es el "lienzo" gráfico directo de todo el sistema de curvas emocionales paramétricas que consumen `midi_dna_unified.py`, `arc_supervisor.py` y varias otras herramientas: en vez de razonar sobre las curvas mediante parámetros numéricos abstractos, el compositor puede dibujarlas directamente como formas a mano alzada, con la traducción automática a los parámetros CLI exactos que las reproducirán.

theme_transformer.py — Mapeo escalar y transformación temática, incluyendo armonía negativa (v2.1)

Propósito. Implementa técnicas de "*thematic transformation via scalar mapping*": traslada una melodía MIDI a otras escalas o modos preservando su identidad reconocible, con armonización y orquestación coherentes con cada estilo destino.

Algoritmos. La novedad central de la v2.1 es el estilo `negative_harmony`: la **armonía negativa** (Ernst Levy, popularizada recientemente por Jacob Collier) refleja tanto la melodía como los acordes sobre un **eje tónica-dominante** — matemáticamente, una reflexión (inversión especular) del espacio de alturas alrededor de un eje fijo entre la tónica y la dominante de la tonalidad, que transforma sistemáticamente cada intervalo en su "opuesto" especular preservando ciertas relaciones estructurales (la tónica y la dominante se intercambian mutuamente bajo esta reflexión, mientras que acordes mayores tienden a transformarse en sus relativos menores y viceversa). El nuevo método de mapeo `mirror` implementa esta reflexión como **reflexión aritmética directa** (una resta simple respecto al eje de simetría) en vez del reanclaje por grado de escala que usan los métodos de mapeo anteriores — dos formas matemáticamente distintas de lograr un resultado perceptualmente similar, con `--axis-offset` controlando en semitonos dónde se sitúa exactamente el eje de reflexión respecto a la tónica.

Conceptos musicales. La armonía negativa es un marco teórico relativamente reciente y algo controvertido en la teoría musical contemporánea (defendido con entusiasmo por algunos músicos populares, recibido con escepticismo por parte de la teoría académica tradicional) que reinterpreta las relaciones tonales como simetrías especulares en vez de como jerarquías funcionales — su inclusión aquí junto a técnicas de transformación temática más establecidas refleja la amplitud deliberada del ecosistema a la hora de incorporar tanto teoría canónica como marcos teóricos más recientes y debatidos.

theorist.py — De intención musical vaga a plan de composición fundamentado

Propósito. El “cerebro” de planificación de más alto nivel del ecosistema: traduce descripciones vagas de intención musical en lenguaje natural en planes de composición completos, **justificados paso a paso en lenguaje teórico musical explícito** — actúa como un teórico/compositor que razona en voz alta antes de producir ningún número, primero estableciendo semántica, después teoría, y solo entonces parámetros concretos.

Algoritmos. El flujo de seis etapas es notable por hacer explícito el razonamiento intermedio que normalmente permanece implícito en un sistema de generación por parámetros: (1) **semántica**, interpreta la descripción libre y detecta explícitamente **contradicciones** en la petición del usuario (p. ej. pedir algo simultáneamente “calmado” y “frenético” sin resolver la tensión entre ambos); (2) **marco teórico**, elige una escuela compositiva, referencias estilísticas y reglas asociadas coherentes con la intención detectada; (3) **decisiones**, cada parámetro concreto del pipeline se fija con su **justificación teórica explícita** (no solo el valor, sino el razonamiento musical que lo motiva); (4) **curvas**, genera las curvas emocionales correspondientes, exportables directamente a `tension_designer.py`; (5) **pipeline**, traduce todo lo anterior a los argumentos de línea de comandos exactos para cada script del ecosistema que se necesite invocar; (6) **exportación** en múltiples formatos (`.theorist.json`, `.curves.json`, `plan.yaml`, `plan.json`) consumibles por `runner.py`, `narrator.py`, `cadence_designer.py` y otras herramientas de planificación.

Conceptos musicales. La detección explícita de contradicciones en la intención del usuario, antes de comprometerse con ningún parámetro, formaliza una práctica de diseño poco común pero valiosa: en vez de resolver silenciosamente una ambigüedad o contradicción con una elección arbitraria del sistema, la hace explícita y la razona, tratando el diseño de un plan compositivo como un proceso de argumentación teórica justificable, no como una simple aplicación mecánica de reglas.

tonicization_engine.py — Inserción quirúrgica de dominantes secundarios

Propósito. Toma una progresión de acordes ya escrita (MIDI de acordes en bloque, del tipo que exportan `chord_progression_generator.py/reharmonizer.py`) y decide en qué compases y de qué manera introducir dominantes secundarios (V/x) que tonicen sus acordes diatónicos, sin tocar el resto de la obra — el análogo armónico de `expectation_engine.py` (que manipula expectativas en cadencias V→I) pero para tonicización: en vez de negar o retrasar una resolución esperada, **introduce** una tensión nueva y localizada (la quinta de un grado diatónico) que resuelve de inmediato dentro del mismo compás. Complementa a `chord_progression_generator.py` (que ya puede generar V/x desde cero) y a `reharmonizer.py` (que sustituye la progresión entera): esta herramienta opera quirúrgicamente sobre una progresión ya fijada, sin rehacerla.

Algoritmos. La mecánica central consiste en, para cada acorde diatónico candidato a tonicizar, calcular su quinta superior (la dominante de ese grado) y comprobar si insertarla inmediatamente antes —como acorde de dominante secundaria de duración parcial dentro del mismo compás o en el tiempo anterior— es armónicamente viable dado el contexto (el grado no debe ser ya por sí mismo inestable, y la nota añadida no debe entrar en conflicto grave con la melodía existente si la hay). La decisión de **qué** compases tonicizar se basa en criterios de saliencia armónica: los grados ii, IV, V y vi son los candidatos clásicos más productivos a la tonicización secundaria en la práctica tonal común, mientras que tonicizar el propio I sería redundante y tonicizar el vii° es infrecuente por su ya inherente inestabilidad.

Conceptos musicales. La tonicización secundaria —sonar momentáneamente "como si" estuviéramos en la tonalidad de un grado distinto del I, sin modular realmente— es uno de los recursos de enriquecimiento armónico más comunes de la práctica tonal común, desde el barroco hasta el pop contemporáneo, y esta herramienta la formaliza como una operación local e insertable quirúrgicamente sobre una progresión ya compuesta.

tonicizer.py — Motor de modulación tonal y puentes armónicos entre tonalidades

Propósito. Construye (y, sobre melodías reales, inserta) el puente armónico que lleva de una tonalidad A a una tonalidad B, usando una técnica de modulación clásica concreta — cada técnica se modela con su propia lógica armónica, no como variaciones cosméticas de una única plantilla genérica de transición.

Algoritmos. Siete técnicas de modulación, cada una un procedimiento armónico históricamente documentado y estructuralmente distinto: `pivot` (acorde pivote diatónico común a ambas tonalidades — la modulación más suave, donde un acorde perteneciente a ambas tonalidades sirve de bisagra); `common_tone` (modulación por nota común vía mediante cromática); `diatonic` (modulación a una tonalidad cercana, encadenando saltos de quinta o relativa si B no es directamente cercana a A); `chromatic` (acorde de aproximación cromática, fuera de ambas tonalidades, como bisagra disruptiva); `enharmonic` (una séptima disminuida reinterpretada enarmónicamente, explotando su simetría equidistante para pertenecer ambiguamente a varias tonalidades); `direct` (modulación abrupta sin acorde de transición); y `sequential` (secuencia armónica que se desliza progresivamente de una tonalidad a otra mediante repetición transportada de un patrón).

Conceptos musicales. El catálogo de siete técnicas constituye un compendio representativo de las estrategias de modulación tonal documentadas en cualquier tratado de armonía funcional, desde la más conservadora (pivote diatónico) hasta la más disruptiva (modulación directa), cada una con su propio efecto retórico característico: el pivote se percibe como una transición natural e inevitable, la modulación directa como un salto deliberado y marcado.

La familia “unifier”: unificación coherente de secciones-ancla (v1.1 → v3.0)

Tres versiones abordan el mismo problema de diseño reconocido explícitamente: cuando se compone pegando secciones de obras distintas como “anclas emocionales”, la unión suele sonar a collage —faltan motivos comunes, la armonía no encaja, cambia el pulso, la instrumentación es inconexa— y `unifier.py` establece un **ADN musical compartido** que inyecta de forma dosificada en cada sección, cosiéndolas con transiciones coherentes.

v1.1 — ADN compartido deducido. Cuatro componentes del ADN compartido (deducidos automáticamente si no se especifican): motivo semilla (el más saliente entre las secciones dadas, o uno proporcionado), plan tonal (tonalidad hogar + modulaciones lógicas por emoción), plantilla orquestal (piano/cuarteto/cuerdas/cámara/orquesta completa) y arco de tensión tipo “arco” sobre el conjunto de secciones. Los ejes de unificación son dosificables individualmente con `--glue` (0=sección intacta, valores mayores = más inyección del ADN compartido).

v2.0 — fusión de dos líneas de implementación previas. Explícitamente descrita como el *merge* de dos v1 paralelas: de la línea “interna” hereda ancla explícita, tempo global con rampas, *groove* del ancla (histograma de 16 subdivisiones, la misma técnica de `midi_dna_unified.py`), protección del canal 9 (percusión), tonalidad única con relativas y arco áureo; de la línea “externa” hereda **extracción de motivo por n-gramas con saliencia** (identifica el motivo más “memorable” estadísticamente, no solo el más frecuente), transformaciones del motivo (inversión/retrógrado/aumentación, el catálogo estándar del ecosistema), puntos estructurales de siembra, **nudge armónico** con conducción de voces (ajuste mínimo de notas para resolver incompatibilidades sin reescribir), puentes con acorde pivote + V7→I, y un *score* de coherencia antes/después que cuantifica objetivamente la mejora lograda.

v3.0 — modo recombine y plan tonal por emoción. Añade dos capacidades recuperadas de la v1.1 original que el *merge* de v2.0 había dejado fuera: el **modo recombine**, que usa las secciones dadas como “donantes de ADN” y regenera una obra completamente nueva y coherente (progresión armónica + motivo trasplantado + bajo + contrapunto + patrones rítmicos por emoción) en vez de limitarse a pegar las secciones originales con transiciones — una operación cualitativamente distinta de “unificar” (edición conservadora) a “recomponer” (síntesis nueva a partir de material donante); y un **plan tonal por emoción**, donde cada sección viaja a una tónica sesgada específicamente por su emoción (subdominante/dominante/mediante/relativa según el carácter buscado) en vez de una única tonalidad global compartida — explícitamente descrito como “más viaje de color, menos unidad”, una opción deliberada de priorizar la variedad tonal sobre la cohesión tonal estricta cuando así se desee.

Conceptos musicales. La distinción entre “unificar” (v1/v2: coser y suavizar material ya dado) y “recombinar” (v3: usar material dado como donante genético para una síntesis nueva) formaliza dos estrategias editoriales genuinamente distintas frente al mismo problema de origen —fragmentos musicalmente dispares que deben convivir en una obra— y ofrecerlas ambas explícitamente como modos reconoce que la elección entre “editar con respeto al original” y “reinventar libremente a partir del original” es una decisión estética, no técnica.

variation_engine.py — Motor de variaciones musicales automáticas (catálogo de referencia)

Propósito. Dado un MIDI fuente, genera un catálogo completo de variaciones musicales clásicas y contemporáneas manteniendo la identidad temática reconocible — el módulo de referencia que otras herramientas del ecosistema (`leitmotif_analyst.py`, `leitmotif_tracker.py`, `song_architect.py`, `completer.py`) importan para aplicar transformaciones motívicas de forma consistente en todo el sistema.

Algoritmos. El catálogo cubre las transformaciones contrapuntísticas canónicas —inversión (refleja el contorno melódico), retrógrado (invierte el orden temporal), inversión retrógrada (combina ambas), aumentación y diminución rítmicas (duplica/reduce a la mitad las duraciones), ornamentación (añade notas de paso, apoyaturas, mordentes), reharmonización del patrón de acompañamiento, transposición, cambio modal (mayor↔menor u otros modos), y reinterpretación rítmica— formalizando en un único módulo reutilizable el vocabulario completo de variación temática que el resto del ecosistema referencia constantemente por nombre (V01, V02, etc., el mismo esquema de numeración usado en `leitmotif_analyst.py` para clasificar transformaciones detectadas).

Conceptos musicales. Al ser el punto único de implementación de todo el catálogo de transformaciones temáticas, garantiza que “inversión” o “retrógrado” signifiquen exactamente lo mismo (la misma implementación algorítmica concreta) en cualquier herramienta del ecosistema que las use, evitando inconsistencias entre, por ejemplo, la inversión que genera `variation_engine.py` y la inversión que `leitmotif_analyst.py` intenta reconocer como ya presente en una obra.

voice_aligner.py — Alineación de consonancia entre voces MIDI independientes (v2.0)

Propósito. Dadas múltiples voces MIDI generadas independientemente (p. ej. por distintas herramientas del ecosistema sin coordinación explícita entre sí), detecta y resuelve colisiones de disonancia no intencionadas entre ellas, ajustando el mínimo posible para preservar tanto la verticalidad consonante como el contorno melódico horizontal de cada voz.

Algoritmos. La v2.0 añade ocho mejoras sustanciales sobre v1.0: un **índice de importancia por nota** que protege las notas estructurales de ser desplazadas (el mismo principio de "suelo protegido" que `combined_simplifier.py`); una métrica de choque **ponderada por duración** (no solo por presencia binaria — una disonancia sostenida durante todo un compás pesa más que una de paso instantánea); un *score* combinado de coste **vertical** (disonancia armónica) más coste **melódico** (preservación del contorno horizontal de cada voz, para que la resolución de un choque vertical no destroce la línea melódica de la voz ajustada); detección de tonalidad por **ventana deslizante con consenso entre voces** (en vez de una tonalidad global fija, permite que la tonalidad local varíe y se estime combinando la evidencia de todas las voces simultáneamente); "**nudge conjunto**" — una **optimización cartesiana** de todos los *pitches* en conflicto a la vez, en vez de resolver cada choque de forma miope e independiente (que podría introducir un nuevo choque al resolver otro); desplazamiento de zona completa para segmentos muy desalineados; ornamentación (apoyatura, nota de paso) como **estrategia de resolución** alternativa al desplazamiento directo de altura; y refinamiento **iterativo hasta convergencia** (aplicando sucesivas rondas de ajuste hasta que ya no queden choques resolubles por debajo de un umbral). Tres estrategias en cascada, de menor a mayor intervención: *shift* global de voz completa, *shift* local de una nota individual, y *shift* de zona (rango de compases).

Conceptos musicales. El problema que resuelve —coordinar voces generadas independientemente sin que produzcan disonancias accidentales no intencionadas— es estructuralmente el mismo que enfrentaría un arreglista humano combinando partes escritas por distintos colaboradores sin coordinación previa: la optimización conjunta ("nudge") en vez de resolución voraz secuencial es la diferencia clave entre una solución localmente parcheada y una genuinamente coherente en su conjunto.

voice_leader.py — Motor de voice leading SATB para bloques de acordes

Propósito. Toma una progresión de acordes (texto o MIDI de acordes en bloque) y genera el *voicing* SATB (Soprano-Alto-Tenor-Bajo) óptimo compás a compás, aplicando las reglas clásicas de conducción de voces: movimiento mínimo, ausencia de quintas paralelas, sin cruzamientos de voz, rangos vocales idiomáticos y resolución correcta de tensiones — a diferencia de `counterpoint.py` (una voz contra un cantus firmus dado) y de `midi_dna_unified.py` (voicing interno de acompañamiento incidental), este módulo razona explícitamente sobre los **cuatro bloques vocales simultáneos como sistema conjunto**, no voz a voz.

Algoritmos. El problema de encontrar, para cada acorde de la progresión, la disposición de las cuatro voces que (a) respeta los rangos vocales idiomáticos de cada una (soprano aguda, alto media-aguda, tenor media-grave, bajo grave y armónicamente fundacional), (b) minimiza el movimiento total entre el acorde actual y el siguiente, y (c) nunca produce quintas u octavas paralelas ni cruces de voz, es estructuralmente un problema de **optimización combinatoria con restricciones** sobre el conjunto (pequeño pero no trivial) de disposiciones válidas de cada acorde — la misma clase de problema que `counterpoint.py` resuelve mediante backtracking CSP para una sola voz contra un cantus firmus, aquí generalizado a cuatro voces simultáneas.

Conceptos musicales. Es la formalización algorítmica directa del ejercicio central de cualquier curso de armonía a cuatro voces de conservatorio: dado un cifrado armónico, encontrar la realización vocal SATB que sea a la vez correcta (sin errores de conducción de voces) y elegante (movimiento mínimo, sin saltos innecesarios) — el mismo problema que `harmony_checker.py` evalúa retrospectivamente sobre un intento ya escrito, aquí resuelto de forma prospectiva y generativa.

wfc_composer.py — Composición mediante Wave Function Collapse + SMT solver (v3.0)

Propósito. Aplica **Wave Function Collapse** (WFC, un algoritmo de generación procedimental popularizado en el diseño de niveles de videojuegos, que "colapsa" progresivamente las posibilidades de un grafo de restricciones locales hasta obtener una configuración global consistente) combinado con un **solver SMT (Satisfiability Modulo Theories)** real — Z3, de Microsoft Research — a la generación de una textura polifónica de varias voces, modelada como un grafo de ventana temporal con tres tipos de arista tipados: MELÓDICA, ARMÓNICA y CADENCIAL.

Estructuras de datos. El grafo tiene un nodo por evento de nota candidato (voz, posición temporal, altura posible) y tres familias de arista con alcance de ventana distinto: aristas MELÓDICAS conectan eventos consecutivos dentro de la misma voz (dentro de una ventana `W × N_VOICES`), aristas ARMÓNICAS conectan eventos simultáneos de voces distintas (verticalidad), y aristas CADENCIALES marcan restricciones específicas de los puntos de cadencia.

Algoritmos. A diferencia de una implementación WFC clásica (que resuelve las restricciones locales mediante propagación de restricciones y *backtracking* heurístico), aquí las restricciones se codifican como fórmulas lógicas de primer orden y se delegan directamente en **Z3**, un solver SMT de propósito general capaz de encontrar (o demostrar la inexistencia de) una asignación que satisfaga simultáneamente **todas** las restricciones declaradas — entre las corregidas explícitamente en v3 respecto a v2: una restricción global `c_no_voice_overlap` que ahora se aplica universalmente y no solo a las aristas armónicas (corrigiendo un caso donde voces podían solaparse ilegalmente sin que ninguna arista existente lo detectara), distribución temporal ponderada por el hueco disponible real (en vez de asumir espacio uniforme), una restricción `AtLeast` de cobertura mínima por frase (garantizando que ninguna frase quede vacía de eventos tras la resolución), un **arco de tensión** incorporado directamente en la función de verosimilitud como una curva parabólica según la posición dentro de la frase (más tensión hacia el centro/final, menos al principio — coherente con el modelo de tensión de Lerdahl usado también en `wfc_composer.py` para puntuar la tensión armónica de cada altura candidata: tónica=0, dominante=2, subdominante=4, supertónica=6, sensible=8, cromatismos=10), y duraciones con no-solapamiento real garantizado entre nodos de la misma voz.

Conceptos musicales. Usar un solver SMT exacto en vez de una heurística de propagación de restricciones aproximada es una elección deliberada de **corrección garantizada**: si Z3 encuentra una solución, esa solución satisface con certeza matemática absoluta todas las restricciones de conducción de voces y armonía declaradas simultáneamente (no "en su mayoría" o "con alta probabilidad" como produciría una heurística), al coste de que el problema pueda ser computacionalmente costoso o, en el peor caso, resultar insatisfacible si las restricciones combinadas son demasiado estrictas para el material dado.

yue_composer.py — Generación de canciones completas con letra mediante YuE

Propósito. Adaptación del modelo open-source **YuE** (música generativa lyrics→canción): dado un texto de letra y etiquetas de género, genera una canción completa de varios minutos con voz y acompañamiento instrumental separados.

Estructuras de datos. Arquitectura en cascada de cuatro componentes: una etapa 1 de 7.000 millones de parámetros (un modelo de lenguaje que genera tokens de audio comprimidos con un único *codebook*), una etapa 2 de 1.000 millones de parámetros (un *upsampler* que expande esos tokens a ocho *codebooks*, aumentando la fidelidad progresivamente — el mismo principio de refinamiento jerárquico grueso→fino visto en `audiolm.py`), un códec neuronal **XCodec Mini** que convierte los tokens finales de vuelta a audio, y **dos decoders separados** para voz e instrumental — permitiendo, tras la generación conjunta, mezclar o remezclar voz e instrumental de forma independiente.

Algoritmos. El comando `generate` opera en modo **CoT** (*Chain-of-Thought*): la generación procede en varios pasos razonados en vez de un único salto directo de letra+género a audio completo — coherente con la tendencia reciente de aplicar razonamiento paso a paso, popularizado originalmente en LLMs de texto, a tareas de generación de contenido más largo y estructuralmente complejo como una canción completa. El modo `icl` (*in-context learning*) extiende `generate` aceptando además un audio de referencia como ejemplo dentro del propio contexto de generación, guiando el estilo del resultado por demostración directa en vez de solo por descripción textual.

Conceptos musicales. La separación de *decoders* de voz e instrumental desde una representación de tokens compartida permite un grado de control post-generación —remezclar el balance voz/instrumental, o incluso sustituir uno de los dos componentes— que un modelo que generara la mezcla completa directamente como una única señal de audio no ofrecería, preservando cierta modularidad de producción incluso dentro de un pipeline de generación end-to-end.